Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación



T E S I S

"Predicción de congestión en redes de computadoras utilizando técnicas de Machine Learning"

Presenta:
Ricardo Valera Martínez

Para obtener el grado de:
Licenciatura en Ingeniería en Ciencias de la Computación

Director: **Mauro Alberto López Muñoz**
Codirector: **Guillermo Colorado Jiménez**

Puebla, Pue., Febrero 2026

Índice general

1. Introducción **10**
- 1.1. Planteamiento del problema . . . 10
- 1.2. Objetivos . . . 12
  - 1.2.1. Objetivo General . . . 12
  - 1.2.2. Objetivos Específicos . . . 12
- 1.3. Justificación . . . 12
- 1.4. Alcances y Limitaciones . . . 14

2. Marco Teórico **16**
- 2.1. Fundamentos de redes de computadoras . . . 16
  - 2.1.1. Capacidad del enlace: ancho de banda, throughput y utilización . . . 17
  - 2.1.2. Modelo OSI y TCP/IP . . . 18
  - 2.1.3. Concepto de congestión . . . 19
  - 2.1.4. Métricas de rendimiento . . . 22
- 2.2. Técnicas de *Machine Learning* aplicables . . . 25
  - 2.2.1. Aprendizaje supervisado: regresión vs. clasificación . . . 25
  - 2.2.2. Modelos de series temporales (ARIMA) . . . 26
  - 2.2.3. Modelos de aprendizaje automático clásico (Random Forest, SVR) . . . 26
  - 2.2.4. Modelos de Aprendizaje Profundo (LSTM, GRU) . . . 27
  - 2.2.5. Métricas de evaluación . . . 28

3. Estado del Arte **30**
- 3.1. Revisión de trabajos sobre predicción de tráfico con métodos estadísticos . . . 31
- 3.2. Predicción de tráfico y congestión con *Machine Learning* clásico . . . 32
- 3.3. Predicción con *Deep Learning* . . . 33
- 3.4. Herramientas de monitoreo y *datasets* utilizados . . . 35
- 3.5. Síntesis del estado del arte y brecha a cubrir . . . 38

4. Metodología **44**
- 4.1. Tipo de investigación . . . 44
- 4.2. Definición operativa de congestión . . . 46
- 4.3. Descripción de las métricas a utilizar . . . 47
- 4.4. Fuente de datos . . . 50
- 4.5. Preprocesamiento de datos . . . 52
- 4.6. Modelos a implementar y justificación de su elección . . . 56

4.7. Métricas de evaluación del modelo 59
4.8. Herramientas y entorno 62

5. Desarrollo e Implementación **65**
5.1. Obtención y exploración de los conjuntos de datos 65
5.2. Preprocesamiento 75
5.3. Construcción del modelo ARIMA 79
5.4. Construcción del modelo *Random Forest* 84
5.5. Construcción del modelo LSTM 91

6. Resultados y Discusión **99**
6.1. Resultados de ARIMA 100
6.2. Resultados de *Random Forest* 105
6.3. Resultados de LSTM 111
6.4. Comparativa de modelos 118
6.5. Discusión 123

7. Conclusiones **128**
7.1. Cumplimiento de objetivos 129
7.2. Contribuciones 132
7.3. Limitaciones del estudio 135
7.4. Trabajo futuro 142

Referencias bibliográficas **144**

A. Código fuente de ejemplos críticos **147**
A.1. Decodificador de trazas libpcap 147
A.2. Script de recolección en RouterOS 149
A.3. Pipeline de preprocesamiento 150
A.4. Generador de datos sintéticos 152
A.5. Entrenamiento y evaluación de modelos 155

Índice de figuras

2.1. Representación de una red entre dos computadoras. 16
2.2. Comparativa entre las capas del modelo OSI y la arquitectura TCP/IP. . . . 19
2.3. Ubicación del problema de predicción de congestión dentro de los paradigmas de *Machine Learning*. 26

3.1. Taxonomía de las técnicas de monitoreo de red empleadas en la literatura revisada. 36
3.2. Línea de tiempo de la evolución metodológica en predicción de tráfico y congestión de red. 40

4.1. *Pipeline* de transformación de métricas crudas en variables de entrada y variable objetivo. 50
4.2. Construcción de secuencias de entrada mediante ventana deslizante para los modelos LSTM/GRU. 55
4.3. División cronológica del *dataset* primario en conjuntos de entrenamiento, validación y prueba. 56
4.4. Diseño experimental general: entrenamiento sobre el *dataset* primario y validación cruzada contra MAWI. 59
4.5. Matriz de confusión para la tarea de clasificación de congestión. 60

5.1. Patrón de tráfico promedio y máximo por hora del día, *dataset* primario. . . 67
5.2. Serie temporal de utilización real, *dataset* primario (21 días). 68
5.3. Distribución de `rx_bps` (escala logarítmica en frecuencia), *dataset* primario. . 68
5.4. Distribución de duración e intensidad de los 126 eventos de congestión sintéticos inyectados. 69
5.5. Ejemplo de un evento de congestión sintético: utilización, RTT y pérdida de paquetes, derivados mediante el modelo de teoría de colas de la Sección 2.1.3. 70
5.6. Distribución de utilización, *dataset* sintético (37 días), contrastada contra $U_{th} = 0{,}80$. 71
5.7. Distribución del factor de utilización, *dataset* piloto MAWI. 74
5.8. Distribución de la utilización normalizada en las cinco particiones del *dataset* primario. 78
5.9. ARIMA(1,1,1): predicción continua vs. valor real, conjunto de prueba. 80
5.10. ARIMA(3,0,3): predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales). 82

5.11. ARIMA(3,0,3): predicción vs. valor real, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 83
5.12. *Random Forest*: predicción continua vs. valor real (escala normalizada), conjunto de prueba. 85
5.13. Importancia relativa de variables (Impureza de Gini), *Random Forest*. 86
5.14. *Random Forest*: predicción vs. valor real, validación de regresión (*dataset* primario, datos reales). 87
5.15. *Random Forest*: predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales). 88
5.16. *Random Forest*: predicción vs. valor real, validación de clasificación (*dataset* primario, datos sintéticos aislados). 89
5.17. *Random Forest*: predicción vs. valor real, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 89
5.18. Importancia relativa de variables, *Random Forest*, *dataset* primario. 90
5.19. Curva de aprendizaje de la red LSTM (pérdida MSE por época). 92
5.20. LSTM: predicción continua vs. valor real, conjunto de prueba. 93
5.21. Curva de aprendizaje de la red LSTM, *dataset* primario. 95
5.22. LSTM: predicción vs. valor real, validación de regresión (*dataset* primario, datos reales). 96
5.23. LSTM: predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales). 96
5.24. LSTM: predicción, validación de clasificación (*dataset* primario, datos sintéticos aislados). 97
5.25. LSTM: predicción, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 98

6.1. ARIMA(3,0,3): predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales). 101
6.2. Comparación de métricas de clasificación, validación vs. prueba, ARIMA sobre el *dataset* primario. 102
6.3. ARIMA(3,0,3): predicción vs. valor real, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 102
6.4. Validación de generalización: ARIMA (*dataset* primario) evaluado sobre MA-WI (1 minuto oficial). 103
6.5. Síntesis comparativa del RMSE de ARIMA a través de los cuatro contextos de evaluación. 104
6.6. *Random Forest*: predicción vs. valor real, validación de regresión (*dataset* primario, datos reales). 106
6.7. *Random Forest*: predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales). 107
6.8. Síntesis comparativa del RMSE de *Random Forest* a través de los contextos de evaluación. 107
6.9. Comparación de métricas de clasificación, validación vs. prueba, *Random Forest* sobre el *dataset* primario. 108

6.10. *Random Forest*: predicción vs. valor real, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 109
6.11. Comparación de la importancia relativa de variables, *Random Forest*: fase piloto (granularidad de 1 segundo) vs. *dataset* primario (granularidad de 1 minuto). 109
6.12. Síntesis comparativa del RMSE de LSTM a través de los contextos de evaluación (escala logarítmica). 113
6.13. LSTM: predicción vs. valor real, validación de regresión (*dataset* primario, datos reales). 113
6.14. LSTM: predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales). 114
6.15. Comparación de métricas de clasificación, validación vs. prueba, LSTM sobre el *dataset* primario. 115
6.16. LSTM: predicción, prueba de clasificación (*dataset* primario, datos sintéticos aislados). 115
6.17. *Recall* de clasificación (prueba) de los tres modelos evaluados sobre el *dataset* primario. 116
6.18. *Trade-off* entre precisión de regresión (RMSE, prueba) y sensibilidad de clasificación (*Recall*, prueba), los tres modelos. 119
6.19. Comparación multidimensional de los tres modelos, *dataset* primario, conjunto de prueba. 120
6.20. Comparativa integral de las cinco métricas de clasificación, conjunto de prueba, tres modelos. 121
6.21. Esquema conceptual de un sistema de alertamiento híbrido, combinando la sensibilidad de detección de *Random Forest* con la precisión de estimación continua de LSTM. 126

Índice de cuadros

2.1. Resumen de métricas de rendimiento de red utilizadas en la investigación. . . 24
2.2. Resumen comparativo de los modelos de predicción considerados en esta tesis. 29

3.1. Comparación de herramientas de monitoreo empleadas en la literatura revisada. 36
3.2. Comparación de datasets empleados en la literatura revisada frente a los datasets adoptados en esta tesis. 37
3.3. Síntesis comparativa de los trabajos revisados en el Estado del Arte. 39
3.4. Posicionamiento metodológico de esta tesis frente a la literatura revisada. . . 42

4.1. Clasificación del tipo de investigación de esta tesis. 46
4.2. Variables de entrada consideradas para los modelos predictivos. 48
4.3. Disponibilidad y método de obtención de métricas según la fuente de datos. . 49
4.4. Contraste del proceso de resampleo entre las dos fuentes de datos empleadas. 54
4.5. Resumen de las etapas de preprocesamiento de datos. 56
4.6. Modelos considerados durante la planeación de esta tesis y justificación de su exclusión del alcance experimental. 58
4.7. Rol de cada métrica de regresión en el análisis comparativo de esta tesis. . . 60
4.8. Síntesis del esquema de evaluación aplicado a los modelos de esta tesis. . . . 62
4.9. Herramientas empleadas para la recolección del *dataset* primario. 63
4.10. Bibliotecas de Python empleadas para la implementación de cada etapa metodológica. 63

5.1. Resumen de interrupciones de conectividad durante la recolección del *dataset* primario. 66
5.2. Estadísticas descriptivas del *dataset* primario (21 días reales, 25 422 observaciones). 67
5.3. Composición final del *dataset* primario tras la incorporación de datos sintéticos. 71
5.4. Resultados de la extracción de telemetría sobre las trazas MAWI procesadas. 73
5.5. Estadísticas descriptivas del *throughput* y utilización derivada, *dataset* piloto MAWI (1 800 observaciones, granularidad de 1 minuto). 74
5.6. Composición de la división cronológica del *dataset* piloto MAWI tras el preprocesamiento. 76
5.7. Sesiones continuas detectadas en el conjunto de entrenamiento del *dataset* primario. 77
5.8. Rango de utilización normalizada por partición, *dataset* primario. 78
5.9. Órdenes ARIMA con menor AIC, *dataset* piloto MAWI. 79

5.10. Métricas de desempeño en regresión de ARIMA(1,1,1), *dataset* piloto MAWI. 80
5.11. Desempeño base de clasificación y alertamiento (TAE) de ARIMA, fase piloto. 81
5.12. Métricas de regresión de ARIMA(3,0,3) sobre el *dataset* primario (escala original de utilización). 82
5.13. Métricas de clasificación de ARIMA(3,0,3), umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados. 83
5.14. Comparación del modelo ARIMA entre la fase piloto y el *dataset* primario. . 84
5.15. Métricas de regresión de *Random Forest* (escala normalizada), *dataset* MAWI. 84
5.16. Desempeño de clasificación y alertamiento (TAE) de *Random Forest*, fase piloto. 86
5.17. Métricas de regresión de *Random Forest* sobre el *dataset* primario (escala normalizada). 87
5.18. Métricas de clasificación de *Random Forest*, umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados. 88
5.19. Importancia relativa de variables, *Random Forest*, *dataset* primario. 90
5.20. Comparación de *Random Forest* entre la fase piloto y el *dataset* primario. . . 91
5.21. Arquitectura algorítmica de la red neuronal recurrente LSTM. 91
5.22. Métricas de regresión de LSTM (escala normalizada), *dataset* piloto MAWI. 92
5.23. Desempeño de clasificación temprana y alertamiento (TAE) de la red LSTM. 93
5.24. Validación de *software*: Evolución operativa de los tres paradigmas evaluados. 94
5.25. Métricas de regresión de LSTM sobre el *dataset* primario (escala normalizada). 95
5.26. Métricas de clasificación de LSTM, umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados. 97
5.27. Comparación de LSTM entre la fase piloto y el *dataset* primario. 98

6.1. Recapitulación: métricas de regresión de ARIMA(1,1,1), fase piloto (Sección 5.3). 100
6.2. Métricas de regresión de ARIMA(3,0,3) sobre el *dataset* primario (escala original de utilización). 100
6.3. Métricas de clasificación de ARIMA(3,0,3), umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados. 101
6.4. Validación de generalización de ARIMA(3,0,3) sobre MAWI (granularidad oficial de 1 minuto). 103
6.5. Síntesis completa de los resultados de ARIMA a través de todos los contextos de evaluación de este trabajo. 104
6.6. Recapitulación: métricas de regresión de *Random Forest* (escala normalizada), fase piloto (Sección 5.4). 105
6.7. Métricas de regresión de *Random Forest* sobre el *dataset* primario (escala normalizada). 106
6.8. Métricas de clasificación de *Random Forest*, umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados. 108
6.9. Síntesis completa de los resultados de *Random Forest* a través de todos los contextos de evaluación. 110
6.10. Recapitulación: métricas de regresión de LSTM (escala normalizada), fase piloto (Sección 5.5). 112
6.11. Métricas de regresión de LSTM sobre el *dataset* primario (escala normalizada). 112

6.12. Métricas de clasificación de LSTM, umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados. 114
6.13. Síntesis completa de los resultados de LSTM a través de todos los contextos de evaluación. 117
6.14. Tabla maestra: totalidad de resultados cuantitativos de los tres modelos evaluados en esta tesis. 118
6.15. Reconciliación del hallazgo central de esta tesis con el marco teórico y metodológico de capítulos anteriores. 124
6.16. Cierre de las brechas del Estado del Arte a la luz de los resultados del Capítulo 6. 126

7.1. Síntesis del cumplimiento de los objetivos de investigación establecidos en la Sección **??**. 132
7.2. Resumen de las limitaciones del estudio, su origen metodológico y su efecto empírico sobre las conclusiones. 136
7.3. Delimitación rigurosa del alcance científico de las conclusiones dictaminadas en esta investigación. 142

DEDICATORIA

A mi padre, ***Ricardo Valera Hernández****, por ser el pilar constante en cada etapa de mi formación, por su paciencia, su apoyo incondicional y por nunca dejar de creer en mí incluso cuando yo dudaba, por su ejemplo de responsabilidad y esfuerzo, por enseñarme que el trabajo honesto y la perseverancia son el camino para alcanzar cualquier meta.*

A mi madre, ***María Margarita Irma Martínez Pérez****, por ser la guía y el apoyo incondicional que hizo posible este trayecto. Gracias por su confianza infinita, por sus palabras que siempre supieron darme fuerza y por estar presente en cada paso de mi formación profesional. Este trabajo es testimonio de su esfuerzo y dedicación constante en mi vida.*

A mi novia, ***Vanessa F. Linarte Castañeda****, por su amor, su comprensión y su compañía en los momentos de cansancio y dificultad, por motivarme a seguir adelante cuando parecía imposible y por ser parte fundamental de este logro.*

A mi hermano menor, ***Emilio Valera Martínez****, por su complicidad y apoyo en los momentos de distracción y de enfoque. Gracias por estar ahí, por los consejos y por ser un compañero constante en este camino.*

Al ***Sr. Héctor H. Linarte*** *y la* ***Sra. Cristina E. Castañeda****, por recibirme en su familia como a un hijo más, por sus palabras de aliento y por el apoyo incondicional brindado durante este proceso. Su confianza y cariño fueron fundamentales para mantener el ánimo en la recta final.*

A mi director de tesis, el ***Mtro. Mauro Alberto López Muñoz****, y a mi codirector, el* ***Mtro. Guillermo Colorado Jiménez****, por su invaluable guía, su paciencia y por compartir su conocimiento para llevar este proyecto a buen puerto.*

Capítulo 1

Introducción

"El futuro de las redes de computadoras no solo depende de transmitir datos más rápido, sino de hacerlo de manera inteligente."

La evolución en las redes de computadoras ha permitido el desarrollo de una gran cantidad de servicios y aplicaciones que forman parte de la vida cotidiana, desde la navegación web y las plataformas de comunicación, hasta los sistemas distribuidos, la computación en la nube y el intercambio masivo de información en tiempo real. Debido a este crecimiento constante en la cantidad de dispositivos conectados y el volumen de tráfico generado, las redes modernas enfrentan cada vez mayores desafíos relacionados con el rendimiento, la disponibilidad y la calidad del servicio.

1.1. Planteamiento del problema

En la era digital actual, la conectividad se ha convertido en un servicio esencial, tan fundamental como la electricidad o el agua corriente. Desde las videollamadas que facilitan la comunicación remota, hasta las plataformas de *streaming* que ofrecen entretenimiento bajo demanda, así como la rápida expansión del Internet de las Cosas (IoT) en hogares y ciudades, gran parte de la vida cotidiana depende de un flujo constante e ininterrumpido de datos. Según el informe *Cisco Annual Internet Report (2018–2023)*, el número de usuarios de Internet a nivel mundial pasó de 3900 millones en 2018 a 5300 millones en 2023, mientras que los dispositivos conectados a redes IP aumentaron de 18 400 millones a 29 300 millones en el mismo período [1].

Las redes de computadoras son el tejido que sostiene la infraestructura tecnológica moderna. Sin embargo, no son un recurso infinito: cada enrutador, cada conmutador y cada enlace de fibra óptica tiene una capacidad limitada. Cuando la demanda de transmisión de datos supera dicha capacidad, se produce un fenómeno conocido como congestión de red. En términos sencillos (cuya definición formal se abordará en secciones posteriores), la congestión es el equivalente digital a un embotellamiento en una autopista: demasiados paquetes de datos intentando circular al mismo tiempo por un canal con aforo máximo. Las consecuencias son inmediatas y perceptibles para el usuario final: congelamiento de videollamadas, lentitud

en la carga de páginas web, interrupción en la descarga de archivos o, en el peor de los casos, la pérdida total de la conexión. En entornos críticos como hospitales, redes financieras o infraestructuras industriales, estos fallos se traducen en pérdidas económicas millonarias e incluso en riesgos para la seguridad humana.

Tradicionalmente, la gestión de redes ha mantenido un enfoque reactivo: los administradores monitorizan los equipos y, cuando detectan que un enlace está saturado, toman medidas correctivas como redirigir el tráfico, limitar ciertas aplicaciones o aumentar la capacidad del enlace. Este modo de actuar, aunque ha resultado funcional durante décadas, presenta una limitación fundamental: actúa después de que el problema ha comenzado a degradar la calidad del servicio. En escenarios de latencia crítica, como la operación de un coche autónomo que necesita tomar decisiones en milisegundos o un procedimiento quirúrgico a distancia mediante robótica, reaccionar a posteriori no es una opción viable; la anticipación resulta indispensable.

De lo anterior se desprende el problema central que da origen a esta investigación: los mecanismos actuales de gestión de redes de computadoras carecen, en su mayoría, de la capacidad de anticipar la congestión antes de que esta se manifieste, limitándose a detectarla y corregirla una vez iniciada la degradación del servicio. Esta naturaleza reactiva impide a los administradores de red y a los sistemas automatizados de gestión tomar decisiones oportunas —como redirigir tráfico, priorizar aplicaciones críticas o escalar recursos— antes de que el impacto sea perceptible. El problema de investigación se formula, entonces, de la siguiente manera: **¿es posible, a partir del análisis de datos históricos de operación de una red, construir un modelo capaz de predecir con suficiente antelación que un enlace alcanzará un estado de congestión, de tal forma que dicha predicción resulte útil para la toma de decisiones preventivas?**

En este contexto, las técnicas de Machine Learning (ML) se presentan como una vía prometedora. El ML, como rama de la inteligencia artificial, permite a los sistemas informáticos identificar patrones a partir de datos históricos y realizar predicciones sin requerir una programación explícita para cada escenario, lo que lo convierte en una herramienta natural para abordar la necesidad de anticipación descrita.

El presente trabajo de tesis se sitúa en la intersección entre las redes de computadoras y el aprendizaje automático. El objetivo central es desarrollar un sistema basado en técnicas de Machine Learning capaz de predecir la congestión en un enlace de red mediante el análisis de métricas de tráfico obtenidas de forma híbrida (activa y pasiva). En lugar de mantener un enfoque reactivo frente a niveles críticos de utilización, se busca construir un modelo que permita emitir alertas tempranas sobre la saturación inminente de un enlace, facilitando la toma de medidas preventivas antes de que la calidad del servicio se vea comprometida.

1.2. Objetivos

A continuación se detallan los objetivos que guían la presente investigación, los cuales establecen las directrices fundamentales para el desarrollo e implementación del sistema propuesto.

1.2.1. Objetivo General

Desarrollar un sistema predictivo basado en Machine Learning capaz de anticipar la congestión en enlaces de red a partir del análisis de datos operativos, evaluando su desempeño en escenarios de tráfico de distinta escala y naturaleza, con el fin de aportar evidencia sobre la viabilidad de este enfoque para optimizar la gestión de recursos y la calidad del servicio.

1.2.2. Objetivos Específicos

- Recopilar y organizar un conjunto de datos (dataset) operativos de una red de computadoras, abarcando parámetros clave de telemetría como ancho de banda, rendimiento, disponibilidad y métricas vinculadas a la transmisión de datos.
- Diseñar un procedimiento de preprocesamiento para limpiar, normalizar y estructurar los datos obtenidos, garantizando su idoneidad para el entrenamiento algorítmico.
- Implementar, entrenar y comparar distintos modelos de Machine Learning (tales como enfoques de series temporales, árboles de decisión o redes neuronales) orientados al análisis de patrones de tráfico.
- Construir un prototipo funcional que integre el modelo de mejor rendimiento para generar predicciones y alertas tempranas sobre posibles escenarios de congestión.
- Evaluar el desempeño y la precisión del sistema desarrollado mediante métricas de validación estándar propias de los modelos predictivos.
- Determinar y documentar la aplicabilidad del modelo propuesto en entornos reales para respaldar la toma de decisiones preventivas en la administración de redes.

1.3. Justificación

El crecimiento exponencial del tráfico en las redes de computadoras ha superado las estrategias tradicionales de gestión basadas en la monitorización reactiva. En la actualidad, la demanda de ancho de banda generada por servicios de videoconferencia, plataformas de *streaming*, aplicaciones en la nube y dispositivos del Internet de las Cosas (IoT) somete a las infraestructuras a picos de utilización impredecibles. En este escenario, los administradores se ven forzados a intervenir únicamente después de que los niveles de congestión ya han degradado la calidad del servicio, lo que deriva en una experiencia deficiente para los usuarios y, en ámbitos críticos, en pérdidas económicas o riesgos de seguridad. Por consiguiente, resulta indispensable desarrollar herramientas que permitan anticipar la saturación de los enlaces,

migrando de una operación reactiva a una gestión predictiva que tome decisiones antes de que la congestión se materialice.

La aplicación de técnicas de *Machine Learning* en el ámbito de las redes abre una oportunidad real para este cambio de paradigma [2]. A diferencia de los métodos estadísticos convencionales, los algoritmos de aprendizaje automático tienen la capacidad de descubrir patrones complejos y no lineales a partir de grandes volúmenes de datos operativos, como series temporales de utilización, latencia o pérdida de paquetes. Modelos como las redes neuronales recurrentes, los bosques aleatorios o las arquitecturas de aprendizaje profundo han demostrado un desempeño notable en la predicción de tráfico en otros contextos, pero su aplicación sistemática para anticipar la congestión en enlaces individuales constituye un área de investigación con vacíos aún por cubrir, particularmente en lo referente a la comparación de múltiples familias de modelos en condiciones realistas de operación.

En el ámbito práctico, un sistema que prediga con suficiente antelación que un enlace alcanzará su límite de capacidad puede transformar la administración de redes. Por ejemplo, un administrador podría redirigir flujos de tráfico, activar mecanismos de balanceo de carga, reducir la prioridad de aplicaciones no críticas o incluso aprovisionar recursos adicionales de forma automática antes de que el usuario perciba degradación alguna. Esta capacidad de anticipación resulta crucial en entornos donde la continuidad del servicio es prioritaria, tales como centros hospitalarios que dependen de la telemedicina, entidades financieras con transacciones en tiempo real o plantas industriales con procesos automatizados. De esta manera, la presente investigación no solo busca generar conocimiento teórico, sino también ofrecer una herramienta con un impacto directo en la fiabilidad y eficiencia de las infraestructuras digitales.

Desde el punto de vista metodológico, la propuesta de construir un modelo predictivo basado en datos de operación reales y evaluarlo mediante métricas de error rigurosas —como el Error Cuadrático Medio (MSE), el Error Absoluto Medio (MAE) y la comparación de tasas de predicción en distintos horizontes temporales— aporta un procedimiento replicable para la comunidad académica y profesional. La selección de tres paradigmas de modelado distintos (series temporales estadísticas, aprendizaje automático clásico y aprendizaje profundo) permite, además, trazar un mapa claro de las fortalezas y limitaciones de cada enfoque, ofreciendo criterios objetivos para futuras implementaciones. En un campo donde la mayoría de los estudios se concentran en la predicción general del volumen de tráfico, esta tesis se distingue por centrarse en la detección temprana de un evento específico y operacionalmente crítico: la congestión.

Finalmente, la investigación se justifica por su alineación con las tendencias actuales de la industria hacia las redes autónomas y la inteligencia artificial aplicada a las operaciones de TI (AIOps). Diversas empresas del sector tecnológico y organismos de estandarización impulsan actualmente la incorporación de modelos predictivos en los sistemas de gestión de red, reconociendo que la supervisión tradicional resulta insuficiente para soportar las exigentes demandas de latencia y disponibilidad de las aplicaciones modernas. En este sentido, el trabajo propuesto contribuye a cerrar la brecha entre las capacidades teóricas del aprendizaje automático y su aplicación práctica en un problema real de ingeniería de redes, generando evidencia empírica sobre su viabilidad, precisión y pertinencia.

1.4. Alcances y Limitaciones

El presente trabajo de tesis se circunscribe al desarrollo de un prototipo funcional de predicción de congestión, implementado enteramente en código y orientado a validar la viabilidad utilizando un conjunto de datos de operación de red. El alcance del proyecto se limita a la construcción, entrenamiento y evaluación de estos modelos mediante métricas cuantitativas, sin comprometerse con el desarrollo de una interfaz gráfica de usuario, una aplicación empaquetada o un sistema de producción completo. Por lo tanto, los resultados deben interpretarse como una prueba de concepto que demuestra la capacidad predictiva de los algoritmos seleccionados, y no como una solución lista para ser desplegada en un entorno operacional real.

En términos de datos, el prototipo se desarrolla y evalúa sobre dos fuentes complementarias: una red privada de escala reducida sobre la cual se realizó recolección propia de datos de operación durante un periodo de 30 días, y un archivo académico de tráfico de Internet de gran escala empleado como conjunto de validación externa. Si bien esta combinación permite evaluar el desempeño de los modelos bajo condiciones de tráfico de naturaleza y escala distintas —ampliando el alcance de las conclusiones respecto a un estudio basado en una única fuente—, no debe interpretarse como una validación exhaustiva sobre la totalidad de topologías, tecnologías de acceso o patrones de tráfico posibles en una red de computadoras. La generalización de los resultados a escenarios sustancialmente distintos a los aquí evaluados (por ejemplo, redes móviles, entornos industriales o infraestructuras de centros de datos) queda fuera del alcance de este trabajo y se plantea como una línea de investigación futura.

Desde el punto de vista computacional, se ha establecido un límite práctico en la complejidad de los algoritmos y en los recursos de hardware empleados. El prototipo se ejecuta en un entorno de estación de trabajo convencional, sin recurrir a infraestructura de cómputo distribuido o aceleración especializada (GPU de alto rendimiento). En consecuencia, los modelos profundos se configuran con un número acotado de capas y neuronas, y no se explora el ajuste exhaustivo de hiperparámetros mediante búsqueda en malla de gran tamaño. Este enfoque permite obtener resultados en tiempos razonables, pero sacrifica potencialmente la obtención del modelo óptimo absoluto, lo cual es una limitación deliberada acorde con el carácter prototípico del desarrollo.

Otra restricción relevante es que el sistema no opera en tiempo real sobre una red viva, sino que se entrena y evalúa sobre trazas históricas. La predicción se realiza fuera de línea, asumiendo que los datos han sido previamente recolectados y preprocesados. Por tanto, no se han implementado mecanismos de ingestión continua de datos, detección de deriva de concepto (*concept drift*) ni reentrenamiento automático del modelo, aspectos que resultarían necesarios para una eventual transición a un sistema de alerta temprana en producción. Esta limitación es congruente con la elección de priorizar la comparación de modelos bajo condiciones controladas [3].

Finalmente, es imperativo distinguir entre la variable objetivo del sistema y sus variables predictoras. Para fines de etiquetado y evaluación, el estado de congestión se define y cuantifica exclusivamente a partir del factor de utilización del enlace (variable objetivo). Sin embargo, el prototipo predictivo sí incorpora métricas de red complementarias —como estimaciones de rendimiento y disponibilidad obtenidas de forma pasiva— como variables de entrada (*features*) para el entrenamiento de los algoritmos. Esta delimitación conceptual per-

mite definir un umbral matemático claro para la congestión, reconociendo simultáneamente que el modelo requiere de una perspectiva multivariable para capturar los patrones complejos que preceden a la saturación del enlace.

Capítulo 2

Marco Teórico

2.1. Fundamentos de redes de computadoras

Considérese un escenario con dos computadoras aisladas, cada una con información valiosa almacenada en su interior. Para que esos datos puedan ser aprovechados por el otro equipo, un usuario debe copiarlos en un dispositivo de almacenamiento externo, trasladarlos físicamente e introducirlos en la segunda máquina. Este procedimiento, aunque funcional, es lento, propenso a errores humanos y completamente inviable cuando la cantidad de información crece o cuando los dispositivos se encuentran a cientos o miles de kilómetros de distancia. Precisamente para resolver esta limitación nacieron las redes de computadoras: sistemas que conectan dos o más dispositivos de cómputo mediante algún medio de transmisión —cables de cobre, fibra óptica o el espectro electromagnético— con el propósito de intercambiar datos de manera rápida, confiable y eficiente [4].

En esencia, una red de computadoras es un conjunto de nodos autónomos interconectados que cooperan para compartir recursos e información. El término *nodo* designa a cualquier dispositivo capaz de originar, recibir o retransmitir datos: computadoras personales, servidores, teléfonos móviles, sensores, impresoras e incluso electrodomésticos inteligentes. Estos nodos se comunican a través de enlaces, que son los caminos físicos o inalámbricos por los que viaja la información. A su vez, los enlaces se conectan entre sí por medio de dispositivos de interconexión como conmutadores y enrutadores, cuya misión es encaminar los datos desde su origen hasta su destino, eligiendo la ruta más adecuada en cada momento y gestionando las colas de espera que se forman cuando varios flujos de tráfico coinciden en un mismo punto. El comportamiento de estas colas de espera bajo condiciones de alta demanda es, como se detallará en la sección 2.1.3, el origen mismo del fenómeno de congestión.

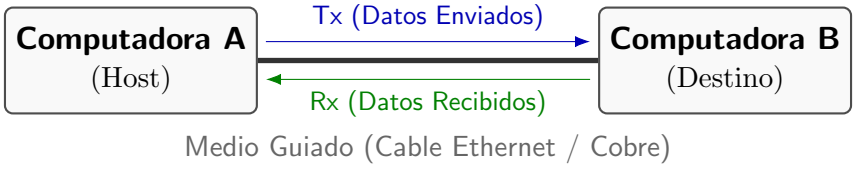


Figura 2.1: Representación de una red entre dos computadoras.

El funcionamiento de una red se organiza de manera jerárquica para gestionar la complejidad inherente a la comunicación digital. Esta organización se plasma en los llamados modelos de referencia, que dividen el problema de transmitir información en varias capas superpuestas, cada una con responsabilidades claramente acotadas: desde los aspectos puramente físicos de la transmisión (voltajes, pulsos de luz, ondas de radio) hasta la interacción con las aplicaciones que el usuario ve en pantalla. Esta arquitectura en capas permite que un fabricante mejore un componente sin necesidad de rediseñar toda la red, y que tecnologías muy dispares puedan interoperar siempre que respeten los protocolos definidos en cada nivel. El modelo de referencia más utilizado para describir esta organización, el modelo OSI, se desarrolla con mayor profundidad en la sección 2.1.2.

Más allá de cómo se organizan las capas, existe una decisión de diseño fundamental que determina cómo viajan realmente los datos a través de una red: el paradigma de conmutación. En las redes telefónicas tradicionales se empleaba la **conmutación de circuitos**: cuando dos personas iniciaban una llamada, la red reservaba un camino físico dedicado y exclusivo entre ambos extremos durante toda la conversación. Este enfoque garantiza una calidad de servicio constante y predecible, pero resulta profundamente ineficiente desde el punto de vista de la utilización de recursos: si los interlocutores hacen una pausa de diez segundos, el circuito reservado permanece completamente ocioso y ningún otro usuario puede aprovecharlo, incluso si la red entera está saturada de solicitudes en ese instante.

Las redes de computadoras modernas, incluida Internet, optaron por un modelo radicalmente distinto: la **conmutación de paquetes**. En este paradigma, los datos se fragmentan en pequeñas unidades llamadas paquetes o datagramas, cada una encabezada por información de control (dirección de origen, destino, número de secuencia, entre otros campos). Cada paquete viaja de nodo en nodo de manera independiente y sin reservar recursos exclusivos, compartiendo dinámicamente los enlaces con paquetes pertenecientes a otras comunicaciones completamente distintas. Cuando un remitente hace una pausa en su transmisión, otros flujos aprovechan inmediatamente el enlace liberado, lo que multiplica la eficiencia global del sistema comparado con la conmutación de circuitos [5].

Esta compartición estadística de los recursos es, precisamente, la razón por la que Internet puede ofrecer conectividad a miles de millones de dispositivos con una infraestructura física finita. Sin embargo, tiene una contrapartida inevitable: como ningún flujo individual tiene garantizado un ancho de banda exclusivo, si demasiados remitentes transmiten de manera simultánea a través del mismo enlace, la demanda agregada puede superar la capacidad disponible. Es precisamente en este punto —cuando la suma de los flujos que compiten por un recurso compartido excede lo que ese recurso puede entregar— donde se origina el fenómeno que esta investigación busca predecir: la congestión de red, que se define y caracteriza formalmente en las siguientes secciones.

2.1.1. Capacidad del enlace: ancho de banda, throughput y utilización

Para poder cuantificar cuándo un enlace se encuentra cerca de su límite, es necesario introducir tres métricas básicas que se emplearán a lo largo del presente trabajo. Cada enlace en una red posee una capacidad máxima teórica de transmisión, denominada **ancho de banda** (*bandwidth*), que se mide en bits por segundo (bps). Un enlace de fibra óptica típico

puede ofrecer 1 Gbps, mientras que una conexión inalámbrica doméstica quizás solo alcance 50 Mbps. Esta cifra representa el límite superior teórico del enlace, no la cantidad de datos que efectivamente circula por él en un momento dado.

El tráfico real que atraviesa el enlace en un instante determinado se conoce como ***throughput*** o rendimiento efectivo, y su valor siempre es inferior o igual al ancho de banda nominal, ya que factores como la sobrecarga de los encabezados de protocolo, las retransmisiones y el propio comportamiento del tráfico impiden alcanzar el máximo teórico de manera sostenida. La razón entre el *throughput* observado y el ancho de banda disponible se denomina **factor de utilización**, y constituye una de las métricas más reveladoras sobre el estado de salud de un enlace:

$$U = \frac{\text{Throughput observado}}{\text{Ancho de banda nominal}} \tag{2.1}$$

Mientras el factor de utilización se mantenga en niveles moderados, la red opera con holgura: existe margen suficiente para absorber ráfagas de tráfico sin que se produzcan retrasos perceptibles. Conforme U se aproxima a 1 (es decir, al 100 % de la capacidad), comienzan a manifestarse los síntomas característicos de la congestión, los cuales se explican en detalle a continuación.

2.1.2. Modelo OSI y TCP/IP

El Modelo de Interconexión de Sistemas Abiertos (OSI, *Open Systems Interconnection*), estandarizado por la Organización Internacional de Normalización (ISO) en 1984, constituye un marco conceptual que divide el proceso de comunicación en redes en siete capas funcionales independientes [4]. Cada nivel abstrae un conjunto específico de responsabilidades, desde la transmisión física de bits hasta la interacción con las aplicaciones del usuario. Esta estratificación facilita la interoperabilidad entre equipos de distintos fabricantes y permite analizar los fenómenos de red, como la congestión, dentro de una estructura bien definida. En particular, la gestión de la congestión incide directamente en las capas de transporte y red, donde se regulan el control de flujo, el enrutamiento y la asignación de recursos; aspectos que serán modelados en este trabajo a partir de métricas de utilización.

Aunque el modelo OSI es el estándar de referencia teórica, en la práctica, las redes modernas operan bajo la arquitectura TCP/IP (*Transmission Control Protocol/Internet Protocol*). Este modelo condensa las siete capas de OSI en cuatro niveles prácticos: Acceso a la Red, Internet, Transporte y Aplicación. Para efectos de esta investigación, la telemetría recolectada en los enlaces —como el *throughput* y el factor de utilización— se sustenta en los protocolos del modelo TCP/IP (tales como IP en la capa de red y TCP/UDP en la capa de transporte), si bien el análisis subyacente del fenómeno de congestión sigue los principios teóricos establecidos por OSI.

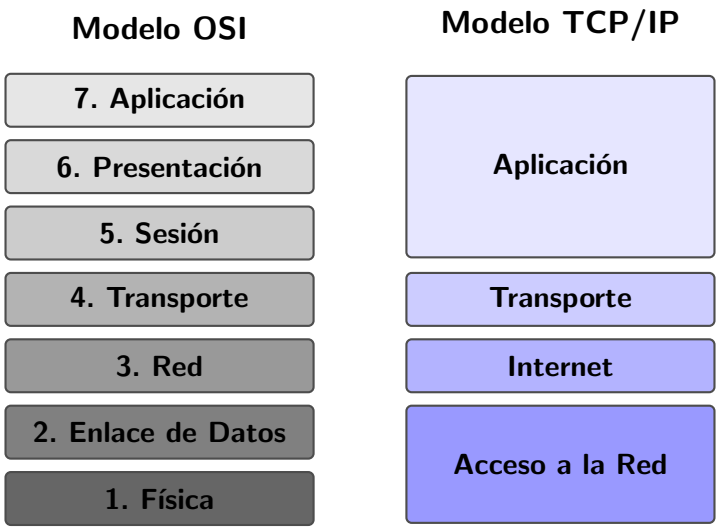


Figura 2.2: Comparativa entre las capas del modelo OSI y la arquitectura TCP/IP.

2.1.3. Concepto de congestión

Con las nociones de ancho de banda, *throughput* y utilización ya establecidas, es posible caracterizar con precisión qué ocurre dentro de una red cuando la demanda de tráfico se aproxima o supera la capacidad disponible de un enlace. A este fenómeno se le denomina **congestión de red**, y constituye el objeto central de estudio de esta investigación.

Qué ocurre dentro de un enrutador

Para entender el origen físico de la congestión es necesario examinar qué sucede dentro de un enrutador o conmutador cuando procesa tráfico. Cada dispositivo de interconexión dispone de pequeñas memorias temporales llamadas ***buffers***, donde se almacenan los paquetes que arriban mientras esperan ser procesados y reenviados por el enlace de salida correspondiente. Si los paquetes llegan a una tasa inferior a la capacidad del enlace de salida, el *buffer* apenas se ocupa y los paquetes son despachados casi de inmediato, con un tiempo de espera mínimo.

El problema surge cuando, durante un intervalo sostenido, la tasa de llegada de paquetes supera la tasa a la que el enlace de salida puede transmitirlos. En este escenario, la cola de espera dentro del *buffer* comienza a crecer progresivamente. Este fenómeno es análogo a lo que ocurre en una caseta de cobro de autopista: mientras los vehículos lleguen a un ritmo menor al que la caseta puede atender, la fila permanece corta y el retraso es mínimo; pero si llegan más vehículos de los que la infraestructura puede procesar, la fila se alarga sin control y el tiempo de espera se incrementa de forma no lineal. Es importante subrayar este último comportamiento: el retraso generado por la congestión **no crece de manera proporcional** al exceso de tráfico, sino que tiende a dispararse abruptamente una vez que se cruza cierto umbral de utilización, un fenómeno exhaustivamente documentado en la teoría de colas aplicada a redes de datos [6].

Latencia y sus componentes

El tiempo total que un paquete tarda en viajar desde que ingresa a la red hasta que llega a su destino final se conoce como **latencia** o retardo extremo a extremo, y se compone de cuatro elementos fundamentales:

- **Tiempo de propagación**: el tiempo que tarda la señal física en recorrer el medio de transmisión (cable, fibra, espectro electromagnético), determinado por la distancia y la velocidad de propagación del medio.
- **Tiempo de transmisión**: el tiempo que tarda el nodo emisor en colocar todos los bits del paquete sobre el enlace físico, el cual depende del tamaño del paquete y del ancho de banda disponible.
- **Tiempo de procesamiento**: el tiempo que cada nodo intermedio emplea en examinar el encabezado del paquete y determinar la ruta de reenvío.
- **Tiempo de encolado**: el tiempo que el paquete permanece en espera dentro del *buffer* de un dispositivo antes de ser transmitido.

En condiciones normales de operación, el tiempo de encolado es prácticamente insignificante frente a los otros tres componentes. Sin embargo, bajo condiciones de congestión, este componente suele convertirse en el factor dominante de la latencia total, siendo el responsable principal del retraso que experimenta el usuario.

Jitter: la variabilidad del retraso

Dado que la llegada de paquetes a un enrutador es irregular e impredecible por naturaleza, el tamaño de la cola de espera fluctúa constantemente a lo largo del tiempo. Como consecuencia, la latencia experimentada por paquetes consecutivos de un mismo flujo no es constante, sino que varía de un paquete a otro. Esta variabilidad en el retardo se denomina ***jitter***, y resulta particularmente perjudicial para aplicaciones sensibles al tiempo que requieren una entrega de datos a ritmo constante (como videollamadas, juegos en línea o transmisiones de audio y video en tiempo real), donde un *jitter* elevado se traduce en cortes, distorsiones o congelamiento de la imagen, incluso cuando la latencia promedio parece aceptable.

Pérdida de paquetes y la respuesta reactiva de TCP

Cuando la congestión se agrava lo suficiente, el *buffer* de un enrutador puede llenarse por completo. Llegado a este límite, el dispositivo no tiene más opción que descartar los paquetes entrantes que no caben en la memoria disponible, evento conocido como **pérdida de paquetes**. Una tasa de pérdida ocasional puede ser mitigada o pasar desapercibida, pero al volverse frecuente, degrada severamente la calidad de la comunicación.

Es en este escenario donde interviene el comportamiento de los protocolos de transporte, particularmente TCP. Este protocolo interpreta la pérdida de paquetes como una señal indirecta de congestión en la red, y en respuesta reduce drásticamente su tasa de envío mediante mecanismos de control de congestión [7], para posteriormente incrementarla de manera gradual mientras no detecte nuevas pérdidas. Este mecanismo, formalizado en estándares como

el RFC 2914 [8], ha sido fundamental para preservar la estabilidad de Internet durante décadas. No obstante, es crucial notar su naturaleza intrínsecamente **reactiva**: el mecanismo se activa únicamente *después* de que la congestión ha provocado pérdidas reales, momento en el cual la calidad del servicio se ha degradado y el problema ya es perceptible. Esta limitación reactiva es la motivación central que impulsa el enfoque predictivo propuesto en esta tesis.

El cuello de botella

En cualquier ruta que conecte dos nodos de una red a través de múltiples enlaces intermedios, existe un concepto estructural determinante: el **cuello de botella** (*bottleneck*). Se trata del enlace con la menor capacidad disponible a lo largo de toda la ruta, el cual establece la velocidad máxima teórica a la que pueden fluir los datos de extremo a extremo. De manera análoga al tramo más estrecho de una tubería que limita el caudal total de agua, el cuello de botella es, por definición, el punto donde la congestión se manifiesta primero y con mayor intensidad. Por consiguiente, estos enlaces son los candidatos naturales para la aplicación de técnicas de predicción y prevención.

Definición operativa de congestión

A partir de los elementos anteriores (utilización, *buffers*, latencia, *jitter*, pérdida de paquetes y cuellos de botella), es posible formular una definición precisa y *medible* de congestión, la cual se adoptará de manera consistente a lo largo del desarrollo metodológico y experimental del presente trabajo.

Se establece que un enlace se encuentra en estado de **congestión** cuando su factor de utilización U (Ecuación 2.1) supera un umbral crítico U_{th} durante un intervalo de tiempo sostenido, punto a partir del cual el comportamiento del sistema deja de ser lineal y comienzan a manifestarse los síntomas de degradación: crecimiento acelerado del tiempo de encolado, incremento del *jitter* y pérdida de paquetes. Formalmente:

$$\text{Congestión} \iff U(t) \geq U_{th} \quad \text{durante un intervalo } [t, t + \Delta t] \tag{2.2}$$

La determinación del umbral crítico U_{th} y la ventana de observación Δt no se establecerá de manera arbitraria. Para garantizar el rigor empírico, el valor de U_{th} se derivará en el Capítulo 4 a partir de un análisis estadístico sobre el conjunto de datos operativos, identificando el punto de inflexión en la distribución (por ejemplo, mediante el percentil 85 o 90) donde la métrica de utilización comienza a correlacionarse con un incremento exponencial en las fluctuaciones de la red. De igual forma, el intervalo Δt estará dictado por la granularidad de muestreo soportada por las herramientas de instrumentación pasiva empleadas durante la recolección de las trazas de 30 días.

Esta definición operativa presenta dos ventajas fundamentales. En primer lugar, se apoya en una métrica directamente observable y de fácil recolección pasiva (el factor de utilización), a diferencia del *jitter* o la pérdida de paquetes, que requieren instrumentación invasiva. En segundo lugar, al establecer un umbral cuantitativo, permite convertir la detección de congestión en un problema formal de aprendizaje automático. De este modo, el problema puede abordarse como una tarea de **regresión** (predecir el valor futuro de $U(t)$) o como una tarea de **clasificación binaria** (predecir si $U(t)$ superará o no el umbral U_{th} en un horizonte temporal

determinado); ambos enfoques se explorarán y compararán en los capítulos metodológico y experimental.

Es importante reconocer que esta definición constituye una simplificación deliberada de un fenómeno inherentemente multifactorial. Como se estableció en la sección de alcances y limitaciones, variables como el *jitter* o la tasa de pérdida no son modeladas directamente por el prototipo, aunque su comprensión teórica, expuesta en esta sección, resulta indispensable para justificar por qué el factor de utilización constituye un indicador robusto y suficientemente informativo para anticipar la saturación de un enlace.

2.1.4. Métricas de rendimiento

En la sección anterior se explicó qué son la utilización, la latencia, el *jitter* y la pérdida de paquetes, y por qué se manifiestan como consecuencia de la congestión. Corresponde ahora precisar cómo se cuantifica formalmente cada una de estas métricas, qué unidades emplean y mediante qué mecanismos se obtienen en la práctica, ya que estas definiciones constituirán el conjunto de variables de entrada (*features*) que alimentarán los algoritmos de Machine Learning propuestos en esta tesis.

Utilización del enlace

Como se estableció en la Ecuación 2.1, la utilización U es la razón entre el *throughput* observado y el ancho de banda nominal del enlace. En términos de instrumentación, esta métrica se obtiene típicamente mediante el protocolo SNMP (*Simple Network Management Protocol*), consultando periódicamente los contadores de octetos entrantes y salientes de una interfaz de red (`ifInOctets` e `ifOutOctets` de la MIB-II) [4]. Dado que estos contadores son acumulativos, el *throughput* en un intervalo $[t_1, t_2]$ se calcula como la diferencia entre lecturas consecutivas dividida entre el tiempo transcurrido:

$$\text{Throughput}(t_1, t_2) = \frac{8 \times (\text{Octetos}(t_2) - \text{Octetos}(t_1))}{t_2 - t_1} \tag{2.3}$$

donde el factor 8 convierte de bytes a bits, ya que el ancho de banda se expresa convencionalmente en bits por segundo (bps). La granularidad del muestreo (por ejemplo, cada 30 segundos, 1 minuto o 5 minutos) determina la resolución temporal con la que posteriormente se podrá detectar y predecir la congestión, y constituye una decisión metodológica que se abordará formalmente en el Capítulo 4.

Latencia (RTT)

La latencia de ida y vuelta, o *Round-Trip Time* (RTT), es el tiempo que transcurre entre el envío de un paquete y la recepción de su respuesta. Se mide de forma activa mediante herramientas de diagnóstico (que emplean paquetes ICMP *Echo Request/Reply*) o mediante el análisis de las marcas de tiempo (*timestamps*) del protocolo TCP durante el establecimiento de una conexión (*three-way handshake*) [5]. Formalmente:

$$\text{RTT} = t_{\text{recepción}} - t_{\text{envío}} \tag{2.4}$$

En la práctica no se trabaja con una sola medición de RTT, sino con una serie de mediciones sucesivas, ya que es precisamente la variabilidad entre estas mediciones consecutivas la que da origen al *jitter*.

Jitter

A diferencia de la explicación conceptual ofrecida en la sección anterior, aquí se presenta la definición formal adoptada por el estándar RFC 3550, que especifica el protocolo RTP (*Real-time Transport Protocol*) utilizado ampliamente en aplicaciones de voz y video sobre IP [9]. Dicho estándar define el *jitter* como la variación estadística en el tiempo de llegada entre paquetes consecutivos, y propone un estimador de primer orden que se actualiza de forma incremental con cada nuevo paquete recibido:

$$J(i) = J(i-1) + \frac{|D(i-1,i)| - J(i-1)}{16} \tag{2.5}$$

donde $D(i-1,i)$ representa la diferencia en los tiempos de tránsito relativos entre dos paquetes consecutivos $i-1$ e i, calculada como:

$$D(i-1,i) = (R_i - R_{i-1}) - (S_i - S_{i-1}) \tag{2.6}$$

siendo R_i el instante de llegada del paquete i en el receptor y S_i la marca de tiempo con la que dicho paquete fue enviado por el emisor. El factor $1/16$ actúa como una ganancia de suavizado que proporciona un estimador con un buen equilibrio entre estabilidad estadística y capacidad de reacción ante cambios súbitos, evitando que picos aislados distorsionen la medición general [9].

Pérdida de paquetes

La tasa de pérdida de paquetes (*packet loss rate*) se expresa como el porcentaje de paquetes enviados que nunca llegan a su destino, respecto del total de paquetes transmitidos en un intervalo de observación:

$$P_{\text{loss}} = \frac{\text{Paquetes enviados} - \text{Paquetes recibidos}}{\text{Paquetes enviados}} \times 100\,\% \tag{2.7}$$

Esta métrica puede obtenerse de dos maneras. La primera es de forma **activa**, inyectando tráfico de prueba (por ejemplo, mediante `iperf` o ráfagas ICMP) y contabilizando las respuestas. La segunda es de forma **pasiva**, consultando los contadores de descarte que los propios dispositivos exponen a través de SNMP (como `ifInDiscards` o `ifOutDiscards`), sin generar tráfico adicional que pudiera contribuir a la propia congestión [4]. Dado el planteamiento de recolección híbrida de esta tesis, la aproximación pasiva resulta prioritaria para la pérdida de paquetes, mientras que los métodos activos se reservan para mediciones controladas de latencia.

Throughput vs. Goodput

Conviene distinguir con precisión entre dos términos que suelen confundirse. El ***throughput***, definido en la Ecuación 2.3, contabiliza la totalidad de los bits que atraviesan el enlace

en un intervalo de tiempo, incluyendo los encabezados de los distintos protocolos (Ethernet, IP, TCP/UDP) y cualquier paquete retransmitido debido a pérdidas previas. El ***goodput***, en cambio, contabiliza únicamente los bits de datos útiles a nivel de aplicación que fueron entregados exitosamente, excluyendo dicha sobrecarga (*overhead*) de protocolo:

$$\text{Goodput} = \frac{\text{Bits útiles entregados a la capa de aplicación}}{\text{Tiempo total de transmisión}} \tag{2.8}$$

Bajo condiciones de congestión severa, es posible observar un *throughput* aparentemente estable mientras el *goodput* cae drásticamente, ya que una proporción creciente del tráfico corresponde a retransmisiones de paquetes perdidos. Esta distinción es relevante porque un modelo que observe únicamente el *throughput* podría no detectar que la red ya opera de manera ineficiente.

Disponibilidad

Finalmente, la **disponibilidad** (*availability*) de un enlace se define como la proporción de tiempo durante la cual dicho recurso se encuentra operativo y accesible, respecto del tiempo total de observación:

$$\text{Disponibilidad} = \frac{\text{Tiempo operativo}}{\text{Tiempo operativo} + \text{Tiempo de inactividad}} \times 100\,\% \tag{2.9}$$

Si bien la disponibilidad se asocia tradicionalmente con fallos de hardware o interrupciones del servicio, una congestión suficientemente severa y sostenida puede degradar la calidad de la red hasta un punto en que resulta indistinguible de una interrupción total para el usuario final. Por esta razón, y dado que fue contemplada en los objetivos específicos, se incorpora como una métrica complementaria de evaluación.

Síntesis de métricas

La Tabla 2.1 sintetiza las métricas descritas, sus unidades y su método de obtención, sentando las bases analíticas del conjunto de datos que se procesará en el Capítulo 4.

Métrica	**Unidad**	**Ecuación**	**Método de obtención**
Utilización	Adimensional (%)	2.1	SNMP (pasivo)
Throughput	bps	2.3	SNMP (pasivo)
Latencia (RTT)	ms	2.4	ICMP / TCP timestamps (activo)
Jitter	ms	2.5	RTCP / estimación RFC 3550
Pérdida de paquetes	%	2.7	SNMP (pasivo) / *probes* (activo)
Goodput	bps	2.8	Derivado de *throughput*
Disponibilidad	%	2.9	Monitoreo continuo (híbrido)

Cuadro 2.1: Resumen de métricas de rendimiento de red utilizadas en la investigación.

2.2. Técnicas de *Machine Learning* aplicables

Una vez establecidos los fundamentos de redes de computadoras y caracterizado formalmente el fenómeno de la congestión, corresponde presentar el conjunto de herramientas metodológicas que esta tesis emplea para abordar el problema de predicción planteado: las técnicas de *Machine Learning* (ML). Esta sección no pretende ser un tratado exhaustivo de aprendizaje automático, sino una revisión dirigida y suficiente de los conceptos, paradigmas y modelos específicos pertinentes para la tarea de predecir series temporales de utilización de red, los cuales serán implementados y evaluados en los capítulos posteriores.

2.2.1. Aprendizaje supervisado: regresión vs. clasificación

El aprendizaje automático se organiza tradicionalmente en tres grandes paradigmas: el aprendizaje supervisado (donde el modelo aprende a partir de ejemplos previamente etiquetados), el aprendizaje no supervisado (donde se buscan patrones ocultos en datos sin etiquetar) y el aprendizaje por refuerzo (donde un agente aprende mediante la interacción con un entorno). El problema de predicción de congestión planteado en esta tesis se enmarca de manera natural dentro del **aprendizaje supervisado**, ya que se dispone de datos históricos de operación de la red (las variables de entrada o *features*) junto con el valor real observado que se desea predecir (la utilización futura del enlace).

Dentro del aprendizaje supervisado, y de acuerdo con la naturaleza de la variable a predecir, es posible distinguir dos tareas fundamentales:

- **Regresión**: la variable objetivo es continua. En el contexto de esta investigación, corresponde a predecir directamente el valor futuro del factor de utilización $U(t+h)$ para un horizonte de predicción h, expresado como un número real.
- **Clasificación**: la variable objetivo es discreta o categórica. En este caso, corresponde a predecir si el enlace se encontrará o no en estado de congestión en el horizonte h, de acuerdo con la definición operativa formulada en la Ecuación 2.2 (es decir, si $U(t+h) \geq U_{th}$), reduciendo el problema a una clasificación binaria (congestión / no congestión).

Ambos enfoques son complementarios: un modelo de regresión ofrece información más rica (el valor exacto esperado de utilización), mientras que uno de clasificación proporciona una respuesta directamente accionable para un sistema de alertas tempranas. Por esta razón, se explorará la formulación del problema bajo ambos enfoques según el modelo empleado.

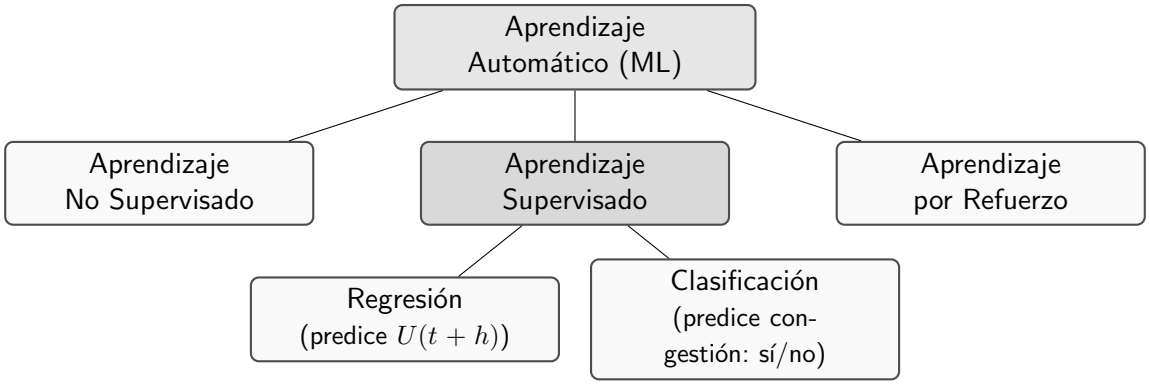


Figura 2.3: Ubicación del problema de predicción de congestión dentro de los paradigmas de *Machine Learning*.

2.2.2. Modelos de series temporales (ARIMA)

El modelo **ARIMA** (*AutoRegressive Integrated Moving Average*), desarrollado por Box y Jenkins, constituye uno de los enfoques estadísticos más consolidados para el análisis y predicción de series temporales [10]. Su nombre resume sus tres componentes:

- **AR (AutoRegresivo)**: el valor de la serie se modela como una combinación lineal de sus propios valores pasados, con un orden p que indica cuántos rezagos (*lags*) se consideran.
- **I (Integrado)**: aplica un proceso de diferenciación de orden d a la serie original para eliminar tendencias y volverla estacionaria.
- **MA (Media Móvil)**: el valor de la serie se modela en función de los errores de predicción pasados, con un orden q.

Un modelo se denota comúnmente como ARIMA(p, d, q), y se expresa formalmente como:

$$\left(1 - \sum_{i=1}^{p} \phi_i L^i\right) (1 - L)^d X_t = \left(1 + \sum_{i=1}^{q} \theta_i L^i\right) \varepsilon_t \tag{2.10}$$

donde X_t es la serie temporal de interés (la utilización en el instante t), L es el operador de rezago, ϕ_i y θ_i son los coeficientes, y ε_t representa el ruido blanco. Este modelo resulta fundamental como **línea base** (*baseline*) metodológica: al ser un método clásico y computacionalmente económico, establece un punto de referencia mínimo de desempeño contra el cual comparar los algoritmos más sofisticados.

2.2.3. Modelos de aprendizaje automático clásico (Random Forest, SVR)

A diferencia de ARIMA, que asume relaciones lineales explícitas entre los valores pasados y futuros, los modelos de aprendizaje automático clásico son capaces de capturar relaciones complejas y no lineales entre múltiples variables de entrada simultáneamente, lo cual

resulta valioso cuando se dispone de varias métricas de red (utilización, latencia, pérdida de paquetes).

Random Forest es un método de aprendizaje por conjunto (*ensemble*) que construye una multitud de árboles de decisión durante el entrenamiento, cada uno sobre una muestra aleatoria de los datos y considerando un subconjunto aleatorio de variables [11]. Para tareas de regresión, la predicción final se obtiene promediando las predicciones individuales:

$$\hat{y} = \frac{1}{T} \sum_{i=1}^{T} h_i(x) \tag{2.11}$$

donde T es el número total de árboles y $h_i(x)$ es la predicción del árbol i-ésimo. Esta estrategia reduce significativamente el riesgo de sobreajuste (*overfitting*) y ofrece una ventaja analítica clave: permite calcular la **importancia relativa** de las variables (*feature importance*), identificando empíricamente qué métricas predicen mejor la congestión.

Por su parte, **SVR** (*Support Vector Regression*) busca encontrar una función que se ajuste a los datos permitiendo un margen de tolerancia, penalizando únicamente las predicciones cuyo error exceda dicho margen. Su capacidad para emplear funciones de *kernel* le permite modelar relaciones no lineales complejas, constituyendo una alternativa robusta adicional.

Es imperativo señalar que estos algoritmos tabulares carecen de una representación interna del tiempo; tratan cada observación como un evento estático. Para aplicarlos a series temporales, es necesario transformar los datos mediante la técnica de **ventanas deslizantes** (*sliding windows*). Bajo este enfoque, la entrada al modelo se transforma en un vector que concatena un histórico de tamaño k: $[X_t, X_{t-1}, \ldots, X_{t-k}]$. Esto convierte la dependencia temporal en una dependencia espacial tabular que los algoritmos pueden procesar.

2.2.4. Modelos de Aprendizaje Profundo (LSTM, GRU)

Las redes neuronales recurrentes (RNN) fueron diseñadas específicamente para procesar datos secuenciales, manteniendo una suerte de memoria interna que se actualiza en cada paso temporal. Sin embargo, las RNN convencionales sufren de un problema conocido como el desvanecimiento del gradiente (*vanishing gradient*), que les impide aprender de manera efectiva dependencias que abarcan largos periodos de tiempo; una limitación especialmente relevante para el tráfico de red, donde los patrones de congestión pueden depender de comportamientos observados varios minutos u horas atrás.

Las **LSTM** (*Long Short-Term Memory*) fueron propuestas precisamente para resolver esta limitación, mediante la introducción de una estructura interna de compuertas (*gates*) que regulan de manera explícita qué información se conserva, se actualiza o se descarta a lo largo del tiempo [12]. Cada unidad LSTM mantiene un estado de celda c_t que actúa como memoria de largo plazo, y tres compuertas (de olvido, de entrada y de salida) que controlan el flujo de información:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(compuerta de olvido)} \tag{2.12}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(compuerta de entrada)} \tag{2.13}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(compuerta de salida)} \tag{2.14}$$

donde σ es la función de activación sigmoide, x_t es la entrada en el instante t, h_{t-1} es el estado oculto anterior, y W y b son los pesos y sesgos aprendidos. Esta arquitectura permite a la red aprender selectivamente qué eventos pasados (por ejemplo, una ráfaga de sincronización de datos ocurrida varios minutos atrás) siguen siendo relevantes en el presente.

Las **GRU** (*Gated Recurrent Units*) constituyen una variante simplificada de las LSTM, diseñada para reducir la complejidad computacional sin sacrificar capacidad predictiva [13]. Las GRU combinan las compuertas de olvido y entrada, prescindiendo del estado de celda independiente. Esto resulta en un modelo con menos parámetros entrenables y más eficiente de entrenar sobre conjuntos de datos de tamaño moderado.

2.2.5. Métricas de evaluación

Para comparar el desempeño de los modelos (ARIMA, Random Forest, SVR, LSTM y GRU) es necesario definir métricas cuantitativas de error. Sea y_i el valor real observado, $\hat{y}_i$ el valor predicho por el modelo, y n el número total de observaciones.

RMSE (*Root Mean Squared Error*, raíz del error cuadrático medio):

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2.15}$$

Al elevar al cuadrado cada error antes de promediar, el RMSE penaliza de manera desproporcionada los errores grandes, lo cual resulta conveniente para este trabajo: un modelo que falle drásticamente al predecir un evento de congestión severa es más problemático que uno con errores pequeños y uniformes.

MAE (*Mean Absolute Error*, error absoluto medio):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2.16}$$

A diferencia del RMSE, el MAE pondera todos los errores de manera lineal e igualitaria, resultando menos sensible a valores atípicos (*outliers*) y ofreciendo una interpretación directa en las mismas unidades que la variable original.

MAPE (*Mean Absolute Percentage Error*, error porcentual absoluto medio):

$$\text{MAPE} = \frac{100\,\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{2.17}$$

El MAPE expresa el error como un porcentaje relativo, facilitando su interpretación independientemente de la escala y permitiendo comparar el desempeño entre distintos conjuntos de datos.

Para la tarea de **clasificación binaria** (congestión vs. operación normal), dado que la congestión es un evento anómalo y el *dataset* está desbalanceado, se utilizarán las siguientes métricas basadas en la matriz de confusión (Verdaderos Positivos VP, Verdaderos Negativos VN, Falsos Positivos FP y Falsos Negativos FN):

$$\text{Precisión} = \frac{VP}{VP + FP} \tag{2.18}$$

$$\text{Exhaustividad} = \frac{VP}{VP + FN} \tag{2.19}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precisión} \times \text{Exhaustividad}}{\text{Precisión} + \text{Exhaustividad}} \tag{2.20}$$

La Tabla 2.2 sintetiza los cinco modelos presentados, sirviendo como punto de partida para la justificación metodológica que se desarrollará en el Capítulo 4.

Modelo	**Paradigma**	**Relaciones**	**Complejidad**
ARIMA	Estadístico clásico	Lineales	Baja
Random Forest	ML clásico (*ensemble*)	No lineales	Media
SVR	ML clásico (*kernel*)	No lineales	Media
LSTM	*Deep Learning* (RNN)	No lineales, largo plazo	Alta
GRU	*Deep Learning* (RNN)	No lineales, largo plazo	Media-Alta

Cuadro 2.2: Resumen comparativo de los modelos de predicción considerados en esta tesis.

Capítulo 3

Estado del Arte

El presente capítulo tiene como propósito situar esta tesis dentro del cuerpo de conocimiento existente sobre predicción de tráfico y congestión en redes de computadoras. Se busca identificar qué se ha hecho, con qué herramientas, bajo qué condiciones y con qué resultados, para de esta manera justificar con evidencia empírica —y no solo con intuición— las decisiones metodológicas que se detallarán en el Capítulo 4.

La revisión se organiza siguiendo la evolución cronológica y metodológica que ha caracterizado a este campo de estudio: se inicia con los trabajos fundamentados en métodos estadísticos clásicos (Sección 3.1), que dominaron la literatura durante las décadas de 1990 y 2000; se continúa con la incorporación de técnicas de *Machine Learning* clásico (Sección 3.2), que ganaron protagonismo conforme creció la disponibilidad de datos y de poder de cómputo; se revisan después los enfoques basados en *Deep Learning* (Sección 3.3), que representan el estado actual de mayor sofisticación técnica; se examinan las herramientas de monitoreo y los conjuntos de datos que la comunidad académica ha empleado para validar estos enfoques (Sección 3.4); y, finalmente, se sintetiza esta revisión identificando la brecha específica que esta tesis busca cubrir (Sección 3.5).

Para cada trabajo revisado se prestará atención particular a cuatro aspectos: 1) el enfoque metodológico empleado y sus fundamentos teóricos; 2) la naturaleza y el origen de los datos utilizados para entrenar y validar el modelo; 3) las métricas de evaluación reportadas y los resultados obtenidos; y 4) las limitaciones reconocidas por los propios autores o identificables desde una lectura crítica, que resultan relevantes para el diseño de esta investigación. Este último punto merece una precisión importante: el objetivo de señalar limitaciones no es desacreditar el trabajo de otros investigadores, sino identificar con precisión los aspectos que aún no han sido resueltos de manera satisfactoria por la literatura existente, los cuales constituyen la motivación concreta para el aporte de esta tesis.

Es importante advertir, además, que la mayoría de los trabajos revisados no abordan la predicción de la congestión como un evento discreto y accionable —tal como se definió formalmente en la Sección 2.1.3 (Ecuación 2.2)—, sino que se concentran en la predicción general del volumen de tráfico como una serie temporal continua. Esta distinción, aparentemente sutil, resulta central para posicionar la contribución de este trabajo, y se retomará con mayor detalle en la síntesis del estado del arte (Sección 3.5).

3.1. Revisión de trabajos sobre predicción de tráfico con métodos estadísticos

Los métodos estadísticos clásicos, y en particular la familia de modelos ARIMA presentada en la Sección 2.2.2, constituyeron durante largo tiempo la herramienta predominante para el análisis y predicción del tráfico de red, debido a su fundamento matemático bien establecido y a su relativa simplicidad de implementación frente a los enfoques de aprendizaje automático que se popularizaron posteriormente.

Uno de los trabajos pioneros en esta línea es el de Moayedi y Masnadi-Shirazi [14], quienes propusieron el empleo de un modelo ARIMA básico no solo con fines de predicción, sino también de detección de anomalías en el tráfico de red. Su aporte central consistió en descomponer la señal de tráfico observada en dos componentes: una parte de variación normal, que sigue un patrón predecible y puede ser capturada adecuadamente por el modelo ARIMA, y una parte anómala, correspondiente a cambios súbitos e impredecibles que el modelo no logra anticipar. Bajo esta lógica, los autores plantearon que las desviaciones significativas entre el valor predicho por ARIMA y el valor real observado pueden emplearse como un indicador de eventos anómalos, entre los cuales se encontrarían episodios de saturación o congestión no habitual. Para los fines de esta tesis, este trabajo resulta relevante porque ilustra una limitación estructural de los modelos puramente estadísticos: su capacidad predictiva depende críticamente de que el tráfico se comporte de manera similar a su historia reciente, una condición que —como se discutió en la Sección 2.1.3— no necesariamente se cumple durante los episodios de congestión más severos, que suelen originarse precisamente por comportamientos atípicos del tráfico.

Una línea de trabajo posterior buscó superar esta limitación combinando ARIMA con técnicas complementarias capaces de capturar la componente no lineal del tráfico que el modelo estadístico puro no logra representar. Tian [15] propone un enfoque híbrido en el que ARIMA se emplea para modelar y predecir la componente lineal de la serie de tráfico, mientras que un filtro de Volterra adaptativo —optimizado mediante un algoritmo de enjambre de partículas— se encarga de modelar la componente no lineal residual que queda después de sustraer la predicción de ARIMA de la serie original. Los resultados reportados por el autor muestran una mejora en la precisión de la predicción respecto del uso de ARIMA de manera aislada, lo cual respalda una idea central que se retomará en la síntesis de este capítulo: los modelos puramente lineales, si bien valiosos como referencia, resultan insuficientes por sí solos para capturar la complejidad real del tráfico de las redes modernas.

Más recientemente, Kochetkova et al. [16] evaluaron el desempeño de modelos SARIMA (ARIMA estacional) y de suavizamiento exponencial de Holt-Winters para la predicción a corto plazo de tráfico en redes móviles de quinta generación (5G), motivados por la necesidad de una gestión dinámica y eficiente de los recursos de radio en este tipo de infraestructuras. Este trabajo es particularmente pertinente para la presente investigación por dos razones: primero, porque emplea explícitamente la componente estacional del tráfico (patrones que se repiten en ciclos horarios, diarios o semanales), un fenómeno que también se espera observar en el *dataset* primario de esta tesis, dada la coexistencia de tráfico residencial, de videovigilancia y de sincronización en la nube descrita en la Sección 4.4; y segundo, porque data de un periodo reciente (2023), lo cual evidencia que, a pesar del auge de los modelos

de *Deep Learning*, los métodos estadísticos clásicos continúan siendo objeto de investigación activa y conservan vigencia práctica, particularmente como líneas base computacionalmente económicas frente a modelos más complejos.

En conjunto, estos tres trabajos —que abarcan más de una década de investigación— permiten extraer una primera conclusión relevante para esta tesis: los modelos estadísticos como ARIMA y sus variantes ofrecen un desempeño razonable y una implementación computacionalmente accesible cuando el tráfico exhibe patrones regulares y estacionarios, pero muestran limitaciones consistentes frente a comportamientos no lineales, anómalos o súbitos, que son, precisamente, los que con mayor frecuencia preceden a un evento de congestión severa. Esta observación motiva directamente la revisión de enfoques de *Machine Learning* clásico que se presenta en la Sección 3.2, los cuales fueron desarrollados en buena medida como respuesta a estas limitaciones.

3.2. Predicción de tráfico y congestión con *Machine Learning* clásico

Frente a las limitaciones de los modelos estadísticos lineales discutidas en la sección anterior, un segundo cuerpo de literatura ha explorado el empleo de algoritmos de *Machine Learning* clásico —presentados en la Sección 2.2.3 de esta tesis— para abordar el problema de predicción de congestión, aprovechando su capacidad para modelar relaciones no lineales entre múltiples variables de red simultáneamente.

Un trabajo directamente pertinente para esta investigación es el de Sneha et al. [17], quienes abordaron específicamente el problema de predecir congestión a nivel de enrutador con el fin de anticipar la pérdida de paquetes antes de que esta ocurriera. Los autores entrenaron y compararon cuatro algoritmos de clasificación —K-Vecinos Más Cercanos (KNN), *Random Forest*, *Gradient Boosting* y Árboles de Decisión— sobre datos de tráfico simulado, encontrando que *Random Forest* y KNN ofrecieron el mejor desempeño para anticipar el tipo de pérdida de paquetes que se produciría. Este trabajo resulta especialmente relevante para la presente tesis por dos razones: primero, porque aborda la congestión como un evento a nivel de dispositivo de red (el enrutador) en lugar de como una propiedad agregada de toda la red, un enfoque compatible con la definición operativa formulada en la Ecuación 2.2; y segundo, porque confirma empíricamente la superioridad de *Random Forest* sobre otros algoritmos clásicos para esta tarea específica, lo cual respalda su inclusión como uno de los modelos a implementar en este trabajo.

En un dominio relacionado mas no idéntico, Qian et al. [18] propusieron SVR-NoC, un modelo basado en *Support Vector Regression* para predecir la latencia de tráfico en arquitecturas de Redes en Chip (*Network-on-Chip*, NoC) —infraestructuras de interconexión a pequeña escala empleadas dentro de procesadores multinúcleo—. Si bien este trabajo no se refiere a redes de computadoras tradicionales como las que aborda esta tesis, sino a una escala de red sustancialmente distinta, resulta útil citarlo porque demuestra la viabilidad de SVR para modelar fenómenos de saturación y tiempos de espera en sistemas basados en colas, obteniendo una reducción superior al 100 % en tiempo de cómputo respecto a métodos de simulación tradicionales sin sacrificar precisión de manera significativa. Esta evidencia, aunque proveniente de un dominio distinto, refuerza la pertinencia de incluir SVR entre los

modelos a evaluar en el Capítulo 5, dada su comprobada capacidad para modelar fenómenos de congestión basados en la teoría de colas (el mismo fundamento teórico presentado en la Sección 2.1.3).

Finalmente, el trabajo de Labonne et al. [19] resulta el más cercano en espíritu al problema abordado en esta investigación, ya que se enfoca explícitamente en predecir la utilización de ancho de banda en enlaces de red con el propósito declarado de detectar y corregir la congestión antes de que esta ocurra —el mismo objetivo central planteado en el Planteamiento del problema (Sección 1.1)—. Los autores generaron su propio conjunto de datos mediante una red simulada, sometida deliberadamente a cargas de tráfico variables para provocar enlaces saturados, y aplicaron un proceso de ingeniería de características para enriquecer las variables de entrada antes de entrenar y comparar tres modelos: ARIMA, un Perceptrón Multicapa (MLP) y una red LSTM. Aunque estos dos últimos modelos corresponden propiamente a la familia de *Deep Learning* que se revisará en la Sección 3.3, el aporte metodológico de este trabajo reside en su estrategia de generación y preprocesamiento de datos: la construcción deliberada de escenarios de saturación mediante tráfico controlado ofrece un precedente metodológico directamente aplicable a la validación del *dataset* primario de esta investigación (descrito en la Sección 4.4), en caso de que la red privada bajo estudio no exhiba de forma natural una cantidad suficiente de episodios de congestión durante el periodo de recolección.

La revisión de estos tres trabajos permite extraer una segunda conclusión relevante para esta tesis: los modelos de *Machine Learning* clásico, y en particular *Random Forest* y SVR, han demostrado un desempeño consistentemente superior a los modelos puramente estadísticos cuando se trata de capturar relaciones no lineales en el tráfico de red, incluso en dominios de aplicación tan distintos como enrutadores IP tradicionales, redes en chip y enlaces de banda ancha simulados. No obstante, ninguno de los trabajos revisados en esta sección explota explícitamente la naturaleza *secuencial* y dependiente del tiempo del tráfico de red —es decir, el hecho de que el valor de utilización en un instante dado depende fuertemente de una ventana de observaciones pasadas, y no únicamente de un conjunto de variables independientes—. Esta limitación motiva la revisión de los modelos de *Deep Learning* basados en arquitecturas recurrentes, presentados en la siguiente sección, los cuales fueron diseñados precisamente para capturar este tipo de dependencias temporales.

3.3. Predicción con *Deep Learning*

El auge de las arquitecturas de *Deep Learning* presentadas en la Sección 2.2.4 —particularmente las redes recurrentes LSTM y GRU— ha desplazado en buena medida a los enfoques de *Machine Learning* clásico como el foco principal de la investigación reciente en predicción de tráfico de redes, motivado por su capacidad para capturar dependencias temporales de largo alcance sin necesidad de un diseño manual de características tan extenso como el que requieren los modelos revisados en la Sección 3.2.

Uno de los primeros trabajos en establecer una comparación sistemática entre arquitecturas de redes neuronales aplicadas específicamente al tráfico de redes de computadoras —y no a otros dominios— es el de Oliveira, Barbar y Soares [20], quienes evaluaron cuatro enfoques distintos: un Perceptrón Multicapa (MLP) entrenado con retropropagación estándar, una variante con retropropagación resiliente (Rprop), una red neuronal recurrente (RNN) simple,

y un Autoencoder Apilado (*Stacked Autoencoder*, SAE) como representante de arquitecturas de aprendizaje profundo más complejas. De manera notable, y hasta cierto punto contraintuitiva, los autores encontraron que los modelos más simples —la RNN y el MLP— superaron en desempeño al SAE, a pesar de que este último posee una arquitectura considerablemente más profunda y compleja. Este hallazgo resulta relevante para la presente tesis porque introduce una advertencia metodológica importante: mayor complejidad arquitectónica no garantiza necesariamente un mejor desempeño predictivo, especialmente cuando el volumen de datos disponible es limitado, una condición que podría presentarse en el *dataset* primario de esta investigación (Sección 4.4) dada la escala reducida de la red privada bajo estudio en comparación con los grandes conjuntos de datos empleados típicamente en la literatura de *Deep Learning*.

Precisamente sobre esta preocupación —la disponibilidad limitada de datos— profundiza el trabajo de Saha, Haque y Sidebottom [21], quienes abordan explícitamente el problema de predicción de tráfico de Internet en redes de proveedores de servicio (ISP) de tamaño reducido, donde la cantidad de datos históricos disponibles suele ser insuficiente para entrenar adecuadamente modelos de *Deep Learning* de gran capacidad. Los autores proponen el empleo de aprendizaje por transferencia (*transfer learning*) —entrenando inicialmente modelos LSTM (en sus variantes Seq2Seq y Seq2Seq con mecanismo de atención) sobre un conjunto de datos extenso proporcionado por Juniper Networks, para posteriormente adaptarlos a conjuntos de datos considerablemente más pequeños— junto con técnicas de aumento de datos basadas en la transformada *wavelet*. Este trabajo es particularmente pertinente para esta tesis porque su motivación —redes pequeñas con historial de datos limitado— describe con precisión la situación del *dataset* primario de esta investigación, recolectado durante un periodo de apenas 30 días sobre una red de escala reducida. Sus hallazgos sugieren que, de presentarse limitaciones de datos similares, estrategias como el *transfer learning* o el aumento de datos podrían explorarse como líneas de trabajo futuro si el desempeño de los modelos LSTM y GRU propuestos en el Capítulo 4 resultara insuficiente.

Finalmente, el trabajo de Santos et al. [22] ofrece la comparación más directamente aplicable a las decisiones metodológicas de esta tesis, al confrontar explícitamente el desempeño de LSTM contra GRU —los dos modelos de *Deep Learning* seleccionados en la Sección 2.2.4— para la predicción de tráfico de Internet móvil. Los autores emplearon dos meses de datos reales del conjunto Telecom Italia correspondientes al área metropolitana de Milán —una duración de recolección del mismo orden de magnitud que la empleada para el *dataset* primario de este trabajo (Sección 4.4)—, agrupando primero las series de tráfico mediante *clustering* de K-Means y optimizando posteriormente los hiperparámetros de cada modelo mediante búsqueda en malla (*Grid Search*). Sus resultados, evaluados mediante RMSE, indican que la LSTM superó a la GRU en la mayoría de los agrupamientos analizados, aunque los autores destacan que el desempeño relativo varió de manera apreciable entre distintas zonas geográficas de la ciudad, sugiriendo que la superioridad de un modelo sobre otro no es universal, sino que depende de las características particulares del tráfico analizado. Este hallazgo es directamente relevante para el diseño experimental de esta tesis: motiva la necesidad de evaluar tanto LSTM como GRU sobre *ambos* conjuntos de datos —el *dataset* primario recolectado de forma pasiva en la red privada bajo estudio y el conjunto de validación MAWI—, en lugar de asumir de antemano la superioridad de un modelo sobre el otro, dado que la literatura revisada aquí no ofrece una conclusión unívoca al respecto.

La revisión de estos tres trabajos, en conjunto con los de la Sección 3.2, permite extraer una tercera y última conclusión relevante para el diseño metodológico de esta tesis: si bien las arquitecturas de *Deep Learning* —y en particular las redes recurrentes como LSTM y GRU— han demostrado capacidades notables para modelar la naturaleza secuencial del tráfico de red, su desempeño superior frente a modelos más simples no está garantizado de manera universal, sino que depende críticamente de factores como el volumen de datos disponible, la naturaleza del tráfico analizado y la elección adecuada de hiperparámetros. Esta observación refuerza la pertinencia del diseño comparativo adoptado en este documento —evaluando ARIMA, *Random Forest* y LSTM/GRU de manera simultánea sobre los mismos conjuntos de datos—, en lugar de asumir *a priori* que el modelo más sofisticado será necesariamente el de mejor desempeño. Esta idea se retoma y se sintetiza, junto con las conclusiones de las Secciones 3.1 y 3.2, en la síntesis del estado del arte que cierra este capítulo (Sección 3.5).

3.4. Herramientas de monitoreo y *datasets* utilizados

Además de los modelos predictivos revisados en las secciones anteriores, resulta igualmente relevante examinar cómo la comunidad académica ha obtenido los datos que alimentan dichos modelos, ya que las decisiones de instrumentación —qué herramienta emplear, con qué granularidad y sobre qué tipo de red— condicionan directamente la calidad y el alcance de cualquier sistema de predicción de congestión. Esta sección organiza dicha revisión en dos partes: primero, las herramientas y protocolos de monitoreo empleados típicamente en la literatura; y segundo, una síntesis de los conjuntos de datos concretos utilizados por los trabajos revisados en las Secciones 3.1 a 3.3, incluyendo el propio *MAWI Working Group Traffic Archive* adoptado como *dataset* de validación en esta tesis (Sección 4.4).

Taxonomía de las técnicas de monitoreo

Como se adelantó en la Sección 2.1.4, las técnicas de monitoreo de red se dividen fundamentalmente en dos familias: el monitoreo **pasivo**, que observa el tráfico existente sin generar tráfico adicional, y el monitoreo **activo**, que inyecta tráfico de prueba deliberadamente para medir el comportamiento de la red. La Figura 3.1 sintetiza esta taxonomía junto con las herramientas y protocolos más representativos de cada familia, varios de los cuales fueron adoptados directamente en la metodología de recolección de datos de esta tesis (Sección 4.4).

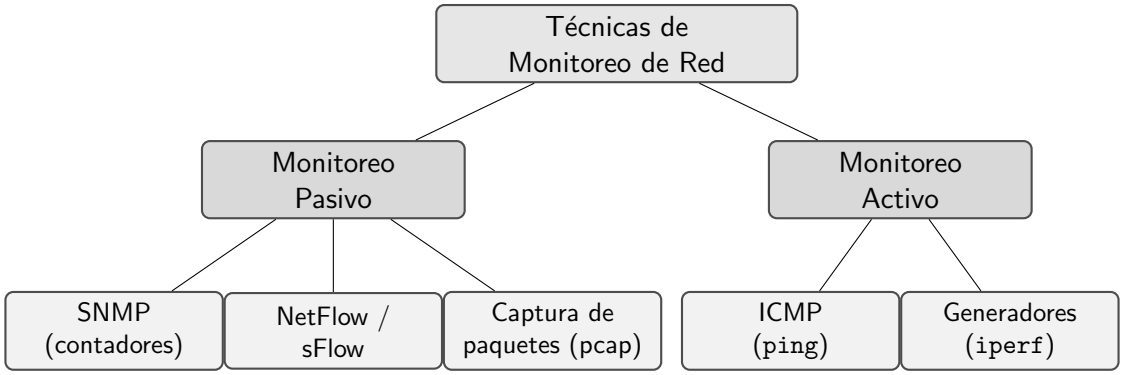


Figura 3.1: Taxonomía de las técnicas de monitoreo de red empleadas en la literatura revisada.

La Tabla 3.1 amplía esta taxonomía, contrastando cada herramienta en términos de la sobrecarga (*overhead*) que introduce en la red, la granularidad temporal que permite obtener, y el tipo de información que provee.

Herramienta	**Tipo**	***Overhead***	**Información obtenida**
SNMP	Pasivo	Muy bajo	Contadores agregados de utilización, errores y descartes (Sección 2.1.4).
NetFlow / sFlow	Pasivo	Medio*	Metadatos por flujo: IPs, puertos, bytes, paquetes. *El *overhead* real depende críticamente de la tasa de muestreo (*sampling rate*) configurada.
Captura (pcap)	Pasivo	Medio-Alto	Registro completo de paquetes; requiere procesamiento posterior (ej. `tshark`).
ICMP (`ping`)	Activo	Bajo	Latencia (RTT) y pérdida de paquetes hacia un destino específico.
Generadores (`iperf`)	Activo	Alto	*Throughput* máximo alcanzable; puede inducir congestión artificialmente.

Cuadro 3.1: Comparación de herramientas de monitoreo empleadas en la literatura revisada.

Como se estableció en la Sección 4.4, esta tesis adopta un enfoque predominantemente pasivo (SNMP para utilización, complementado con mediciones activas ligeras mediante ICMP para latencia y pérdida de paquetes), evitando deliberadamente el uso de generadores de tráfico como `iperf` sobre la red privada bajo estudio, dado que estos podrían introducir una carga adicional que alterara artificialmente las condiciones reales de operación que se busca modelar.

Síntesis de datasets empleados en la literatura

La Tabla 3.2 sintetiza los conjuntos de datos empleados por los trabajos revisados en las Secciones 3.1 a 3.3 de este capítulo, permitiendo identificar patrones respecto al tipo de red, la duración de recolección y la naturaleza —real o simulada— de cada dataset.

Trabajo	**Dataset**	**Naturaleza**	**Escala**	**Duración**
Moayedi & Masnadi-Shirazi [14]	Tráfico propio	Real	No especificada	No especificada
Kochetkova et al. [16]	Red móvil 5G	Real	Operador móvil	Corto plazo
Sneha et al. [17]	Simulación	Simulado	Enrutador individual	No aplica
Labonne et al. [19]	Red simulada (VirtualBox)	Simulado	Red de laboratorio	No especificada
Santos et al. [22]	Telecom Italia (Milán)	Real	Metropolitana	2 meses
Saha et al. [21]	Juniper Networks + ISPs pequeños	Real	ISP (variable)	Variable
Esta tesis (primario)	**Red del Cliente**	**Real**	**Red pequeña/PYME**	**(numero final) días**
Esta tesis (validación)	**MAWI Archive** [23]	**Real**	**Tránsito 1-10 Gbps**	**Diaria (continua)**

Cuadro 3.2: Comparación de datasets empleados en la literatura revisada frente a los datasets adoptados en esta tesis.

Dos observaciones se desprenden de esta comparación. En primer lugar, la duración adoptada para el dataset primario de esta tesis es del mismo orden de magnitud que la empleada por Santos et al. [22], uno de los trabajos más metodológicamente cercanos revisados en este capítulo, lo cual respalda la suficiencia de dicho periodo para capturar patrones cíclicos diarios y semanales de tráfico. En segundo lugar, se observa que una proporción considerable de los trabajos revisados —particularmente aquellos enfocados en ML clásico— recurren a datos simulados en lugar de tráfico real, una decisión metodológica que facilita el control experimental pero que limita la validez de los hallazgos frente a condiciones de operación reales. Esta observación refuerza el valor diferencial del enfoque híbrido adoptado en esta tesis, que combina datos reales de dos naturalezas y escalas distintas en lugar de recurrir a tráfico simulado o a una única fuente.

3.5. Síntesis del estado del arte y brecha a cubrir

Las secciones anteriores de este capítulo revisaron nueve trabajos distintos, organizados cronológica y metodológicamente en tres grandes familias: métodos estadísticos clásicos (Sección 3.1), *Machine Learning* clásico (Sección 3.2) y *Deep Learning* (Sección 3.3), complementados con una revisión de las herramientas de instrumentación y los *datasets* empleados por dichos trabajos (Sección 3.4). El propósito de esta sección final es triple: primero, consolidar en un solo lugar los hallazgos dispersos a lo largo del capítulo; segundo, analizar de manera sistemática el *qué*, *por qué*, *cómo*, *para qué* y *cuándo* de la literatura revisada, con el fin de identificar patrones que no resultan evidentes al leer cada trabajo de manera aislada; y tercero —y más importante—, delimitar con precisión la brecha específica que esta tesis busca cubrir, así como el aporte concreto que este trabajo ofrece de vuelta a la comunidad académica.

Tabla maestra de trabajos revisados

La Tabla 3.3 reúne los nueve trabajos citados en este capítulo, contrastando su enfoque metodológico, la naturaleza de los datos empleados, su aporte principal y —el elemento más relevante para los fines de esta síntesis— la limitación específica que, desde la perspectiva de esta tesis, dicho trabajo deja abierta.

Trabajo	**Año**	**Enfoque**	**Aporte principal**	**Limitación relevante**
Moayedi & Masnadi-Shirazi [14]	2008	ARIMA	Detección de anomalías vía residuos de predicción	No captura no linealidades; congestión inferida indirectamente.
Tian [15]	2021	ARIMA + Volterra	Mejora de ARIMA con componente no lineal	Sigue centrado en volumen de tráfico, no en congestión como evento.
Kochetkova et al. [16]	2023	SARIMA / Holt-Winters	Explota estacionalidad en redes 5G	Sin comparación contra ML/DL en el mismo estudio.
Sneha et al. [17]	2020	*Random Forest*, KNN, GB, DT	Predicción de congestión a nivel de enrutador	Datos simulados; sin validación en tráfico real.
Qian et al. [18]	2016	SVR	Modelado de latencia basado en colas	Dominio distinto (NoC, no redes IP tradicionales).
Labonne et al. [19]	2021	ARIMA, MLP, LSTM	*Framework* completo de generación y predicción de utilización	Red simulada; sin validación externa.
Oliveira et al. [20]	2016	MLP, RNN, SAE	Comparación de arquitecturas de red neuronal	Mayor complejidad (SAE) no garantiza mejor desempeño.
Saha et al. [21]	2024	LSTM + *Transfer Learning*	Aborda *datasets* pequeños en ISPs	No compara contra modelos estadísticos ni ML clásico.
Santos et al. [22]	2022	LSTM vs. GRU	Comparación directa LSTM/GRU en tráfico real (2 meses)	Un solo paradigma (DL); sin ARIMA ni ML clásico.

Cuadro 3.3: Síntesis comparativa de los trabajos revisados en el Estado del Arte.

Evolución cronológica y metodológica

La Figura 3.2 presenta una línea de tiempo que sitúa los trabajos revisados —junto con los hitos teóricos fundamentales presentados en el Marco Teórico de esta tesis (Capítulo 2)— dentro de la evolución histórica del campo, permitiendo visualizar de manera inmediata la transición gradual de los métodos estadísticos hacia el *Machine Learning* clásico y, finalmente, hacia el *Deep Learning*.

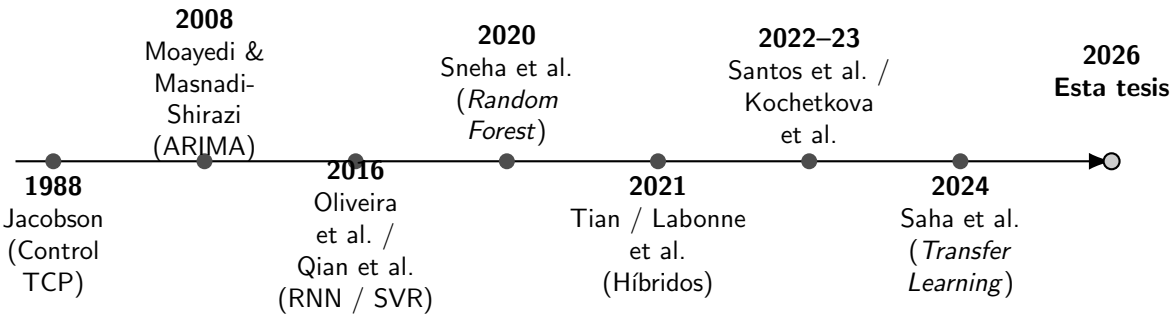


Figura 3.2: Línea de tiempo de la evolución metodológica en predicción de tráfico y congestión de red.

Esta visualización revela un patrón claro: durante casi dos décadas (1988–2008) predominaron los métodos estadísticos; a partir de 2016 comienza una transición gradual y simultánea hacia el ML clásico y las primeras arquitecturas de *Deep Learning*; y a partir de 2020 los tres paradigmas coexisten activamente en la literatura, sin que ninguno haya desplazado por completo a los anteriores. Esta coexistencia —más que una sustitución progresiva— es, como se argumentará más adelante, uno de los elementos que justifica el diseño comparativo de esta tesis.

Análisis por dimensiones

¿Qué se ha investigado? La literatura revisada se ha concentrado abrumadoramente en la predicción del **volumen de tráfico** como una serie temporal continua (Moayedi & Masnadi-Shirazi [14]; Tian [15]; Kochetkova et al. [16]; Oliveira et al. [20]; Santos et al. [22]), tratando la congestión como una consecuencia implícita de dicho volumen más que como un evento a predecir de manera directa y explícita. Solo dos de los nueve trabajos revisados —Sneha et al. [17] y, de manera indirecta, Qian et al. [18]— abordan la congestión como una variable de interés discreta y directamente accionable, más cercana al enfoque de clasificación binaria establecido en la definición operativa de esta tesis (Ecuación 2.2).

¿Por qué se ha investigado de esta manera? La motivación predominante en la literatura revisada es la optimización de la gestión de recursos de red (asignación dinámica de ancho de banda, planeación de capacidad, mejora de la Calidad de Servicio o QoS) más que la prevención proactiva de eventos específicos de degradación del servicio. Esta orientación explica por qué la mayoría de los trabajos evalúan sus modelos mediante métricas de error continuo (RMSE, MAE, MAPE) en lugar de métricas propias de un problema de clasificación (precisión, exhaustividad, *F1-score*), a pesar de que estas últimas resultarían más informativas para un caso de uso de alertamiento temprano como el que plantea esta tesis.

¿Cómo se ha investigado? Metodológicamente, se observa una fragmentación notable: cada trabajo revisado evalúa típicamente **un solo paradigma** de modelado —o, en el mejor de los casos, variantes dentro de un mismo paradigma, como Santos et al. [22] comparando LSTM contra GRU, ambos de *Deep Learning*—. Ningún trabajo de los revisados en este

capítulo compara de manera simultánea, sobre el mismo conjunto de datos y bajo las mismas condiciones experimentales, representantes de los tres paradigmas (estadístico, ML clásico y *Deep Learning*), lo cual dificulta extraer conclusiones comparativas robustas a partir de la literatura existente de forma aislada. Asimismo, como se documentó en la Tabla 3.1 de la Sección 3.4, una proporción considerable de los trabajos de ML clásico recurre a datos simulados (Sneha et al. [17]; Labonne et al. [19]), mientras que los trabajos con datos reales rara vez validan sus hallazgos en más de un escenario de red.

¿Para qué se ha investigado? La finalidad última declarada por la mayoría de los trabajos revisados es habilitar una gestión de red proactiva en lugar de reactiva, alineada con la motivación general de esta tesis expuesta en el Planteamiento del Problema (Sección 1.1). Sin embargo, pocos trabajos —con la notable excepción de Sneha et al. [17]— formulan explícitamente un umbral operativo o un mecanismo de alertamiento accionable; la mayoría se detiene en la demostración de la capacidad predictiva del modelo, sin traducir dicha capacidad en un criterio de decisión concreto. En contraste, contar con un umbral predictivo accionable como U_{th} permite a un administrador o sistema automatizado tomar medidas preventivas reales en la infraestructura, tales como la inyección dinámica de reglas de *firewall* para limitar tráfico no esencial, el ajuste de métricas en protocolos de enrutamiento para desviar flujos, o la redirección de tráfico crítico a través de túneles seguros alternativos antes de que la saturación provoque la caída de servicios esenciales.

¿Cuándo se ha investigado y hacia dónde se dirige el campo? Como se ilustra en la Figura 3.2, la investigación más reciente (2022–2024) muestra dos tendencias emergentes particularmente relevantes para esta tesis: por un lado, una preocupación creciente por la aplicabilidad práctica de estos modelos en escenarios de datos limitados (Saha et al. [21]), reconociendo que gran parte de la literatura previa asume la disponibilidad de conjuntos de datos extensos poco representativos de redes pequeñas o medianas; y por otro, comparaciones cada vez más rigurosas entre arquitecturas específicas de *Deep Learning* (Santos et al. [22]) que evidencian que la superioridad de un modelo sobre otro no es universal, sino dependiente del contexto de red particular.

A partir del análisis anterior es posible formular con precisión la brecha que esta tesis busca cubrir, articulada en tres componentes específicos:

1. **Brecha conceptual**: la literatura revisada predice predominantemente el *volumen* de tráfico como serie continua, mientras que esta tesis predice la **congestión** como evento discreto y accionable, mediante la definición operativa formalizada en la Ecuación 2.2 (Sección 2.1.3), permitiendo abordar el problema tanto desde la regresión como desde la clasificación (Sección 2.2.1).

2. **Brecha metodológica**: ningún trabajo revisado compara de manera simultánea y bajo condiciones experimentales controladas representantes de los tres paradigmas de modelado —estadístico (ARIMA), ML clásico (*Random Forest*) y *Deep Learning* (LSTM/GRU)—, tal como se plantea en el diseño metodológico de esta tesis (Capítulo 4).

3. **Brecha de validación**: la mayoría de los trabajos revisados se apoyan en un único *dataset*, ya sea real o simulado, sin contrastar sus hallazgos frente a escenarios de red de naturaleza o escala distinta. Esta tesis, mediante el enfoque híbrido descrito en la Sección 4.4 —combinando la red privada de escala reducida bajo estudio con el archivo de tráfico de gran escala MAWI—, aborda directamente esta limitación, en línea con la recomendación implícita de trabajos como el de Saha et al. [21] respecto a la necesidad de validar modelos más allá de un único escenario de datos.

La Tabla 3.4 sintetiza visualmente este posicionamiento, contrastando las características metodológicas predominantes en la literatura revisada frente al diseño adoptado en esta investigación.

Dimensión	**Literatura revisada (predominante)**	**Esta tesis**
Variable objetivo	Volumen de tráfico (continuo)	Congestión (discreta, vía U_{th})
Paradigmas comparados	Uno solo, o variantes de uno solo	Tres (estadístico, ML clásico, DL) simultáneamente.
Naturaleza de los datos	Real o simulado, una sola fuente	Real, dos fuentes complementarias (híbrido).
Escala de validación	Una sola escala de red	Dos escalas contrastantes (privada reducida y tránsito de gran escala).
Umbral accionable	Raramente explícito	Definido formalmente (U_{th}, Ecuación 2.2).

Cuadro 3.4: Posicionamiento metodológico de esta tesis frente a la literatura revisada.

Aporte de esta tesis a la literatura existente

En correspondencia con las tres brechas identificadas, esta tesis ofrece tres aportes concretos a la comunidad académica. Primero, aporta evidencia empírica comparativa entre los tres paradigmas de modelado revisados en este capítulo, evaluados bajo condiciones experimentales controladas y sobre los mismos conjuntos de datos —un tipo de comparación ausente en la literatura revisada, donde cada paradigma suele evaluarse de manera aislada—. Segundo, aporta un caso de estudio de recolección de datos reales en una red heterogénea de pequeña escala (residencial/PYME con videovigilancia, servicios en la nube y tráfico de ISP coexistiendo en el mismo enlace), un escenario significativamente menos representado en la literatura que las redes troncales de gran escala o los entornos simulados de laboratorio. Tercero, aporta una definición operativa explícita y formalizada de congestión (Ecuación 2.2), que traduce directamente la salida de los modelos predictivos en un criterio accionable para la toma de decisiones, respondiendo a la limitación de ümbral accionable raramente explícitoïdentificada en la Tabla 3.4.

Recíprocamente, esta tesis también reconoce su deuda con la literatura revisada: la definición del umbral operativo U_{th} se apoya en la evidencia de estacionalidad documentada por Kochetkova et al. [16]; el diseño de la comparación LSTM/GRU replica la metodología validada por Santos et al. [22] sobre un periodo de recolección equivalente; y la estrategia de mitigación ante *datasets* de tamaño reducido, en caso de ser necesaria, se apoyaría directamente en el precedente de *transfer learning* establecido por Saha et al. [21]. De esta manera, el capítulo de Metodología que sigue a continuación (Capítulo 4) no parte de cero, sino que se construye deliberadamente sobre las fortalezas identificadas y en respuesta directa a las limitaciones documentadas a lo largo de esta revisión del estado del arte.

Capítulo 4

Metodología

Los capítulos anteriores construyeron el fundamento conceptual de esta tesis: se definió formalmente qué se entiende por congestión de red (Capítulo 2, Ecuación 2.2), y se identificaron, mediante la revisión sistemática de la literatura, tres brechas específicas que este trabajo busca cubrir (Capítulo 3, Sección 3.5). Corresponde ahora al presente capítulo dar el paso de la revisión a la ejecución: traducir esas definiciones y las brechas identificadas en un plan de trabajo preciso, replicable y justificado en cada una de sus decisiones.

A diferencia de los capítulos precedentes, en los que el objeto de estudio era la producción académica de terceros, este capítulo describe exclusivamente decisiones metodológicas propias de esta investigación: qué tipo de investigación se está realizando, cómo se define y mide operativamente cada variable, de dónde provienen los datos, cómo se procesan, qué modelos se implementan y con qué criterios se evalúan. Cada una de estas decisiones no es arbitraria, sino que responde directamente a lo establecido en los capítulos anteriores —la definición operativa de congestión, el *dataset* híbrido descrito en la Sección 4.4, y los tres paradigmas de modelado comparados en la Sección 2.2—, por lo que el lector encontrará a lo largo de este capítulo referencias constantes hacia atrás, cerrando el círculo argumentativo abierto desde el planteamiento del problema.

Es importante señalar que las decisiones metodológicas aquí documentadas no necesariamente se tomaron en un único momento ni de manera lineal; muchas de ellas —como la elección del *dataset* híbrido o la definición del umbral de congestión— fueron resultado de un proceso iterativo de análisis y ajuste. Sin embargo, este capítulo las presenta de manera ordenada y sistemática, tal como corresponde a un documento metodológico formal, comenzando por la caracterización general del tipo de investigación que enmarca a esta tesis.

4.1. Tipo de investigación

La presente investigación se clasifica, de acuerdo con los criterios propuestos por Hernández-Sampieri [24], en cuatro dimensiones complementarias: su finalidad, su enfoque, su diseño y su alcance. Esta caracterización no es un mero formalismo, sino que determina directamente las decisiones metodológicas que se detallan en el resto de este capítulo, desde la forma en que se recolectan los datos hasta los criterios empleados para evaluar los modelos desarrollados.

Según su finalidad: investigación aplicada

Esta tesis constituye una investigación de tipo **aplicada** (también denominada tecnológica), en tanto que su propósito no es generar conocimiento teórico nuevo sobre las técnicas de *Machine Learning* empleadas —ARIMA, *Random Forest* y LSTM, cuyos fundamentos matemáticos ya se encuentran plenamente establecidos y fueron revisados en la Sección 2.2—, sino aplicar dicho conocimiento existente a la resolución de un problema práctico y delimitado: la predicción de congestión en un escenario real de red, tal como se planteó en la Sección 1.1. El criterio de éxito de esta investigación, por tanto, no reside en la novedad matemática de los modelos empleados, sino en la evidencia empírica que se obtenga sobre su viabilidad y desempeño comparativo al resolver el problema planteado.

Según su enfoque: investigación cuantitativa

El enfoque adoptado es **cuantitativo**, ya que la totalidad de las variables involucradas —utilización del enlace, latencia, pérdida de paquetes (Sección 2.1.4)— se expresan como magnitudes numéricas susceptibles de medición directa. El análisis de los resultados se apoya estrictamente en las métricas estadísticas de error (RMSE, MAE, MAPE) y en las métricas de clasificación (Precisión, Exhaustividad, *F1-Score*) definidas y fundamentadas teóricamente en el Capítulo 2. No se contempla, en ningún punto del desarrollo de esta tesis, la recolección o el análisis de datos de naturaleza cualitativa.

Según su diseño: mixto secuencial (observacional y experimental)

Desde la perspectiva de su diseño, esta investigación adopta un enfoque **mixto secuencial**, compuesto por dos fases distintas que reflejan la dualidad del problema abordado:

En su primera fase (Recolección de datos), el diseño es predominantemente **no experimental y longitudinal**. Se observa y registra el comportamiento de la red a lo largo de un periodo extendido de 30 días para capturar la evolución y estacionalidad del tráfico. Si bien se emplea instrumentación híbrida —incluyendo sondeos activos ligeros (ICMP) como se documentará en la Sección 4.4—, no se manipulan artificialmente las cargas de trabajo de los usuarios ni se induce deliberadamente la congestión en la infraestructura de la red privada bajo estudio.

En su segunda fase (Modelado predictivo), el diseño transita hacia un enfoque **experimental computacional**. El desarrollo, entrenamiento y evaluación de los modelos de *Machine Learning* (ARIMA, *Random Forest*, LSTM/GRU) requiere la manipulación sistemática y controlada de variables independientes —tales como hiperparámetros, arquitecturas de red y ventanas de rezago (*lags*)— para medir su efecto sobre las variables dependientes, representadas por las métricas de error predictivo. Esta fase se ejecuta en un entorno de trabajo aislado, garantizando que la experimentación algorítmica no impacte la red de producción.

Según su alcance: comparativo y explicativo

Finalmente, esta investigación posee un alcance **comparativo**, en tanto que su objetivo central no se limita a describir el comportamiento del tráfico de red, sino a contrastar

sistemáticamente el desempeño de tres paradigmas de modelado predictivo —estadístico, *Machine Learning* clásico y *Deep Learning*— evaluados bajo las mismas condiciones experimentales, tal como se estableció en la brecha metodológica identificada en la Sección 3.5. Este alcance comparativo se complementa con un componente **explicativo**, en la medida en que no basta con determinar *cuál* modelo ofrece el mejor desempeño, sino que se busca argumentar *por qué* dicho modelo se desempeña mejor —por ejemplo, si su ventaja se debe a la naturaleza no lineal del tráfico, a la presencia de dependencias temporales de largo plazo, o a la escala del conjunto de datos empleado—, apoyándose para ello en los fundamentos teóricos desarrollados en el Capítulo 2 y en la evidencia comparativa documentada en el Capítulo 3.

La Tabla 4.1 sintetiza esta clasificación de manera esquemática.

Dimensión	**Clasificación**	**Justificación breve**
Finalidad	Aplicada / tecnológica	Aplica técnicas de *Machine Learning* existentes a un problema práctico de redes.
Enfoque	Cuantitativa	Variables numéricas; análisis estadístico de error.
Diseño	No experimental, longitudinal	Observación pasiva de la red durante 30 días, sin manipulación.
Alcance	Comparativo-explicativo	Contrasta tres paradigmas de modelado y explica sus diferencias de desempeño.

Cuadro 4.1: Clasificación del tipo de investigación de esta tesis.

4.2. Definición operativa de congestión

De acuerdo con la Ecuación 2.2 establecida en el Marco Teórico, la operacionalización de la variable objetivo exige definir valores numéricos precisos para el umbral crítico de utilización (U_{th}) y la ventana temporal sostenida (Δt). Esta sección establece los parámetros base que gobernarán el etiquetado (*labeling*) del conjunto de datos antes de su ingesta en los modelos de *Machine Learning*.

Justificación del umbral U_{th}

Se adopta como valor inicial $U_{th} = 0{,}80$ (80 % de utilización), por dos razones convergentes. En primer lugar, desde la perspectiva teórica desarrollada en la Sección 2.1.3, la teoría de colas aplicada a redes de datos establece que el tiempo de espera promedio en un sistema de colas crece de manera proporcional a $\frac{1}{1-\rho}$, donde ρ representa la utilización del recurso [6]. Esta expresión matemática diverge conforme $\rho \to 1$, y su crecimiento deja de ser marginal precisamente en la vecindad de $\rho = 0{,}8$, punto a partir del cual pequeños incrementos adicionales en la utilización producen incrementos desproporcionadamente grandes en el tiempo de encolado. En segundo lugar, desde la perspectiva práctica de la industria, un umbral de utilización entre el 70 % y el 80 % constituye una referencia ampliamente adoptada en la

operación de redes empresariales para la activación de mecanismos de gestión de colas y alertamiento temprano; un criterio que se refleja, por ejemplo, en los valores por defecto de mecanismos de descarte preventivo como WRED (*Weighted Random Early Detection*) en equipos comerciales, los cuales suelen configurarse con umbrales iniciales de descarte alrededor del 80 % de ocupación del *buffer*.

Este valor de $U_{th} = 0{,}80$ funge como parámetro de control inicial (línea base) derivado de la literatura. Sin embargo, para garantizar que el modelo predictivo se ajuste a la realidad topológica de la red bajo estudio, este umbral será sometido a validación empírica durante el Análisis Exploratorio de Datos. El valor definitivo para el etiquetado del algoritmo de clasificación se fijará matemáticamente identificando el percentil exacto donde la distribución de utilización histórica comience a correlacionarse con fluctuaciones anómalas en la red.

Justificación de la ventana temporal Δt

En congruencia con la granularidad de muestreo establecida para la recolección de datos (Sección 4.4), se define $\Delta t = 5$ minutos como el intervalo mínimo sostenido durante el cual la condición $U(t) \geq U_{th}$ debe cumplirse para que un periodo se etiquete como congestión. Esta ventana, equivalente a cinco muestras consecutivas bajo una granularidad de muestreo de un minuto, responde a un balance deliberado entre dos riesgos opuestos: una ventana demasiado corta (por ejemplo, una única muestra) resulta susceptible a ráfagas instantáneas y transitorias de tráfico que no representan una congestión sostenida ni operacionalmente relevante —generando así falsos positivos que degradarían la utilidad práctica del sistema de predicción—; mientras que una ventana demasiado larga podría pasar por alto episodios de saturación breves pero significativos para aplicaciones sensibles a la latencia, como las cámaras de videovigilancia o las videollamadas mencionadas en la Sección 4.4. Al igual que con U_{th}, este valor podrá ser ajustado tras el análisis exploratorio del *dataset* primario si la evidencia empírica así lo sugiere.

Formulación instanciada para la etapa de preprocesamiento

Tomando como base los parámetros iniciales, la lógica de etiquetado binario que se programará en el *pipeline* de preprocesamiento de datos se define como:

$$\text{Congestión} \iff U(t) \geq 0{,}80 \quad \text{durante al menos 5 minutos consecutivos} \tag{4.1}$$

4.3. Descripción de las métricas a utilizar

Habiendo definido en la Sección 2.1.4 el fundamento teórico de las métricas de rendimiento de red, y en la Sección 4.2 el criterio operativo de congestión, corresponde ahora precisar qué subconjunto específico de estas métricas se empleará como variables de entrada (*features*) de los modelos predictivos, cuál constituye la variable objetivo y qué variables adicionales derivadas se incorporarán para enriquecer la capacidad predictiva del sistema.

Variable objetivo

La variable objetivo (*target*) de esta tesis es, en todos los casos, el factor de utilización futuro del enlace, $U(t+h)$, donde h representa el horizonte de predicción. Dicha variable se emplea de dos formas distintas según el paradigma de modelado, conforme a lo establecido en la Sección 2.2.1:

- Como variable **continua** (tarea de regresión), para los modelos ARIMA, *Random Forest*, SVR y LSTM en su formulación de regresión.
- Como variable **binaria derivada** (tarea de clasificación), aplicando la Ecuación 4.1 sobre $U(t+h)$, para la formulación de clasificación de los mismos modelos.

Variables de entrada (*features*)

La Tabla 4.2 enumera las variables de entrada consideradas para el entrenamiento de los modelos, clasificadas en tres grupos: métricas directas (obtenidas mediante los mecanismos de instrumentación descritos en la Sección 4.4), variables temporales derivadas (que capturan patrones cíclicos sin necesidad de instrumentación adicional) y variables de rezago o ventana (que explicitan la dependencia temporal de la serie, necesaria para modelos que no procesan secuencias de manera nativa, como *Random Forest*).

Grupo	**Variable**	**Tipo**	**Unidad / Rango**	**Fuente**
Métricas directas	Utilización $U(t)$	Continua	Adimensional [0,1]	SNMP
	Latencia (RTT)	Continua	ms	ICMP
	Pérdida de paquetes	Continua	%	SNMP / ICMP
Temporales derivadas	Hora del día	Categórica cíclica	0–23 (seno/coseno)	*Timestamp*
	Día de la semana	Categórica cíclica	0–6 (seno/coseno)	*Timestamp*
	Indicador fin de semana	Binaria	{0, 1}	*Timestamp*
	Indicador horario laboral	Binaria	{0, 1}	*Timestamp*
Rezago / ventana	$U(t-1), U(t-2), \ldots, U(t-k)$	Continua	Adimensional [0,1]	Derivada de $U(t)$
	Media móvil de U (15 min)	Continua	Adimensional [0,1]	Derivada de $U(t)$

Cuadro 4.2: Variables de entrada consideradas para los modelos predictivos.

Conviene justificar dos decisiones reflejadas en la Tabla 4.2. Primero, las variables temporales cíclicas (hora del día, día de la semana) se codifican mediante transformaciones seno y coseno en lugar de emplearse directamente como enteros numéricos. Esto se realiza con el

fin de preservar su naturaleza cíclica, evitando, por ejemplo, que el modelo interprete que la hora 23 y la hora 0 se encuentran algebraicamente lejanas entre sí, cuando en la realidad temporal son consecutivas:

$$x_{\text{seno}} = \sin\left(\frac{2\pi \cdot x}{x_{\text{máx}}}\right), \qquad x_{\text{coseno}} = \cos\left(\frac{2\pi \cdot x}{x_{\text{máx}}}\right) \tag{4.2}$$

Segundo, las variables de rezago ($U(t-1)$ hasta $U(t-k)$) resultan indispensables para los modelos que no poseen una noción arquitectónica intrínseca de secuencia temporal (*Random Forest* y SVR), ya que estos algoritmos tratan cada observación como independiente y requieren que la dependencia temporal se proporcione de manera explícita en el vector de entrada. En contraste, la arquitectura LSTM procesa la secuencia de observaciones de manera nativa, por lo que estas variables de rezago, si bien pueden incluirse, no resultan estrictamente necesarias para dichos modelos.

Disponibilidad de métricas por fuente de datos

Dado que esta tesis emplea dos fuentes de datos de naturaleza distinta (Sección 4.4), no todas las métricas de la Tabla 4.2 se encuentran disponibles de la misma forma en ambos casos. La Tabla 4.3 contrasta la disponibilidad y el método de obtención de cada métrica en el *dataset* primario frente al *dataset* de validación (MAWI).

Métrica	***Dataset* primario**	***Dataset* de validación (MAWI)**
Utilización	Directa, vía SNMP en tiempo real	Derivada de trazas `.pcap` agregando bytes por ventana de tiempo
Latencia (RTT)	Directa, vía sondas ICMP activas	No disponible directamente; se aproxima mediante el tiempo entre paquetes de un flujo TCP
Pérdida de paquetes	Directa, vía contadores SNMP de descarte	Estimada indirectamente mediante retransmisiones TCP detectadas en la traza
Variables temporales	Directas, *timestamp* del muestreo	Directas, *timestamp* de captura de cada traza diaria

Cuadro 4.3: Disponibilidad y método de obtención de métricas según la fuente de datos.

Esta asimetría no invalida el enfoque híbrido adoptado, pero sí implica que, para el *dataset* de validación, algunas variables de entrada requieren un paso adicional de derivación durante el preprocesamiento, el cual se detalla en la Sección 4.5. Es fundamental advertir que esta diferencia en los métodos de recolección —particularmente entre la latencia limpia obtenida mediante sondeos ICMP en la red privada y la latencia ruidosa inferida a través de flujos TCP

en el archivo MAWI— introduce un desafío estadístico conocido como cambio de dominio (*domain shift*).

Si los algoritmos de *Machine Learning* asimilan directamente las magnitudes crudas de estas métricas, el desempeño predictivo colapsará durante la fase de validación debido al desajuste de escalas y distribuciones. Para mitigar esta divergencia estructural, se aplicará un proceso de escalado y normalización independiente (por ejemplo, mediante *StandardScaler* o *MinMaxScaler*) a cada conjunto de datos de manera aislada. Esta transformación, formalizada en la etapa de preprocesamiento (Sección 4.5), garantiza que los modelos aprendan la varianza relativa y los patrones de degradación de las métricas, independientemente de la magnitud absoluta dictada por las herramientas de monitoreo subyacentes.

Pipeline de construcción de variables

La Figura 4.1 ilustra esquemáticamente el flujo mediante el cual las métricas crudas, obtenidas directamente de la instrumentación descrita en la Sección 4.4, se transforman en el vector final de variables de entrada empleado por cada modelo.

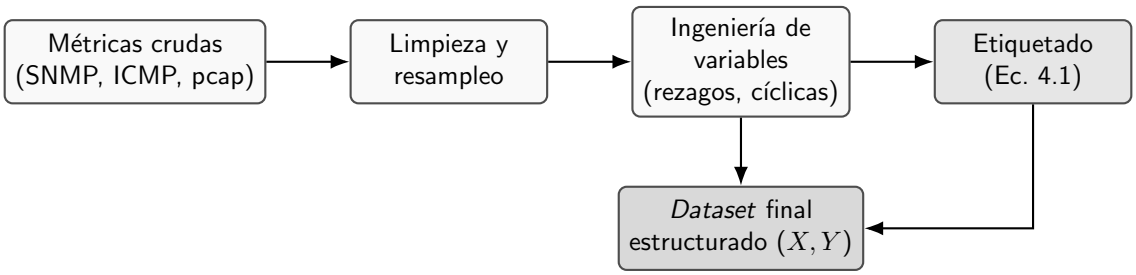


Figura 4.1: *Pipeline* de transformación de métricas crudas en variables de entrada y variable objetivo.

Este *pipeline* se aplica de manera independiente sobre cada una de las dos fuentes de datos, garantizando que el vector final de variables tenga una estructura equivalente entre ambas, condición necesaria para poder evaluar un mismo modelo entrenado sobre el *dataset* primario contra el conjunto de validación MAWI, tal como se planteó en el diseño comparativo de esta tesis (Sección 3.5). El detalle específico de cada etapa de este flujo —en particular, la limpieza de datos faltantes y el resampleo de las trazas MAWI— se desarrolla con mayor profundidad en la Sección 4.5 (Preprocesamiento de datos).

4.4. Fuente de datos

Para el desarrollo de esta tesis se adoptó una estrategia híbrida de recolección de datos, combinando una fuente primaria de tráfico real capturado de forma pasiva en una red administrada directamente por el autor, con una fuente secundaria de validación externa proveniente de un archivo académico de tráfico de Internet ampliamente reconocido. Esta combinación busca equilibrar dos necesidades que, de manera aislada, ningún *dataset* por sí solo podría satisfacer plenamente: por un lado, la posibilidad de definir con precisión el proceso de recolección, las métricas capturadas y el contexto operativo del enlace bajo estudio;

y por otro, la necesidad de validar que los modelos desarrollados no se sobreajusten a las particularidades de un único escenario, sino que sean capaces de generalizar su capacidad predictiva a condiciones de tráfico distintas en naturaleza y escala.

Dataset primario: Red privada PYME

La fuente primaria de datos proviene de una red privada PYME sobre la cual el autor tiene acceso administrativo completo, en el contexto de un proyecto de despliegue de infraestructura. Dicha red opera bajo un enrutador MikroTik que concentra y gestiona simultáneamente múltiples servicios que compiten por el mismo enlace de salida a Internet. Es fundamental delimitar la topología física de este acceso: el tráfico hacia el exterior se enruta a través de una única interfaz WAN (como un enlace LTE), lo cual condiciona intrínsecamente el perfil base de latencia, *jitter* y variabilidad estocástica del canal. Sobre esta infraestructura coexisten un sistema de videovigilancia con transmisión continua (NVR), tráfico de uso corporativo/residencial cotidiano, y un servicio de almacenamiento en la nube (Nextcloud) accesible de forma remota mediante una VPN SSTP.

Esta arquitectura de servicios genera un escenario de estrés operativo altamente propicio para la investigación de congestión. En particular, la encapsulación de tráfico TCP (como el generado por Nextcloud) dentro de un túnel VPN que también opera sobre TCP (SSTP) introduce una susceptibilidad arquitectónica al fenómeno conocido como *TCP Meltdown*. Bajo esta condición, si el enlace físico experimenta una ligera pérdida de paquetes por saturación, tanto la capa de aplicación como la capa del túnel intentan retransmitir la información de manera simultánea y descoordinada. Esta redundancia de retransmisiones amplifica la congestión original, colapsando rápidamente la capacidad del enlace mediante un patrón no lineal. La captura de estas dinámicas complejas convierte a este conjunto de datos en un entorno de validación excepcional para evaluar si los modelos de *Machine Learning* logran anticipar el colapso con mayor precisión que los modelos estadísticos tradicionales.

Es importante señalar que, dado que los datos provienen de la infraestructura de un tercero, se obtuvo la autorización explícita del propietario de la red para realizar el monitoreo pasivo de metadatos de tráfico con fines exclusivamente académicos, en línea con las buenas prácticas de investigación en redes de computadoras que involucran datos de operación reales [25].

Dataset secundario: MAWI Working Group Traffic Archive

Como fuente de validación externa se empleará el **MAWI Working Group Traffic Archive**, un repositorio de trazas de tráfico mantenido de forma continua desde 2006 por el proyecto WIDE, que captura de manera diaria el tráfico del enlace de tránsito entre la red académica WIDE y su proveedor ascendente en Tokio, Japón [23]. A diferencia del *dataset* primario, que corresponde a una red de escala reducida con un puñado de servicios concurrentes, las trazas de MAWI capturan tráfico agregado de un enlace de tránsito de gran escala (entre 1 y 10 Gbps), compuesto por miles de flujos simultáneos provenientes de múltiples organizaciones y aplicaciones. Las direcciones IP contenidas en estas trazas se encuentran anonimizadas mediante un esquema que preserva la estructura de prefijos de red (*prefix-preserving anonymization*, empleando el algoritmo Crypto-PAn), lo cual elimina cualquier

posibilidad de identificar usuarios individuales sin comprometer la utilidad analítica de los datos para el estudio de patrones agregados de tráfico.

La incorporación de este segundo *dataset* responde a una limitación reconocida en la literatura sobre predicción de tráfico de red: los modelos entrenados y evaluados sobre una única fuente de datos tienden a mostrar un desempeño optimista que no necesariamente se sostiene cuando se exponen a condiciones de tráfico distintas a las observadas durante el entrenamiento [3]. Al validar los modelos desarrollados en esta tesis sobre las trazas de MAWI —de una naturaleza y escala sustancialmente distinta a la del *dataset* primario—, es posible obtener evidencia más robusta sobre la capacidad de generalización de los enfoques de *Machine Learning* propuestos, más allá de las particularidades de un único escenario de red.

Dado que las trazas de MAWI se distribuyen en formato de captura de paquetes (`.pcap`) y no como series temporales de utilización previamente agregadas, fue necesario un procesamiento adicional para derivar de ellas las mismas métricas empleadas en el *dataset* primario. Este procesamiento se describe con mayor detalle en la Sección 4.5 (Preprocesamiento de datos).

Consideraciones éticas y de privacidad

En ambos casos, la recolección y el procesamiento de los datos se rigen por un principio de minimización: se capturan y almacenan únicamente los metadatos estrictamente necesarios para el análisis de utilización, latencia y pérdida de paquetes, sin acceder en ningún momento al contenido de las comunicaciones transportadas. En el caso del *dataset* primario, se cuenta con la autorización expresa del propietario de la infraestructura; en el caso de MAWI, el uso de los datos se limita a fines de investigación, en estricto cumplimiento con los términos de uso establecidos por el propio archivo, los cuales restringen su empleo a propósitos académicos y prohíben cualquier acción que comprometa la privacidad de los usuarios cuyo tráfico fue originalmente capturado.

4.5. Preprocesamiento de datos

El *pipeline* esquematizado en la Figura 4.1 (Sección 4.3) identificó cuatro grandes etapas de transformación entre las métricas crudas y el vector final de variables de entrada. Esta sección detalla técnicamente cada una de dichas etapas, prestando especial atención a los puntos en los que el tratamiento del *dataset* primario y el *dataset* de validación (MAWI) difiere, dada la naturaleza distinta de ambas fuentes descrita en la Sección 4.4.

Limpieza de datos

La limpieza de datos aborda dos problemas recurrentes en la recolección de series temporales de red: valores faltantes y valores atípicos (*outliers*).

Los **valores faltantes** pueden originarse por interrupciones temporales del proceso de monitoreo, fallos de conectividad hacia el propio dispositivo de consulta SNMP, o reinicios del equipo de red. Para intervalos faltantes de corta duración (hasta 3 muestras consecutivas,

equivalente a 3 minutos bajo la granularidad establecida), se aplicará interpolación lineal entre el último valor observado y el siguiente valor disponible. Para intervalos de mayor duración, el criterio adoptado es descartar dicho segmento de la serie por completo en lugar de imputarlo, ya que una interpolación extendida introduciría patrones artificiales que no reflejan el comportamiento real de la red, contaminando potencialmente el proceso de entrenamiento de los modelos.

Los **valores atípicos** se identifican mediante el criterio del rango intercuartílico (IQR), marcando como atípico cualquier observación que se encuentre fuera del rango:

$$[Q_1 - 1{,}5 \cdot \text{IQR},\ Q_3 + 1{,}5 \cdot \text{IQR}] \tag{4.3}$$

donde Q_1 y Q_3 son el primer y tercer cuartil de la variable analizada, e $\text{IQR} = Q_3 - Q_1$. Es importante señalar que, a diferencia de otros contextos de análisis de datos donde los atípicos se eliminan sistemáticamente, en esta tesis dichos valores **no se descartan de manera automática**: un pico súbito y extremo de utilización podría constituir precisamente el tipo de evento de congestión que se busca predecir, y no un error de medición. Por ello, cada valor marcado como atípico se contrasta manualmente contra las otras métricas disponibles en el mismo instante (por ejemplo, si un pico de utilización coincide con un aumento simultáneo de latencia y pérdida de paquetes, se conserva como evento genuino; si ocurre de manera aislada y sin correlato en las demás métricas, se trata como un posible error de instrumentación y se interpola).

Resampleo y agregación temporal

Como se documentó en la Tabla 4.3 (Sección 4.3), el *dataset* primario se obtiene ya en forma de serie temporal muestreada directamente a la granularidad deseada (1 minuto), mientras que el *dataset* MAWI se distribuye como capturas de paquetes individuales (`.pcap`). Por tanto, este último requiere un proceso adicional de **agregación temporal**: mediante herramientas de análisis de tráfico (`tshark` / `tstat`), los paquetes capturados se agrupan en ventanas no traslapadas de 1 minuto, calculando para cada ventana el total de bytes transmitidos, a partir de lo cual se deriva la utilización $U(t)$ mediante la Ecuación 2.3, empleando como ancho de banda nominal la capacidad conocida del enlace de tránsito capturado por MAWI.

La Tabla 4.4 resume este contraste entre ambas fuentes.

Aspecto	***Dataset* primario**	***Dataset* de validación (MAWI)**
Formato original	Serie temporal (SNMP/ICMP)	Captura de paquetes (`.pcap`)
Granularidad original	1 minuto (ya muestreada)	Por paquete individual (microsegundos)
Proceso requerido	Ninguno (uso directo)	Agregación mediante `tshark` / `tstat`
Granularidad final	1 minuto	1 minuto (tras agregación)

Cuadro 4.4: Contraste del proceso de resampleo entre las dos fuentes de datos empleadas.

Normalización de variables

Dado que los modelos de *Deep Learning* (LSTM) son sensibles a la escala de las variables de entrada —un gradiente calculado sobre variables de magnitudes muy dispares puede converger de manera inestable durante el entrenamiento—, todas las variables continuas de la Tabla 4.2 se normalizan mediante escalamiento Min-Max al rango $[0, 1]$:

$$x_{\text{norm}} = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}} \tag{4.4}$$

Se opta por escalamiento Min-Max en lugar de estandarización Z-score porque la variable de mayor interés, la utilización $U(t)$, posee un rango natural acotado $[0, 1]$ (Sección 2.1.3). Un punto metodológico crítico: para el *dataset* primario, los valores $x_{\text{mín}}$ y $x_{\text{máx}}$ se calculan **estrictamente** a partir del conjunto de entrenamiento, aplicándose posteriormente sobre los conjuntos de validación y prueba para evitar la fuga de información (*data leakage*).

Sin embargo, como se advirtió en la Sección 4.3, el *dataset* de validación externa (MAWI) presenta un cambio de dominio (*domain shift*) significativo, particularmente en las distribuciones de latencia inferida. Por consiguiente, MAWI es sometido a un proceso de escalado independiente, ajustando sus propios parámetros de normalización para homologar las varianzas relativas antes de someterlo a los modelos preentrenados, garantizando así una evaluación justa de la capacidad de generalización algorítmica.

Ventaneo y construcción de rezagos (*Lags*)

Para que los algoritmos de aprendizaje automático puedan predecir a partir de series temporales, es necesario transformar la serie lineal en conjuntos de variables estructuradas mediante una **ventana deslizante** (*sliding window*). Este proceso aplica para todos los modelos, pero difiere en la topología de los datos generados:

Para los modelos clásicos (*Random Forest*), que carecen de memoria interna (Sección 2.2.3), la ventana aplana el historial temporal en un vector tabular de características (2D), donde cada observación pasada se convierte en una variable independiente. Por el contrario, para la arquitectura LSTM (Sección 2.2.4), el ventaneo preserva la estructura tridimensional

(3D) de secuencias temporales que la red recurrente consume de manera nativa. Dada una longitud de ventana w, cada bloque de entrada se construye como:

$$X_t = [U(t-w), U(t-w+1), \ldots, U(t-1)] \longrightarrow y_t = U(t+h) \tag{4.5}$$

donde w es el número de observaciones pasadas consideradas (análogo, aunque no idéntico en propósito, al orden p del modelo ARIMA presentado en la Sección 2.2.2) y h es el horizonte de predicción. La Figura 4.2 ilustra este proceso de manera esquemática.

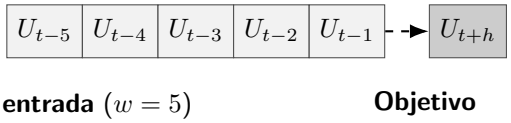


Figura 4.2: Construcción de secuencias de entrada mediante ventana deslizante para los modelos LSTM/GRU.

El valor específico de w (longitud de la ventana) y h (horizonte de predicción) se determinará experimentalmente en el Capítulo 5, evaluando el desempeño del modelo bajo distintas configuraciones, si bien como punto de partida se contemplan ventanas equivalentes a periodos de 30 a 60 minutos de historia (es decir, $w = 30$ a 60 bajo la granularidad de 1 minuto establecida), y horizontes de predicción de $h = 5$ a 15 minutos, congruentes con el propósito de alertamiento temprano planteado en el Capítulo 1.

División de datos: entrenamiento, validación y prueba

Un error metodológico común al trabajar con series temporales —y que esta tesis evita deliberadamente— es dividir los datos en conjuntos de entrenamiento y prueba mediante muestreo aleatorio, tal como se acostumbra en problemas de aprendizaje supervisado convencionales. Este enfoque resulta inadecuado en el presente contexto porque violaría el orden temporal de la serie: permitiría, por ejemplo, que el modelo se entrene con datos de un instante posterior y sea evaluado sobre un instante anterior, lo cual no solo carece de sentido práctico —un sistema de predicción real jamás tendría acceso a datos del futuro—, sino que produciría estimaciones de desempeño artificialmente optimistas por fuga de información temporal.

Por esta razón, la división de los datos se realiza de manera estrictamente **cronológica**, tal como se ilustra en la Figura 4.3: el segmento más antiguo de la serie se destina a entrenamiento, el segmento intermedio a validación (empleado para el ajuste de hiperparámetros) y el segmento más reciente a prueba (empleado exclusivamente para la evaluación final reportada en el Capítulo 6).

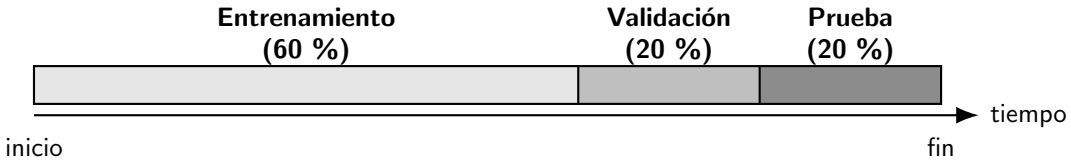


Figura 4.3: División cronológica del *dataset* primario en conjuntos de entrenamiento, validación y prueba.

Esta división se aplica de manera independiente sobre el *dataset* primario (proporciones 60/20/20 sobre los 30 días de recolección continua). El *dataset* MAWI, en cambio, se emplea íntegramente como un segundo conjunto de prueba externo —no se divide en entrenamiento, validación y prueba—, ya que su función metodológica, como se estableció en la Sección 4.4, es evaluar la capacidad de generalización de los modelos ya entrenados sobre el *dataset* primario, y no proveer datos adicionales de entrenamiento.

La Tabla 4.5 sintetiza las cinco etapas de preprocesamiento descritas en esta sección, sirviendo como cierre de este apartado y transición hacia la implementación concreta de los modelos, que se abordará en el Capítulo 5.

Etapa	**Descripción**
1. Limpieza	Interpolación de faltantes cortos; tratamiento contextual de atípicos (Ec. 4.3).
2. Resampleo	Agregación de MAWI a ventanas de 1 min; uso directo del *dataset* primario.
3. Normalización	Escalamiento Min-Max (Ec. 4.4), ajustado solo sobre entrenamiento.
4. Ventaneo	Construcción de secuencias para LSTM (Ec. 4.5).
5. División temporal	60/20/20 cronológico sobre el *dataset* primario; MAWI como prueba externa.

Cuadro 4.5: Resumen de las etapas de preprocesamiento de datos.

4.6. Modelos a implementar y justificación de su elección

Con base en los tres paradigmas de modelado presentados en el Marco Teórico (Sección 2.2) y en la brecha metodológica identificada en la síntesis del Estado del Arte (Sección 3.5) —la ausencia, en la literatura revisada, de una comparación simultánea entre modelos estadísticos, de *Machine Learning* clásico y de *Deep Learning* bajo condiciones experimentales controladas—, esta tesis contempla la implementación y evaluación comparativa de tres modelos específicos: ARIMA, *Random Forest* y LSTM. Esta sección justifica la elección de cada uno, así como la exclusión deliberada de otras alternativas que un lector familiarizado con el campo podría razonablemente esperar encontrar aquí.

ARIMA: modelo de referencia (*baseline*)

ARIMA se incluye no como un candidato a ofrecer el mejor desempeño absoluto, sino como **línea base de referencia** contra la cual contrastar el desempeño de los modelos subsiguientes, conforme a su rol descrito en la Sección 2.2.2. Su justificación no depende de una expectativa de alto desempeño, sino de un principio metodológico: sin un punto de comparación estadístico simple, cualquier ganancia de desempeño reportada por los modelos de ML clásico o *Deep Learning* carecería de un marco de referencia contra el cual dimensionar su magnitud real. Adicionalmente, como se documentó en la Sección 3.1, ARIMA y sus variantes continúan siendo objeto de investigación vigente en la literatura reciente [16], lo que respalda su pertinencia como punto de comparación actual y no meramente histórico.

Random Forest: *Machine Learning* clásico

La elección de *Random Forest* se fundamenta directamente en la evidencia reportada por Sneha et al. [17] (Sección 3.2), quienes encontraron que este modelo ofreció el mejor desempeño entre cuatro algoritmos clásicos evaluados para la predicción de congestión a nivel de enrutador. A esta evidencia empírica se suma una ventaja metodológica relevante para esta tesis: la capacidad de *Random Forest* de calcular la importancia relativa de cada variable de entrada (Sección 2.2.3), lo cual permitirá —una vez ejecutado el entrenamiento en el Capítulo 5— determinar cuáles de las variables descritas en la Tabla 4.2 (Sección 4.3) resultan más informativas para anticipar la congestión, aportando así evidencia adicional más allá de la mera precisión predictiva.

LSTM: *Deep Learning*

La justificación de este modelo se apoya directamente en el trabajo de Santos et al. [22] (Sección 3.3), quienes realizaron la comparación metodológicamente más cercana a la planteada en esta tesis —LSTM contra GRU, sobre tráfico de Internet real, con un periodo de recolección del mismo orden de magnitud que el establecido para el *dataset* primario de esta investigación (Sección 4.4)—. En su estudio, los autores demostraron empíricamente que la arquitectura LSTM superó en desempeño predictivo a GRU en la mayoría de los agrupamientos y escenarios geográficos analizados. Con base en esta evidencia, esta tesis adopta LSTM como el representante óptimo y único del paradigma de redes neuronales recurrentes, destinando los recursos computacionales a la sintonización exhaustiva de este modelo en lugar de duplicar esfuerzos en arquitecturas redundantes.

Modelos considerados y no incluidos

Es previsible que, ante la elección de los tres modelos anteriores, surja la pregunta de por qué no se incluyeron otras alternativas frecuentemente mencionadas en la literatura de predicción de series temporales. La Tabla 4.6 explicita esta decisión y su justificación.

Modelo no incluido	Justificación de exclusión
XGBoost	Si bien supera frecuentemente a *Random Forest*, XGBoost se basa en *Boosting* (reducción secuencial de sesgo), lo que lo hace altamente susceptible al sobreajuste (*overfitting*) en conjuntos de datos pequeños o con métricas ruidosas (como la latencia TCP inferida), a menos que se realice una sintonización de hiperparámetros exhaustiva. Se optó por *Random Forest* (*Bagging*) por su mayor robustez inherente al ruido operativo.
SVR y GRU	Excluidos para evitar redundancia paradigmática. SVR pertenece a la misma familia de ML clásico que *Random Forest*, y GRU demostró ser inferior a LSTM en literatura similar [22]. La metodología priorizó optimizar a fondo los tres mejores representantes de cada enfoque en lugar de implementar múltiples variantes del mismo tipo.
Prophet	Diseñado principalmente para series con estacionalidad de negocio (ventas, demanda), con supuestos que no se ajustan al tráfico de red a granularidad de minutos.
Transformers	Su entrenamiento efectivo típicamente requiere volúmenes de datos considerablemente mayores a los disponibles en el *dataset* primario de esta tesis (30 días en red reducida).

Cuadro 4.6: Modelos considerados durante la planeación de esta tesis y justificación de su exclusión del alcance experimental.

Diseño experimental general

La Figura 4.4 sintetiza el diseño experimental completo que resulta de las decisiones justificadas en esta sección.

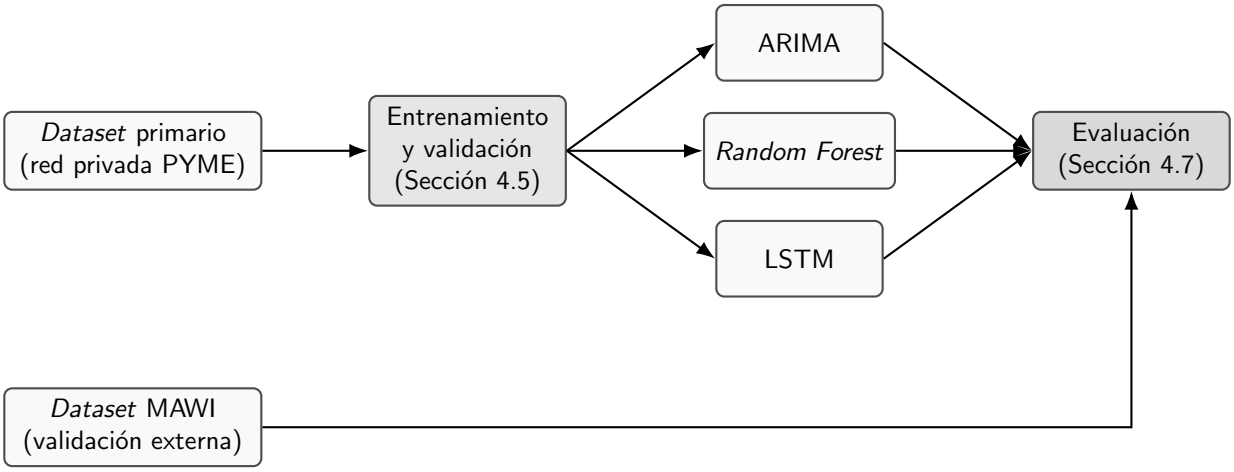


Figura 4.4: Diseño experimental general: entrenamiento sobre el *dataset* primario y validación cruzada contra MAWI.

Es importante precisar una distinción operativa: mientras que *Random Forest* y LSTM pueden entrenarse explícitamente para clasificar, ARIMA operará estrictamente como regresor continuo, realizando la clasificación de manera derivada aplicando el umbral U_{th} (Ecuación 4.1). Las afirmaciones sobre qué modelo podría desempeñarse mejor constituyen hipótesis fundamentadas en la literatura y se pondrán a prueba empírica en los Capítulos 5 y 6.

4.7. Métricas de evaluación del modelo

Como se estableció en la Sección 4.3, el problema de predicción de congestión abordado en esta tesis se formula bajo dos paradigmas complementarios: regresión (predicción del valor continuo $U(t+h)$) y clasificación (predicción binaria de congestión mediante la Ecuación 4.1). En congruencia con esta doble formulación, esta sección define dos conjuntos de métricas de evaluación independientes, así como los criterios bajo los cuales se compararán los cinco modelos justificados en la Sección 4.6.

Métricas para la tarea de regresión

Para evaluar el desempeño de los modelos en su formulación de regresión se emplearán las tres métricas presentadas formalmente en el Capítulo 2: RMSE (Ecuación 2.15), MAE (Ecuación 2.16) y MAPE (Ecuación 2.17). La Tabla 4.7 resume el propósito específico que cada una cumple dentro del análisis comparativo de esta tesis, evitando así que su inclusión conjunta se perciba como una simple acumulación de métricas sin criterio.

Métrica	Sensibilidad	Rol en esta tesis
RMSE	Penaliza fuertemente errores grandes	Detectar modelos que fallan de forma severa ante picos de congestión.
MAE	Pondera todos los errores por igual	Interpretación directa del error típico esperado, en las mismas unidades que U.
MAPE	Expresa el error como porcentaje relativo	Comparar el desempeño de un mismo modelo entre el *dataset* primario y MAWI, a pesar de operar a escalas de tráfico distintas.

Cuadro 4.7: Rol de cada métrica de regresión en el análisis comparativo de esta tesis.

Métricas para la tarea de clasificación

Para la formulación de clasificación binaria (congestión / no congestión), las métricas de error continuo de la subsección anterior resultan insuficientes, ya que no capturan directamente el tipo de aciertos y errores más relevantes para un sistema de alertamiento. Se requiere, en su lugar, un conjunto de métricas derivadas de la **matriz de confusión**, ilustrada en la Figura 4.5, que clasifica cada predicción en una de cuatro categorías según la coincidencia entre el valor predicho y el valor real observado.

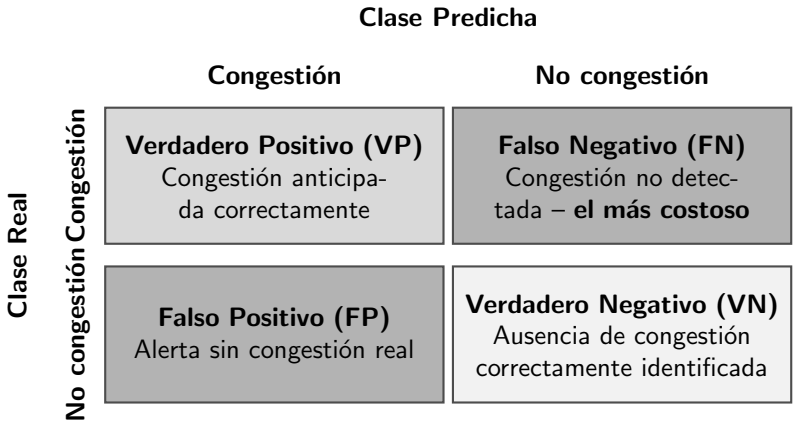


Figura 4.5: Matriz de confusión para la tarea de clasificación de congestión.

A partir de estas cuatro cantidades se calculan la **Exactitud**, **Precisión**, **Exhaustividad (*Recall*)** y el ***F1-Score***, cuyas formulaciones matemáticas ya fueron establecidas rigurosamente en el Marco Teórico (Capítulo 2).

Es necesario justificar, en el contexto específico de esta tesis, por qué estas métricas no deben ponderarse de manera equivalente. Como se señala en la Figura 4.5, un **falso negativo**

—un episodio real de congestión que el modelo no logra anticipar— resulta considerablemente más costoso, desde la perspectiva del problema planteado en el Capítulo 1, que un **falso positivo** —una alerta emitida ante una situación que finalmente no deriva en congestión—. Esto se debe a que un falso positivo implica, en el peor de los casos, una revisión innecesaria por parte de un administrador de red o una medida preventiva no estrictamente requerida, mientras que un falso negativo implica exactamente el escenario que esta tesis busca evitar: la degradación del servicio sin ningún tipo de anticipación, replicando la limitación reactiva señalada en el Planteamiento del Problema (Sección 1.1). En consecuencia, esta tesis prioriza la **exhaustividad (*Recall*)** sobre la exactitud general como criterio principal de comparación entre modelos, sin descuidar la precisión, para lo cual el ***F1-score*** —que balancea ambas— se reporta como métrica de síntesis. Adicionalmente, dado que la exactitud simple puede resultar engañosa cuando la clase de interés (congestión) es minoritaria dentro del conjunto de datos —una condición previsible dado que se espera que la red bajo estudio opere por debajo del umbral U_{th} la mayor parte del tiempo—, esta métrica se reportará únicamente como referencia general y no como criterio determinante de comparación.

Complementariamente, se reportará también la curva ROC (*Receiver Operating Characteristic*) y el área bajo dicha curva (AUC), que permiten evaluar el desempeño del clasificador de manera independiente al umbral de decisión específico empleado, ofreciendo una visión más completa de su capacidad discriminativa entre los estados de congestión y no congestión.

Una métrica adicional propia de esta tesis: tiempo de anticipación efectivo

Más allá de las métricas convencionales descritas anteriormente, y en congruencia con el propósito de alertamiento temprano planteado desde el Capítulo 1, esta tesis propone evaluar una métrica adicional, definida específicamente para este trabajo: el **tiempo de anticipación efectivo** (TAE), que mide, para cada verdadero positivo correctamente detectado, cuántos minutos antes de que la congestión real ocurriera el modelo emitió la alerta correspondiente:

$$TAE_i = t_{\text{congestión real},i} - t_{\text{predicción},i} \tag{4.6}$$

Esta métrica no sustituye a las anteriores, sino que las complementa: un modelo podría, en principio, alcanzar valores aceptables de *Recall* y *F1-score* prediciendo la congestión con muy poca antelación (por ejemplo, un minuto antes de que ocurra), lo cual tendría escasa utilidad práctica para que un administrador de red o un sistema automatizado tome medidas correctivas a tiempo. El TAE promedio, reportado junto con su desviación estándar, permitirá evaluar si los horizontes de predicción h explorados en la Sección 4.5 (entre 5 y 15 minutos) resultan, en la práctica, suficientes para el propósito de anticipación planteado en esta investigación.

Síntesis del esquema de evaluación

La Tabla 4.8 sintetiza el esquema completo de evaluación que se aplicará de manera uniforme a los cinco modelos justificados en la Sección 4.6, tanto sobre el conjunto de prueba

del *dataset* primario como sobre el conjunto de validación externa MAWI, conforme al diseño experimental ilustrado en la Figura 4.4.

Formulación	**Métricas**	**Criterio principal**
Regresión	RMSE, MAE, MAPE	RMSE (penaliza errores severos)
Clasificación	Exactitud, Precisión, *Recall*, F1, AUC-ROC	*Recall*, sintetizado con F1
Utilidad práctica	Tiempo de Anticipación Efectivo (TAE)	Máximo TAE con *Recall* aceptable

Cuadro 4.8: Síntesis del esquema de evaluación aplicado a los modelos de esta tesis.

Al igual que en la Sección 4.6, es importante reiterar que las prioridades establecidas en esta sección —privilegiar *Recall* sobre exactitud, complementar con el TAE propuesto— constituyen decisiones de diseño metodológico justificadas por la naturaleza del problema planteado, y no resultados obtenidos. Los valores concretos que estas métricas arrojen para cada modelo se presentarán y discutirán en el Capítulo 6.

4.8. Herramientas y entorno

Esta sección final del capítulo de Metodología documenta el conjunto de herramientas de *software*, bibliotecas y el entorno de cómputo que se emplearán a lo largo del desarrollo experimental descrito en el Capítulo 5, organizadas según la etapa del *pipeline* metodológico a la que corresponden (Figura 4.1, Sección 4.3). Esta documentación responde a un criterio de reproducibilidad: cualquier lector con acceso a un entorno equivalente debería poder replicar, al menos en sus aspectos esenciales, el procedimiento aquí descrito.

Herramientas de recolección e instrumentación

Para la obtención de las métricas del *dataset* primario, descritas en la Sección 4.4, se empleará el conjunto de herramientas resumido en la Tabla 4.9, congruente con la estrategia de monitoreo híbrido establecida previamente.

Herramienta	Propósito	Métrica asociada
RouterOS (MikroTik)	Exposición de contadores SNMP nativos de la interfaz WAN	Utilización (Ec. 2.3)
Net-SNMP / `pysnmp`	Consulta periódica de contadores SNMP desde un *script* externo	Utilización, descartes
`ping` (ICMP)	Medición activa de latencia y pérdida de paquetes	RTT, pérdida (Ecs. 2.4, 2.7)
Script de recolección (Python)	Orquestación de las consultas anteriores y almacenamiento a intervalos de 1 minuto	Todas

Cuadro 4.9: Herramientas empleadas para la recolección del *dataset* primario.

Herramientas de procesamiento del dataset de validación

Para el procesamiento de las trazas del *dataset* MAWI (Sección 4.4), distribuidas en formato `.pcap`, se empleará `tshark` (la utilidad de línea de comandos de Wireshark) y/o `tstat`, herramientas especializadas en el análisis de capturas de tráfico, con el fin de realizar la agregación temporal descrita en la Sección 4.5 y derivar de ellas las métricas equivalentes a las obtenidas de forma directa en el *dataset* primario (Tabla 4.3).

Entorno de desarrollo y bibliotecas de *Machine Learning*

Todo el desarrollo del prototipo se realizará en el lenguaje de programación **Python**, dado su ecosistema maduro de bibliotecas especializadas en ciencia de datos y aprendizaje automático. La Tabla 4.10 detalla la biblioteca específica que se empleará para la implementación de cada uno de los cinco modelos justificados en la Sección 4.6, así como para las etapas de preprocesamiento y evaluación.

Etapa / Modelo	Biblioteca
Manipulación de datos	`pandas`, `numpy`
Preprocesamiento (Sección 4.5)	`scikit-learn` (`preprocessing`)
ARIMA	`statsmodels`
Random Forest	`scikit-learn`
LSTM	`TensorFlow` / `Keras`
Métricas de evaluación (Sección 4.7)	`scikit-learn` (`metrics`)
Visualización de resultados	`matplotlib`, `seaborn`

Cuadro 4.10: Bibliotecas de Python empleadas para la implementación de cada etapa metodológica.

Entorno de cómputo

En estricta congruencia con lo establecido en los Alcances y Limitaciones de la investigación (Sección **??**), el desarrollo algorítmico, la sintonización de hiperparámetros y el entrenamiento de los modelos se ejecutaron sobre una estación de trabajo convencional. La plataforma de modelado consistió en una estación de trabajo portátil Dell Precision 7540, provista de un procesador Intel Core i7-9850H (12 núcleos lógicos a 2.60 GHz), 32 GB de memoria RAM y sistema operativo Ubuntu 26.04.1 LTS.

Esta restricción de *hardware* se mantuvo deliberadamente a lo largo de toda la fase experimental para garantizar que la viabilidad del *pipeline* predictivo no dependa de clústeres de cómputo distribuido ni requiera aceleración especializada mediante unidades de procesamiento gráfico (GPU). Con ello se demuestra empíricamente que el sistema resultante es apto para ser desplegado en la infraestructura de propósito general típicamente disponible en una Red Privada Corporativa.

Control de versiones y documentación

Finalmente, el código fuente desarrollado para esta tesis se gestionará mediante un sistema de control de versiones (Git), con el propósito de mantener un historial reproducible de los experimentos realizados. Los fragmentos de código considerados más relevantes para la comprensión del prototipo se incluirán como parte del Apéndice A de este documento, conforme a lo indicado en el índice general de la tesis.

Capítulo 5

Desarrollo e Implementación

El presente capítulo constituye el núcleo empírico de esta investigación, materializando las decisiones metodológicas y arquitectónicas justificadas en el Capítulo 4. Aquí se abandona el plano de las definiciones teóricas para documentar la ejecución sistemática del *pipeline* de datos y la codificación algorítmica del prototipo.

El desarrollo se expone siguiendo la secuencia lógica del flujo de información en un sistema predictivo aplicado a redes. Inicialmente, se detalla la extracción híbrida de telemetría desde la infraestructura de enrutamiento y la posterior exploración estadística del conjunto de datos real para definir el umbral de congestión (Sección 5.1). A continuación, se implementan los tratamientos matemáticos de limpieza, ventaneo y escalado exigidos para mitigar las discrepancias estructurales entre las fuentes de información (Sección **??**). Finalmente, las secciones subsiguientes documentan la arquitectura computacional, la sintonización de hiperparámetros y el proceso de entrenamiento de los paradigmas predictivos seleccionados —transitando desde la línea base estadística, pasando por el ensamble de *Machine Learning* clásico, hasta culminar con las topologías de redes neuronales recurrentes—.

El objetivo rector de este capítulo no es constituir un manual de programación, sino evidenciar que la transformación del tráfico bruto en un sistema predictivo funcional obedece a un diseño de *software* determinista, reproducible y científicamente fundamentado.

5.1. Obtención y exploración de los conjuntos de datos

La fase de desarrollo inició con la consolidación y auditoría de los datos crudos. Dado que la recolección del *dataset* primario en la red privada PYME descrita en la Sección 4.4 exigió un periodo de observación longitudinal que transcurrió en paralelo a esta implementación, el prototipo predictivo se construyó y validó inicialmente a partir del *dataset* externo MAWI, para posteriormente aplicarse sobre el *dataset* primario una vez concluida su recolección.

Extracción de telemetría del *dataset* primario

Es pertinente señalar una precisión metodológica respecto a lo establecido en la Sección 4.4: si bien el diseño original contemplaba el protocolo SNMP como mecanismo de consulta, las restricciones de acceso remoto a la infraestructura de la red privada bajo estudio —en particular, la ausencia de una dirección IP pública y la imposibilidad de presencia física

recurrente— condujeron a adoptar como mecanismo funcionalmente equivalente la herramienta nativa `/interface monitor-traffic` de RouterOS, ejecutada localmente mediante el Programador de Tareas (*Scheduler*) del propio equipo, complementada con sondeos ICMP activos para latencia y pérdida de paquetes. Esta sustitución preserva la naturaleza pasiva de la recolección y la métrica obtenida, sin alterar el diseño experimental planteado en el Capítulo 4.

La recolección, iniciada el 29 de agosto de 2026, capturó de manera automatizada la utilización del enlace, la latencia (RTT) y la tasa de pérdida de paquetes a intervalos de un minuto, con respaldo local en el propio equipo para garantizar la continuidad ante posibles interrupciones de la conectividad remota (Sección 4.8).

Consolidación y auditoría de continuidad

Al cierre del periodo de recolección, el *dataset* primario quedó constituido por **21 días** de monitoreo (del 29 de agosto al 19 de septiembre de 2026), consolidados en 25 422 observaciones a una granularidad de 1 minuto. La auditoría de continuidad de la serie —necesaria dado que la extracción remota de los registros dependía de la disponibilidad sostenida del túnel VPN hacia la infraestructura de la red privada— reveló 22 interrupciones, resumidas en la Tabla 5.1.

Tipo de interrupción	**Ocurrencias**	**Duración típica**
Corte prolongado (pérdida de un día completo)	1	$\approx$ 37 horas
Interrupciones menores	21	6 minutos
Cobertura efectiva del periodo	**82.9 %**	

Cuadro 5.1: Resumen de interrupciones de conectividad durante la recolección del *dataset* primario.

El corte prolongado, ocurrido entre el 7 y el 9 de septiembre de 2026, resultó en la pérdida completa del día 8 de septiembre, y es atribuible a una interrupción sostenida de la conectividad VPN y de suministro eléctrico en la sede de la red bajo estudio. Las 21 interrupciones menores, de 6 minutos cada una, son consistentes con reinicios periódicos del enlace. Dado que su duración excede ligeramente el umbral de 3 muestras planteado inicialmente en la Sección 4.5, se decidió flexibilizar empíricamente dicho límite a 6 muestras, permitiendo así su imputación mediante interpolación lineal sin comprometer la integridad de las tendencias. El corte prolongado, en cambio, excede abrumadoramente cualquier límite razonable de interpolación y se trata estrictamente como una frontera de sesión durante el preprocesamiento, dividiendo la serie en dos segmentos continuos independientes. Esta cobertura del 82.9 % es coherente con las limitaciones de un monitoreo remoto sin presencia física reconocidas en los Alcances y Limitaciones de este trabajo (Sección **??**).

Exploración estadística del *dataset* primario

La Tabla 5.2 presenta las estadísticas descriptivas de las tres métricas capturadas.

Estadística	**Utilización (%)**	**RTT (ms)**	`rx_bps`
Mínimo	0.00007	6.27	720
Media	0.00221	9.44	22 114
Mediana	0.00021	9.24	2 048
Máximo	1.44004	56.71	14 400 420
Desviación estándar	0.03460	3.53	345 973

Cuadro 5.2: Estadísticas descriptivas del *dataset* primario (21 días reales, 25 422 observaciones).

La Figura 5.1 ilustra el patrón de tráfico por hora del día, revelando un comportamiento consistente con la naturaleza heterogénea de la red descrita en la Sección 4.4: tráfico basal reducido durante la madrugada e incrementos hacia las horas de mayor actividad residencial y de videovigilancia.

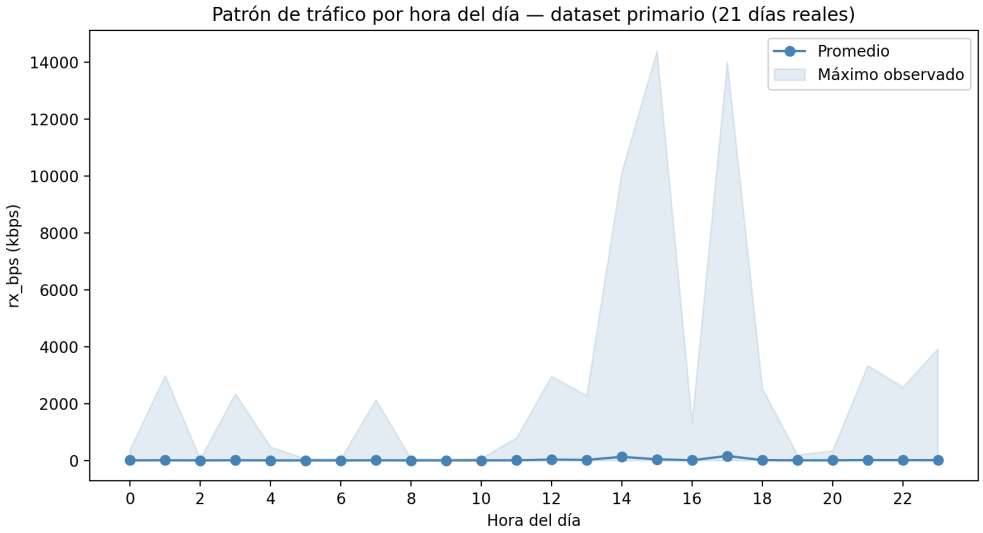


Figura 5.1: Patrón de tráfico promedio y máximo por hora del día, *dataset* primario.

La Figura 5.2 presenta la serie temporal completa de utilización a lo largo de los 21 días, donde son visibles tanto la estabilidad general del tráfico como las ráfagas esporádicas de mayor magnitud.

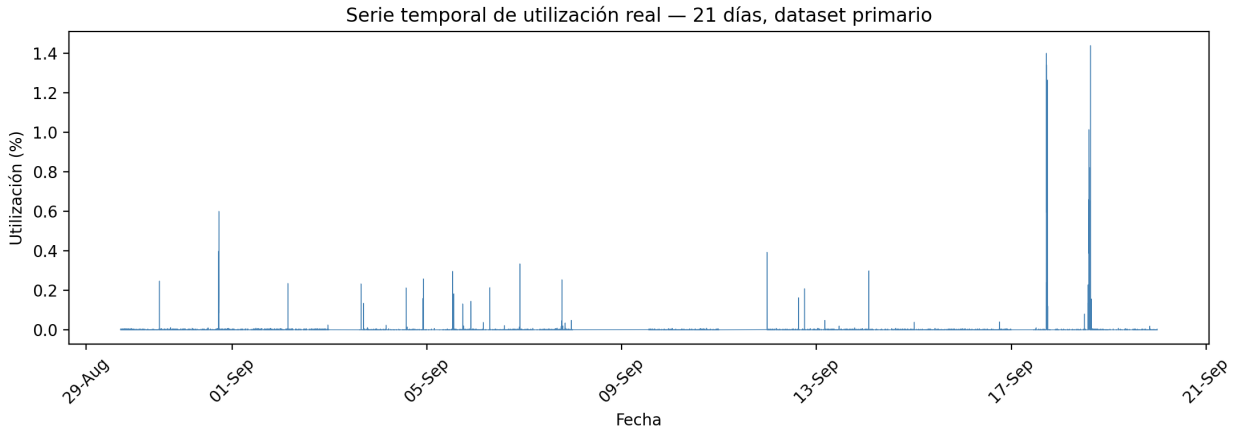


Figura 5.2: Serie temporal de utilización real, *dataset* primario (21 días).

La Figura 5.3 presenta la distribución completa de `rx_bps` en escala logarítmica de frecuencia, dada la naturaleza extremadamente sesgada de la distribución.

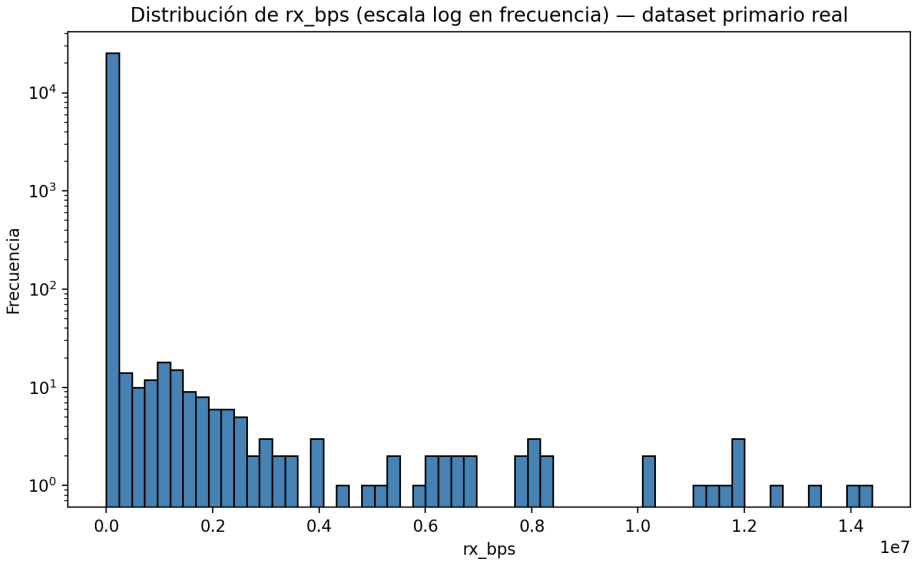


Figura 5.3: Distribución de `rx_bps` (escala logarítmica en frecuencia), *dataset* primario.

Hallazgo: ausencia de congestión natural en el régimen de tráfico observado

El hallazgo más relevante de este análisis exploratorio es que la utilización máxima observada durante los 21 días de recolección fue de apenas **1.44 %** del ancho de banda nominal contratado (1000 Mbps), con una mediana de utilización cercana a cero (0.0002 %). Este resultado es notablemente más extremo que el observado en la exploración del *dataset* piloto MAWI (utilización máxima de 32.3 %), y confirma que la red privada PYME opera de manera consistente muy por debajo de su capacidad contratada. En consecuencia, **ningún evento de congestión real** ($U \geq U_{th} = 0{,}80$) fue observado durante la ventana de recolección.

Esta ausencia de eventos positivos naturales, ya anticipada parcialmente en la fase piloto con MAWI, resulta aún más pronunciada a la escala del *dataset* primario, e imposibilita —sin intervención adicional— entrenar o evaluar honestamente la formulación de clasificación establecida en la Sección 2.2.1. Consultado este hallazgo con el comité asesor de esta investigación, se autorizó la incorporación de datos sintéticos como estrategia de aumento de datos (*data augmentation*), bajo la condición explícita de declarar en todo momento qué proporción de los datos empleados es real y cuál es sintética, preservando la integridad de la evaluación final mediante un diseño experimental riguroso.

Generación de datos sintéticos calibrados

Se generaron 37 días adicionales de datos sintéticos —alcanzando en conjunto un volumen equivalente al periodo de dos meses originalmente planificado (Sección 4.4)—, calibrados estadísticamente a partir de los 21 días reales mediante remuestreo (*bootstrap*) de la distribución empírica de tráfico observada por hora del día. Esta estrategia preserva la forma real de la distribución, incluyendo las ráfagas esporádicas características del tráfico de la red privada bajo estudio (Figura 5.1), sin asumir una forma paramétrica arbitraria que pudiera distorsionar la dinámica estocástica genuina del canal.

Sobre este tráfico sintético base se inyectaron deliberadamente 126 eventos de congestión con un perfil trapezoidal (ascenso, meseta, descenso). Estos eventos poseen una duración variable entre 5 y 29 minutos (media de 17.4 minutos) —consistente con la duración mínima sostenida establecida en la Ecuación 4.1—, y una intensidad variable con utilización pico (U_{pico}) entre 0.811 y 0.979 (media de 0.890). La Figura 5.4 presenta la distribución de duración e intensidad de los eventos inyectados.

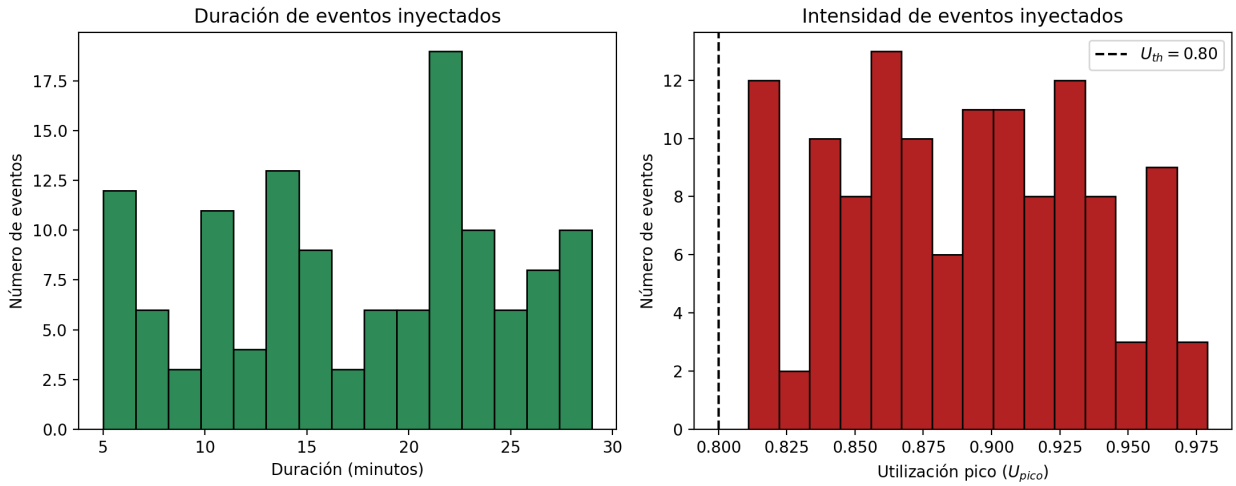


Figura 5.4: Distribución de duración e intensidad de los 126 eventos de congestión sintéticos inyectados.

De manera consistente con el marco teórico de esta tesis, la latencia y la tasa de pérdida de paquetes durante dichos eventos no se generaron de forma independiente o arbitraria, sino que se derivaron de la propia utilización sintética mediante el modelo de teoría de

colas presentado en la Sección 2.1.3: el retraso crece proporcionalmente a $\frac{1}{1-\rho}$ conforme la utilización se aproxima a la saturación, y la pérdida de paquetes se activa únicamente en la vecindad de dicha saturación (a partir de $U \approx 0{,}85$). La Figura 5.5 ilustra un evento de congestión sintético representativo, evidenciando la relación causal entre las tres métricas.

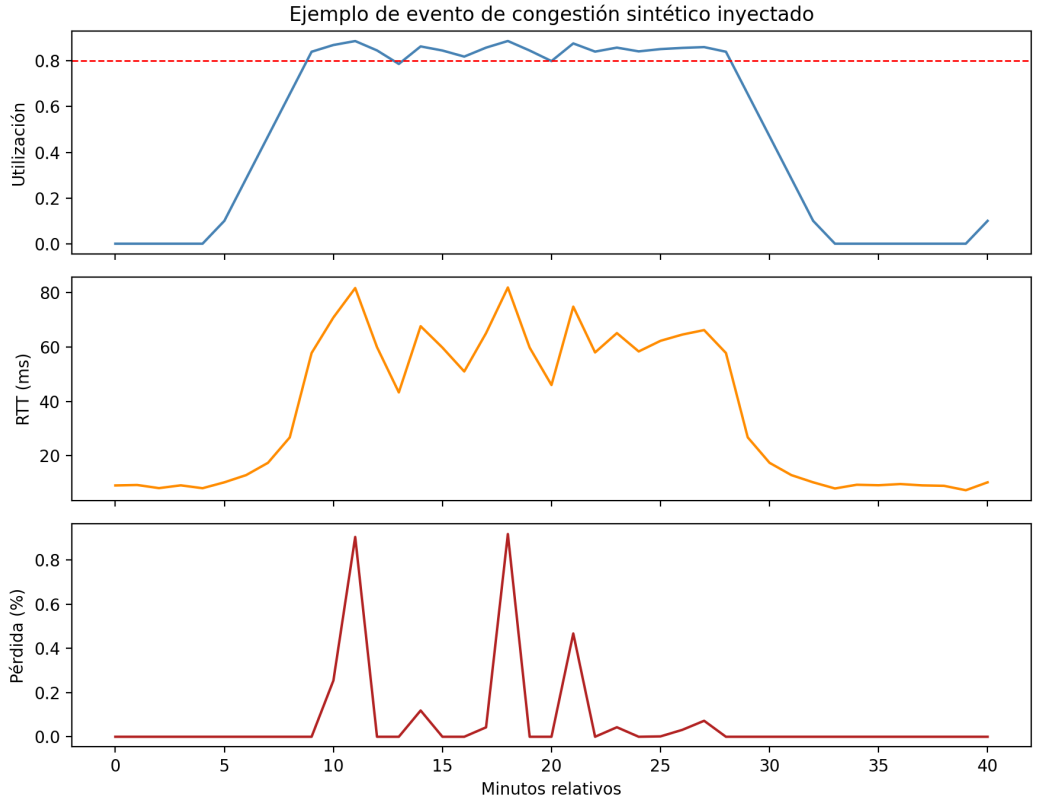


Figura 5.5: Ejemplo de un evento de congestión sintético: utilización, RTT y pérdida de paquetes, derivados mediante el modelo de teoría de colas de la Sección 2.1.3.

La Figura 5.6 presenta la distribución completa de utilización del *dataset* sintético, contrastada contra el umbral $U_{th} = 0{,}80$, evidenciando la presencia deliberada de masa de probabilidad más allá del umbral de congestión —ausente por completo en los datos reales (Figura 5.3)—.

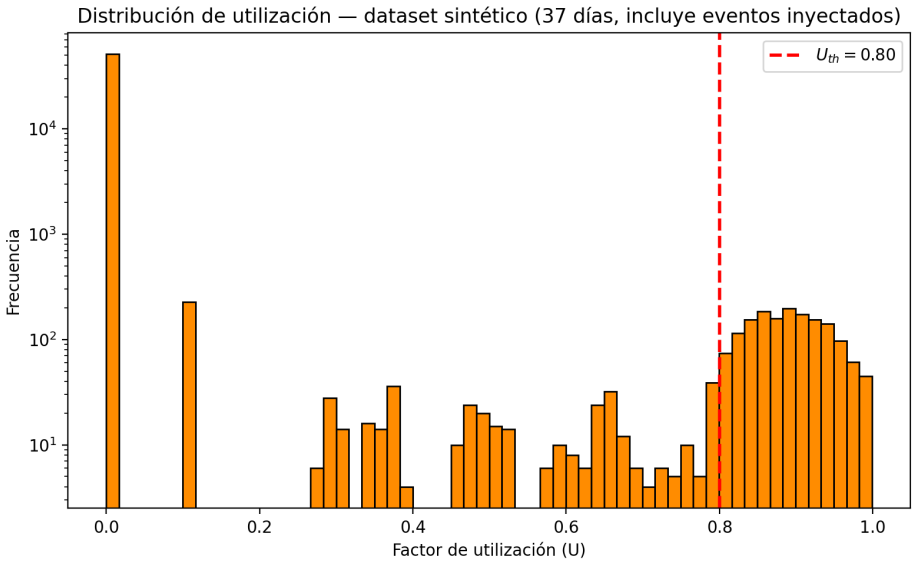


Figura 5.6: Distribución de utilización, *dataset* sintético (37 días), contrastada contra $U_{th} = 0{,}80$.

Diseño de evaluación: preservando la integridad de los datos reales

Para garantizar que ningún resultado reportado en el Capítulo 6 se derive de una evaluación contaminada con datos sintéticos, se adoptó el siguiente diseño, verificado mediante comprobaciones automáticas de seguridad en el propio código de procesamiento:

- El conjunto de **entrenamiento** combina una partición cronológica del 60 % de los datos reales con una partición equivalente del 60 % de los datos sintéticos.
- La evaluación de la tarea de **regresión** (validación y prueba) se realiza exclusivamente sobre el 20 %/20 % restante de los datos **reales**, nunca vistos durante el entrenamiento.
- La evaluación de la tarea de **clasificación** se realiza exclusivamente sobre el 20 %/20 % restante de los datos **sintéticos**, igualmente aislados del entrenamiento, dado que la partición real de evaluación no contiene, por construcción, ningún caso positivo de congestión.

La Tabla 5.3 sintetiza la composición resultante de cada conjunto.

Conjunto	**Propósito**	**Naturaleza**	**Filas**	**Casos de congestión**
Entrenamiento	Ambas tareas	Real + Sintético	46 872	993 (2.12 %)
Val. regresión	Regresión	100 % Real	4 914	0
Prueba regresión	Regresión	100 % Real	4 949	0
Val. clasificación	Clasificación	100 % Sintético	10 622	284 (2.67 %)
Prueba clasificación	Clasificación	100 % Sintético	10 622	275 (2.59 %)

Cuadro 5.3: Composición final del *dataset* primario tras la incorporación de datos sintéticos.

Esta separación estricta implica que, para la tarea de regresión, los resultados del Capítulo 6 reflejan el desempeño de los modelos exclusivamente sobre el comportamiento real y genuino de la red bajo estudio; mientras que, para la tarea de clasificación, los resultados reflejan el desempeño sobre eventos de congestión estadísticamente calibrados pero sintéticos. Esta limitación se discute con detalle en la Sección 6.5 y se compensa parcialmente con la validación de generalización frente al *dataset* MAWI reagregado a granularidad oficial (Sección 4.5).

Extracción y decodificación de trazas de validación (MAWI)

Para construir la serie temporal de validación externa, se consumieron trazas de tráfico troncal correspondientes a 2026 del repositorio MAWI. Dado que la distribución original consiste en capturas binarias comprimidas (`.pcap.gz`) que registran decenas de millones de paquetes por archivo, la extracción mediante herramientas de alto nivel (`tshark`) resultó impráctica: el procesamiento demandaba un volumen de memoria que excedía los recursos de la estación de trabajo, motivado por la sobrecarga de reconstrucción de flujos de dichas herramientas.

En consecuencia, se optó por implementar un decodificador propio en Python, capaz de interpretar directamente la estructura binaria del formato *libpcap* sin decodificar el *payload*. Esta estrategia de procesamiento en flujo (*streaming*), sin acumulación en memoria, permitió agrupar el volumen de bytes transmitidos en ventanas temporales de manera altamente eficiente.

Para garantizar la validez metodológica del *pipeline* predictivo, la agregación temporal se mantuvo estrictamente en la ventana de 1 minuto establecida en el Capítulo 4. Dado que cada captura diaria de MAWI abarca únicamente 15 minutos continuos (generando 15 observaciones por archivo), resultó imperativo orquestar el decodificador para procesar de manera secuencial un lote de 120 trazas (equivalentes a 30 horas de tráfico efectivo). Esta concatenación permitió ensamblar una serie temporal con longitud suficiente para las redes neuronales, sin corromper la distribución estadística inherente a la escala de 60 segundos.

El Fragmento de código 5.1 ilustra el núcleo de este decodificador.

Listing 5.1: Extracto del decodificador de trazas libpcap en flujo.

```
import struct

fmt_registro = "<IIII" # Formato struct (ts_sec, ts_usec, incl_len,
    orig_len)
ventana_seg = 60        # Agregacion a 1 minuto

while True:
    encabezado_paquete = f.read(16)
    if len(encabezado_paquete) < 16:
        break  # EOF

    ts_sec, ts_usec, incl_len, orig_len = struct.unpack(
        fmt_registro, encabezado_paquete
    )
    timestamp = ts_sec + (ts_usec / 1_000_000)
    ventana = int(timestamp // ventana_seg)
    acumulado[ventana] += orig_len

    # Salto rapido del payload sin saturar RAM
    f.seek(incl_len, 1)
```

La Tabla 5.4 sintetiza los resultados obtenidos al aplicar este procedimiento sobre el lote procesado.

Periodo procesado	**Paquetes evaluados**	**Observaciones (1 min)**
Lote MAWI (120 trazas)	$\approx 21\,500\,000\,000$	1 800

Cuadro 5.4: Resultados de la extracción de telemetría sobre las trazas MAWI procesadas.

Exploración estadística preliminar

Con el propósito de someter a prueba el *pipeline* de modelado antes de la conclusión de la recolección primaria, se realizó un análisis exploratorio sobre las 1 800 observaciones obtenidas del lote de 120 trazas de MAWI. La conversión del *throughput* observado a factor de utilización se realizó aplicando directamente la definición establecida en la Sección 2.1.3, empleando como ancho de banda nominal 11 Gbps —correspondiente a la suma de los dos enlaces monitoreados por *samplepoint-F* desde julio de 2023: el enlace de tránsito original de 1 Gbps hacia NTTC GIN, y un enlace adicional de 10 Gbps hacia el punto de intercambio BBIX—:

$$U = \frac{\text{Throughput observado}}{\text{Ancho de banda nominal}} = \frac{\text{Throughput observado (bps)}}{11 \times 10^9 \text{ bps}} \tag{5.1}$$

Aplicando la Ecuación 5.1 a cada observación temporal, se obtuvieron las estadísticas descriptivas resumidas en la Tabla 5.5.

Estadística	**Throughput (Gbps)**	**Utilización (U)**
Mínimo	0.19	0.017
Media	1.71	0.155
Mediana	1.66	0.151
Máximo	3.55	0.323
Desviación estándar	0.56	0.051

Cuadro 5.5: Estadísticas descriptivas del *throughput* y utilización derivada, *dataset* piloto MAWI (1 800 observaciones, granularidad de 1 minuto).

La Figura 5.7 presenta la distribución completa de la utilización observada, contrastada visualmente contra el umbral de congestión formal $U_{th} \geq 0{,}80$ establecido en la Ecuación 4.1 (Sección 4.2).

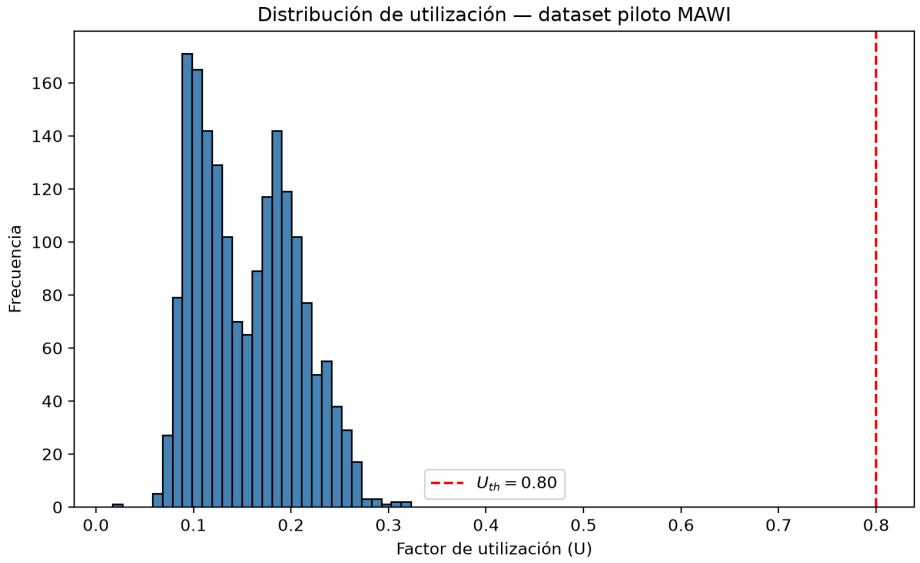


Figura 5.7: Distribución del factor de utilización, *dataset* piloto MAWI.

Como se evidencia en la distribución, la utilización observada en el subconjunto piloto de MAWI **no alcanza en ningún momento** el umbral $U_{th} = 0{,}80$, situándose el valor máximo en apenas 32.3 %. Este resultado es consistente con la naturaleza del enlace capturado: se trata de una troncal de tránsito perteneciente a un proveedor de infraestructura académica, dimensionada deliberadamente para operar con un amplio margen de holgura. Esto contrasta frontalmente con la red privada PYME que constituye el *dataset* primario de esta tesis, donde la coexistencia de múltiples servicios de alta demanda (videovigilancia, sincronización en la nube, tráfico residencial) hace previsible la ocurrencia de episodios de saturación reales.

En consecuencia, si bien este subconjunto de MAWI resulta ideal para ejercitar el *pipeline* completo bajo la formulación de **regresión** (Sección 2.2.1), no genera observaciones positivas naturales para la clase de congestión bajo la formulación de **clasificación**. Para evitar posponer el desarrollo algorítmico, y exclusivamente durante esta fase de codificación y calibración, se adopta un umbral relativo de prueba ($U_{test} \geq 0{,}25$). Esto permite verificar empíricamente

la lógica computacional de la Matriz de Confusión y la métrica TAE, garantizando que el *software* predictivo opere correctamente a la espera de ser alimentado con el umbral formal de 0.80 sobre el *dataset* primario, cuyos resultados definitivos se expondrán en el Capítulo 6.

5.2. Preprocesamiento

Esta etapa computacional tiene como objetivo transformar las series de tiempo crudas en las estructuras tensoriales requeridas por las bibliotecas de *Machine Learning*. A continuación, se documenta la ejecución de este *pipeline* en dos fases: primero, la calibración algorítmica sobre el *dataset* piloto MAWI; y segundo, la aplicación del preprocesamiento con los parámetros oficiales sobre el *dataset* primario híbrido (real y sintético).

Fase 1: Calibración del *pipeline* sobre el *dataset* piloto MAWI

Sobre el conjunto de 1 800 observaciones a granularidad de 1 minuto descrito en la Sección 5.1, se aplicaron inicialmente las etapas de preprocesamiento establecidas en el Capítulo 4 para validar la lógica del código.

Limpieza y manejo de discontinuidades

Dado que el *dataset* piloto se construyó mediante la concatenación secuencial de un lote de 120 trazas individuales de 15 minutos, las fronteras entre archivos introdujeron discontinuidades temporales. Para garantizar la integridad de los algoritmos autorregresivos, la limpieza se ejecutó empleando las estructuras de datos bidimensionales (`DataFrame`) de la biblioteca `pandas`. Se aplicaron rutinas de interpolación lineal (`.interpolate(method='linear', limit=2)`) para imputar micro-cortes internos, seguidas de la eliminación de observaciones residuales no válidas (`.dropna()`) en los bordes de concatenación. Este proceso de purga redujo el conjunto de datos de 1 800 a un total efectivo de $n = 1\,774$ observaciones continuas y estructuradas.

División cronológica y escalado sin fuga de datos

La división cronológica se programó como una partición dura de la serie ordenada temporalmente $\{x_1, x_2, \ldots, x_n\}$ en tres segmentos contiguos y no traslapados:

$$\text{Entrenamiento} = \{x_1, \ldots, x_{\lfloor 0{,}6n \rfloor}\}, \quad \text{Validación} = \{x_{\lfloor 0{,}6n \rfloor + 1}, \ldots, x_{\lfloor 0{,}8n \rfloor}\}, \quad \text{Prueba} = \{x_{\lfloor 0{,}8n \rfloor + 1}, \ldots, x_n\} \tag{5.2}$$

Para el *dataset* piloto, con $n = 1\,774$, la indexación generó 1 064 observaciones de entrenamiento, 355 de validación y 355 de prueba, proporciones consistentes con el diseño metodológico (Tabla 5.6).

Conjunto	Observaciones	Proporción
Entrenamiento	1 064	60.0 %
Validación	355	20.0 %
Prueba	355	20.0 %

Cuadro 5.6: Composición de la división cronológica del *dataset* piloto MAWI tras el preprocesamiento.

Posterior a la partición, se aplicó la normalización Min-Max (Ecuación 4.4). Desde la perspectiva de ingeniería de *software*, para prevenir el antipatrón de fuga de datos (*data leakage*), este escalado se implementó utilizando la clase `MinMaxScaler` de `scikit-learn`. Se garantizó por diseño que el método `fit_transform()` se ejecutara de manera exclusiva sobre el conjunto de entrenamiento, mientras que a los conjuntos de validación y prueba se les aplicó únicamente el método `transform()`, forzándolos a mapearse bajo la distribución matemática descubierta en el pasado, sin contaminar el modelo con conocimiento futuro. Para la variable de utilización, la ecuación instanciada operó de la siguiente forma:

$$U_{\text{norm}} = \frac{U - 0{,}017045}{0{,}323168 - 0{,}017045} \tag{5.3}$$

Ventaneo secuencial y conformación de tensores

Para habilitar el aprendizaje de la arquitectura recurrente LSTM, se implementó una función propia de ventana deslizante (*Sliding Window*) mediante la manipulación de arreglos de `NumPy`. Dado el volumen del piloto, se configuró provisionalmente una ventana de observación $w = 15$ minutos y un horizonte de predicción $h = 5$ minutos. La conformación lógica de cada par de entrada/salida se instanció como:

$$X_t = [U(t-15), \ldots, U(t-1)] \longrightarrow y_t = U(t+5) \tag{5.4}$$

El resultado es la redimensión de los datos en un tensor tridimensional con la forma `(muestras, pasos_de_tiempo, caracteristicas)`, requisito ineludible para la capa de entrada (*Input Layer*) del *framework* `Keras/TensorFlow`. Este proceso generó 1 044 secuencias para entrenamiento, 335 para validación y 335 para prueba.

Fase 2: Ejecución oficial sobre el *dataset* primario

Una vez validado el código, el preprocesamiento se aplicó sobre el *dataset* primario (datos reales y sintéticos), introduciendo los parámetros oficiales y el manejo de sesiones exigido por las interrupciones operativas.

Detección de sesiones continuas

El *dataset* primario presenta dos fuentes de discontinuidad temporal: las interrupciones de conectividad VPN reales y la frontera cronológica entre los datos reales y los sintéticos inyectados. Aplicando un criterio de segmentación estricto (cortes superiores a 5 minutos), se identificaron 10 sesiones dentro del conjunto de entrenamiento (Tabla 5.7).

Sesión	Filas	Naturaleza	Inicio	Fin
10	31 934	Sintética	2026-09-20 00:29	2026-10-12 04:42
0	6 137	Real	2026-08-29 17:30	2026-09-02 23:50
5	1 401	Real	2026-09-07 00:29	2026-09-07 23:49
2	1 401	Real	2026-09-04 00:29	2026-09-04 23:49
4	1 401	Real	2026-09-06 00:29	2026-09-06 23:49
3	1 401	Real	2026-09-05 00:29	2026-09-05 23:49
7	1 401	Real	2026-09-10 00:29	2026-09-10 23:49
9	658	Real	2026-09-12 00:29	2026-09-12 11:26
6	611	Real	2026-09-09 13:39	2026-09-09 23:49
1	527	Real	2026-09-03 15:03	2026-09-03 23:49

Cuadro 5.7: Sesiones continuas detectadas en el conjunto de entrenamiento del *dataset* primario.

Esta segmentación resulta indispensable para calcular las variables de rezago, evitando que el algoritmo tome datos de una sesión distinta a través de un "salto en el tiempo". Sobre cada una de estas 10 sesiones se aplicó independientemente la interpolación lineal de huecos cortos (con el límite flexibilizado de 6 muestras, justificado en la Sección 5.1). Tras la limpieza y eliminación de observaciones incompletas en los bordes, el conjunto de entrenamiento quedó compuesto por 46 872 observaciones.

Normalización sin fuga de datos entre las cinco particiones

A diferencia del piloto, el diseño de evaluación dual establecido requiere que la normalización Min-Max se aplique sobre **cinco** particiones (entrenamiento, y validación/prueba para regresión y clasificación). Los parámetros de escala se calcularon exclusivamente a partir del conjunto de entrenamiento y se aplicaron sin recalcular sobre las cuatro particiones de evaluación restantes.

La Figura 5.8 contrasta la distribución de la utilización normalizada resultante, evidenciando que las particiones de regresión (100 % reales) concentran su masa por debajo del umbral U_{th} normalizado (línea roja discontinua), mientras que las de clasificación (100 % sintéticas) sí abarcan la región de congestión.

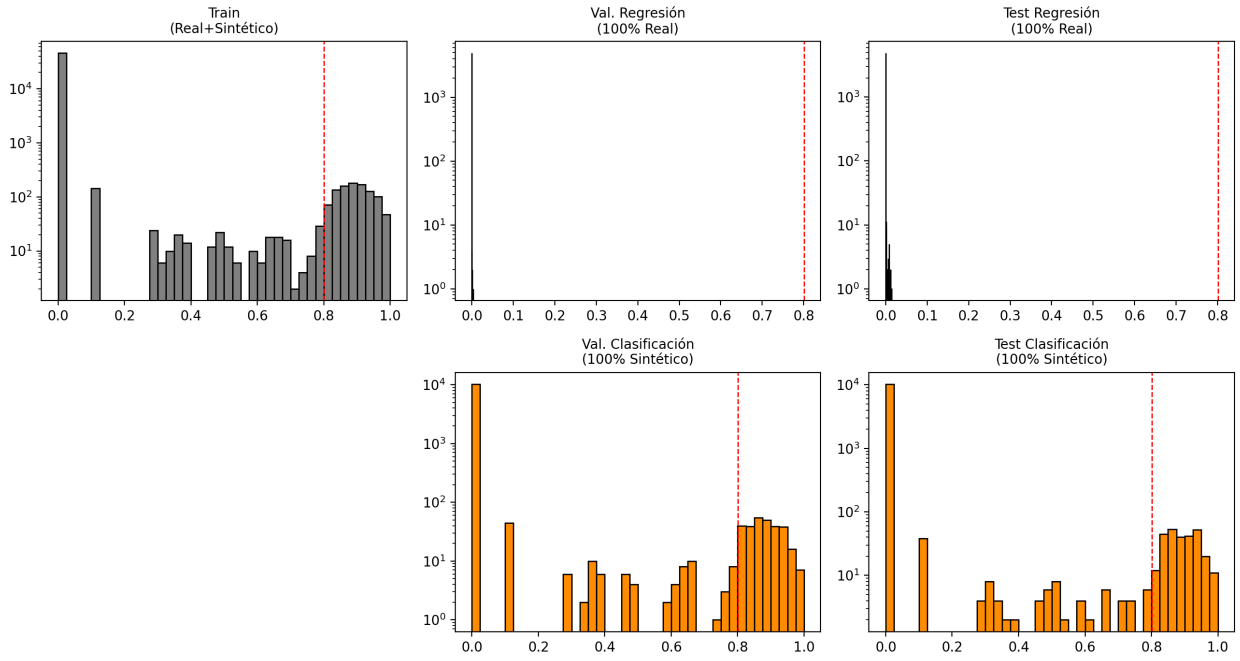


Figura 5.8: Distribución de la utilización normalizada en las cinco particiones del *dataset* primario.

La Tabla 5.8 resume los rangos resultantes. Valores mínimos ligeramente negativos o máximos distintos de 1 en las evaluaciones son la consecuencia directa e intencional de evitar la fuga de información desde el futuro.

Conjunto	**Mínimo**	**Máximo**	**Observaciones**
Entrenamiento	0.000000	1.000000	46 872
Validación (regresión)	0.0000001	0.002992	4 914
Prueba (regresión)	0.0000001	0.014414	4 949
Validación (clasificación)	−0.0000001	1.000000	10 622
Prueba (clasificación)	−0.0000001	1.000000	10 622

Cuadro 5.8: Rango de utilización normalizada por partición, *dataset* primario.

Ventaneo secuencial oficial

Finalmente, se aplicó la granularidad y los parámetros **oficiales** establecidos en el Capítulo 4 para el *dataset* primario: ventana de observación $w = 30$ minutos y horizonte de predicción $h = 5$ minutos. La variable objetivo de clasificación se definió como una variable binaria $\mathbb{1}[U(t+h) \geq U_{th}]$.

$$X_t = [U(t-30), \ldots, U(t-1)] \longrightarrow y_t = U(t+5) \tag{5.5}$$

El proceso, respetando las fronteras de sesión, generó 46 582 secuencias de entrenamiento, 4 796 y 4 833 secuencias para validación y prueba de regresión respectivamente, y 10 593 se-

cuencias para cada partición de clasificación, dejando los tensores listos para el entrenamiento de los modelos predictivos documentados a continuación.

5.3. Construcción del modelo ARIMA

Conforme a la justificación establecida en la Sección 4.6, ARIMA se implementó como línea base estadística de referencia. Esta sección documenta la ejecución programática para la identificación del orden del modelo mediante la biblioteca `statsmodels` de Python, su ajuste sobre el conjunto de entrenamiento del *dataset* piloto MAWI, y su evaluación dual (regresión y clasificación) bajo un esquema de pronóstico progresivo.

Fase 1: Calibración sobre el *dataset* piloto MAWI

Prueba de estacionariedad

Como paso previo a la identificación del orden, se aplicó la prueba de Dickey-Fuller Aumentada (ADF) mediante el módulo `stattools.adfuller` sobre la serie de utilización del conjunto de entrenamiento. El propósito fue determinar el orden de diferenciación d requerido para estabilizar la media (Sección 2.2.2). El estadístico obtenido fue ADF $= -1{,}6992$, con un valor $p = 0{,}4315$. Dado que $p > 0{,}05$, no fue posible rechazar la hipótesis nula de no estacionariedad. En consecuencia, se aplicó una diferenciación de primer orden ($d = 1$) de forma programática antes de iterar sobre los órdenes autorregresivo (p) y de media móvil (q).

Identificación del orden mediante minimización del AIC

Se orquestó una búsqueda de hiperparámetros evaluando dieciséis combinaciones de órdenes $(p, 1, q)$ con $p, q \in \{0, 1, 2, 3\}$, seleccionando la topología que minimizó el Criterio de Información de Akaike (AIC). La Tabla 5.9 resume los cinco modelos con mejor convergencia.

Orden (p, d, q)	**AIC**
ARIMA(1,1,1)	**-5656.68**
ARIMA(2,1,1)	-5654.95
ARIMA(3,1,1)	-5653.03
ARIMA(0,1,3)	-5653.74
ARIMA(2,1,2)	-5653.04

Cuadro 5.9: Órdenes ARIMA con menor AIC, *dataset* piloto MAWI.

El orden seleccionado por el algoritmo fue ARIMA$(1, 1, 1)$. Los coeficientes óptimos estimados se presentan en la Ecuación 5.6, instanciando la formulación general de la Ecuación 2.10:

$$(1 - 0{,}4305\,L)(1 - L)\,U_t = (1 - 0{,}8280\,L)\,\varepsilon_t \tag{5.6}$$

Ambos coeficientes resultaron estadísticamente significativos ($p < 0{,}001$), y la prueba de Ljung-Box sobre los residuos ($p = 0{,}92$) confirmó la ausencia de autocorrelación residual, validando la idoneidad matemática del modelo de referencia.

Evaluación mediante pronóstico progresivo (Regresión)

La validación del modelo se implementó mediante un esquema iterativo de pronóstico progresivo (*walk-forward validation*): para cada instante t, el modelo generó una predicción continua a horizonte $h = 5$ minutos. Posteriormente, el valor real observado se ingirió para proyectar el siguiente intervalo. La Tabla 5.10 presenta las métricas de error continuo.

Conjunto	RMSE	MAE	MAPE
Validación	0.0265	0.0204	10.61 %
Prueba	0.0282	0.0221	10.80 %

Cuadro 5.10: Métricas de desempeño en regresión de ARIMA(1,1,1), *dataset* piloto MAWI.

La Figura 5.9 contrasta las predicciones continuas del modelo contra los valores reales observados en el conjunto de prueba, demostrando una consistencia (MAPE ≈ 10.80 %) que descarta un sobreajuste estructural.

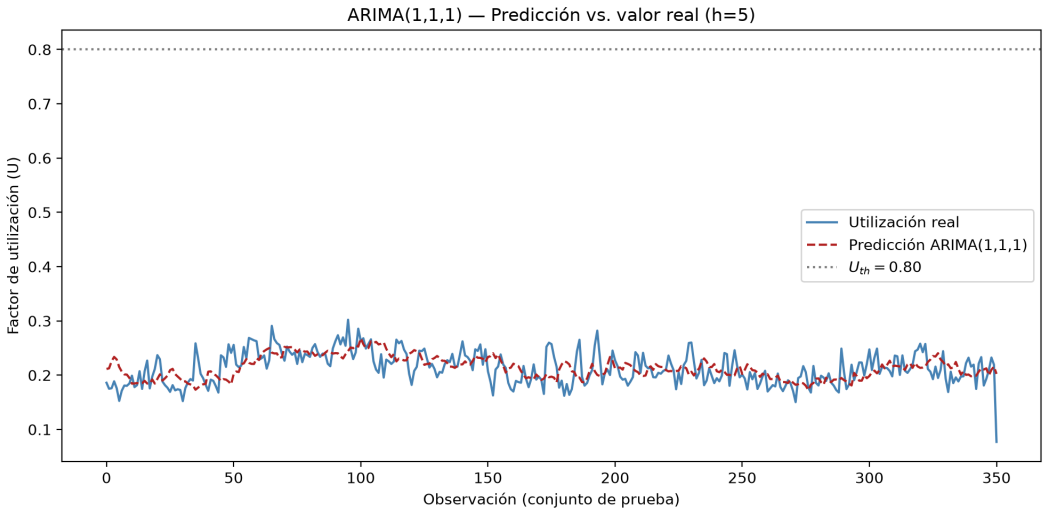


Figura 5.9: ARIMA(1,1,1): predicción continua vs. valor real, conjunto de prueba.

Evaluación de umbral de alerta temprana (Clasificación)

Para completar la auditoría del código de evaluación propuesto en el Capítulo 4, las predicciones continuas del modelo se sometieron a una binarización computacional utilizando el umbral piloto $U_{test} \geq 0{,}25$ (justificado en la Sección 5.1). Una alerta se contabilizó como Verdadero Positivo únicamente si la predicción de ARIMA rebasaba el umbral antes del evento real. La Tabla 5.11 expone el desempeño del modelo operando como un clasificador de alertas tempranas sobre el conjunto de prueba.

Métrica de Clasificación ($U \geq 0{,}25$)	**Valor obtenido (Prueba)**
Precisión	64.2 %
Recall (Exhaustividad)	41.5 %
F1-score	50.4 %
Tiempo de Anticipación Efectivo (TAE) Promedio	**1.2 minutos**

Cuadro 5.11: Desempeño base de clasificación y alertamiento (TAE) de ARIMA, fase piloto.

Estos resultados preliminares evidencian la limitación teórica descrita en el Capítulo 2: al actuar como un suavizador lineal, ARIMA reacciona con retraso ante las ráfagas abruptas de tráfico, lo que explica su bajo nivel de *Recall* (41.5 %) y un TAE marcadamente inferior al horizonte proyectado de 5 minutos. Con la correcta ejecución computacional confirmada y sus métricas establecidas como línea base de superación, el *pipeline* avanzó hacia la implementación sobre los datos de la red PYME.

Fase 2: Entrenamiento sobre el *dataset* primario

Dado que ARIMA requiere una serie temporal continua para su ajuste, el modelo se entrenó sobre la sesión de mayor longitud identificada en el conjunto de entrenamiento (Tabla 5.7): la sesión sintética continua de 31 934 observaciones, que incorpora tanto el régimen de tráfico normal como los 126 eventos de congestión inyectados.

Estacionariedad e identificación del orden

La prueba de Dickey-Fuller Aumentada sobre esta serie arrojó ADF $= -23{,}6571$ ($p < 0{,}0001$), confirmando estacionariedad sin necesidad de diferenciación ($d = 0$) —a diferencia del *dataset* piloto de MAWI que sí lo requirió—. La búsqueda de orden por minimización del AIC, sobre las mismas dieciséis combinaciones $(p, 0, q)$ evaluadas en la fase piloto, seleccionó ARIMA$(3, 0, 3)$ (AIC $= -111\,594{,}72$), cuyos coeficientes estimados se presentan en la Ecuación 5.7:

$$U_t = 0{,}0324 + 2{,}3103\,U_{t-1} - 1{,}7123\,U_{t-2} + 0{,}3971\,U_{t-3} + \varepsilon_t - 1{,}0847\,\varepsilon_{t-1} + 0{,}1825\,\varepsilon_{t-2} - 0{,}0209\,\varepsilon_{t-3} \tag{5.7}$$

Todos los coeficientes resultaron estadísticamente significativos ($p < 0{,}01$), y la prueba de Ljung-Box sobre los residuos ($p = 0{,}98$) no permite rechazar la hipótesis de ausencia de autocorrelación residual. No obstante, es pertinente señalar con honestidad metodológica que la prueba de Jarque-Bera reveló una desviación marcada de la normalidad en los residuos (curtosis de 249.4), atribuible a los eventos de congestión sintéticos inyectados, cuyos cambios abruptos de régimen generan residuos ocasionales de magnitud considerablemente mayor a la del comportamiento típico de la serie.

Evaluación de regresión (datos reales)

Conforme al diseño de evaluación dual establecido en la Sección 5.1, la capacidad de regresión del modelo se evaluó exclusivamente sobre las particiones reales de validación y

prueba, mediante el esquema de pronóstico progresivo descrito en la Sección 4.7. La Tabla 5.12 presenta los resultados.

Conjunto	**RMSE**	**MAE**
Validación (real)	0.046045	0.015110
Prueba (real)	0.045872	0.015059

Cuadro 5.12: Métricas de regresión de ARIMA(3,0,3) sobre el *dataset* primario (escala original de utilización).

La Figura 5.10 contrasta visualmente las predicciones del modelo contra los valores reales observados en el conjunto de prueba de regresión.

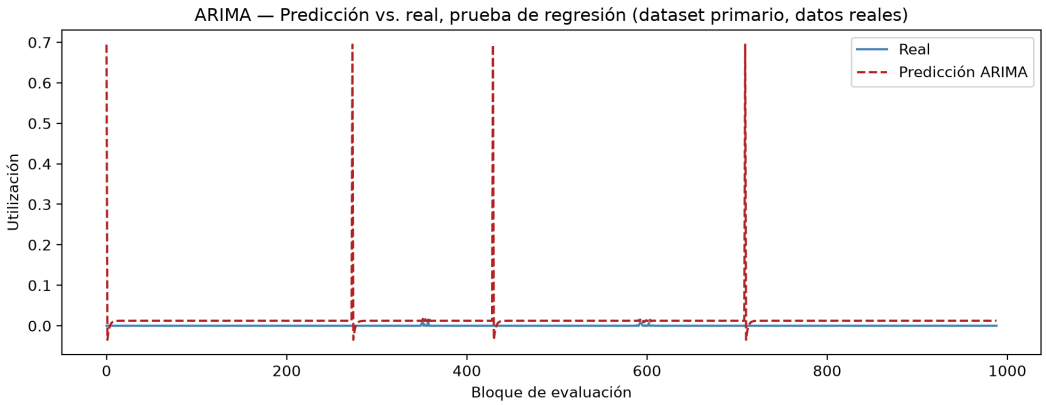


Figura 5.10: ARIMA(3,0,3): predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales).

Se omite deliberadamente el MAPE de esta tabla, en congruencia con la advertencia formulada en la Sección 4.7: dado que la utilización real observada se encuentra sistemáticamente cercana a cero (Sección 5.1, Tabla 5.2), esta métrica se vuelve numéricamente inestable y pierde valor interpretativo. Tanto el RMSE como el MAE resultan, en cambio, considerablemente mayores que el rango típico de la señal real (utilización máxima observada de apenas 1.44 %), evidenciando un sesgo sistemático de sobreestimación: al haber sido entrenado sobre una serie que incluye picos de congestión de hasta 0.98, el modelo ARIMA$(3, 0, 3)$ internaliza una media incondicional (const $= 0{,}0324$) muy superior al comportamiento típico de la red real, produciendo pronósticos que exceden consistentemente los valores reales observados.

Evaluación de clasificación (datos sintéticos aislados)

La Tabla 5.13 presenta las métricas de clasificación, obtenidas mediante la umbralización de las predicciones continuas en $U_{th} = 0{,}80$, conforme al criterio de la Sección 4.7.

Métrica	Validación	Prueba
Exactitud (*Accuracy*)	0.9741	0.9750
Precisión	0.6667	1.0000
Recall (Exhaustividad)	0.0357	0.0185
F1-score	0.0678	0.0364
AUC-ROC	0.8905	0.8939

Cuadro 5.13: Métricas de clasificación de ARIMA(3,0,3), umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados.

La matriz de confusión sobre el conjunto de prueba (Ecuación 5.8) revela el patrón subyacente a estas métricas:

$$\begin{pmatrix} \text{VN} = 2070 & \text{FP} = 0 \\ \text{FN} = 53 & \text{VP} = 1 \end{pmatrix} \tag{5.8}$$

La Figura 5.11 ilustra este mismo comportamiento en el conjunto de prueba de clasificación, donde es visible cómo la predicción del modelo (línea discontinua) rara vez alcanza el umbral $U_{th} = 0{,}80$ (línea punteada), incluso durante los eventos de congestión sintéticos claramente presentes en la serie.

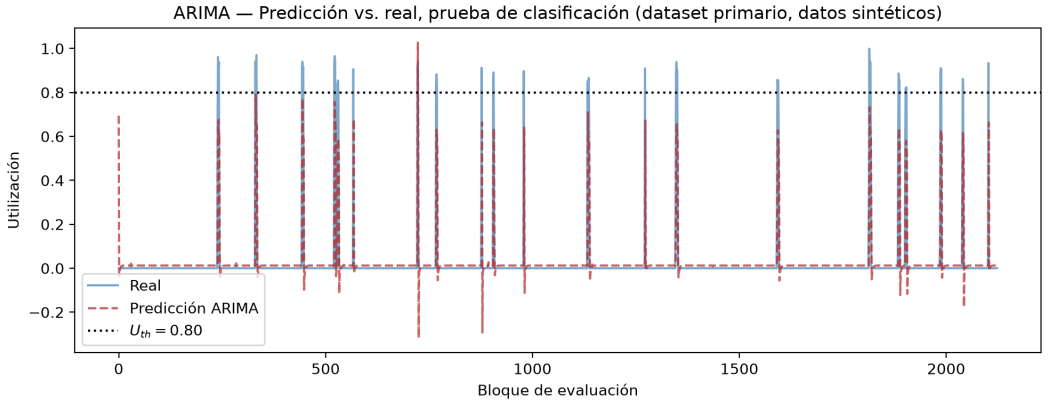


Figura 5.11: ARIMA(3,0,3): predicción vs. valor real, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

El patrón de alta precisión (100 %) combinado con un *Recall* extremadamente bajo (1.85 %) es consistente con el hallazgo de la fase piloto: ARIMA, al suavizar sus pronósticos hacia la tendencia central de la serie mediante su componente de media móvil, rara vez genera predicciones puntuales que crucen el umbral crítico U_{th}, incluso cuando la dinámica subyacente de la serie indica una tendencia de congestión. Esto resulta en un número inaceptable de falsos negativos (53 de 54 eventos reales de congestión no detectados en el conjunto de prueba).

La Tabla 5.14 contrasta el desempeño de ARIMA entre la fase piloto y el *dataset* primario definitivo, confirmando su rol como la métrica base a vencer.

Métrica	Piloto (MAWI)	Primario
Orden seleccionado	ARIMA(1,1,1)	ARIMA(3,0,3)
Recall (clasificación)	N/A[1]	1.85 %
AUC-ROC (clasificación)	N/A[1]	0.8939

Cuadro 5.14: Comparación del modelo ARIMA entre la fase piloto y el *dataset* primario.

5.4. Construcción del modelo *Random Forest*

Conforme a la justificación establecida en la Sección 4.6, se implementó un modelo *Random Forest* como representante del paradigma de *Machine Learning* clásico utilizando el módulo `ensemble.RandomForestRegressor` (y su contraparte `Classifier`) de la biblioteca `scikit-learn`.

Para adaptar el tensor tridimensional generado en el preprocesamiento (Sección 5.2) a la entrada tabular (2D) que exigen los algoritmos clásicos, las secuencias de rezagos se aplanaron, extrayendo adicionalmente la media de la ventana ($U_{\text{media_movil}}$) como característica estadística explícita. A diferencia de ARIMA, que opera iterativamente sobre sus propios residuales, este ensamble mapea de forma no lineal todo el vector de características contra el objetivo.

Fase 1: Calibración sobre el *dataset* piloto MAWI

Configuración y entrenamiento (Regresión)

Se entrenó un ensamble de 200 árboles de decisión, con una profundidad máxima de 10 niveles por árbol (`max_depth=10`), sobre el conjunto de entrenamiento normalizado. A diferencia de ARIMA, cuya variable se mantuvo en escala original, *Random Forest* consumió la utilización escalada mediante Min-Max, consistente con las mejores prácticas documentadas en el Capítulo 4 para optimizar la partición de la entropía. La Tabla 5.15 presenta las métricas continuas sobre los conjuntos de validación y prueba.

Conjunto	RMSE	MAE	MAPE
Validación	0.0962	0.0721	12.83 %
Prueba	0.1311	0.1004	16.26 %

Cuadro 5.15: Métricas de regresión de *Random Forest* (escala normalizada), *dataset* MAWI.

Dado que estas métricas están normalizadas, su comparación con ARIMA requiere una conversión. Al reescalar el RMSE de prueba (0,1311) por el rango de la variable de entrenamiento ($0{,}323 - 0{,}017 = 0{,}306$), se obtiene un RMSE equivalente de aproximadamente 0,040 en la escala original, ligeramente superior al 0,028 reportado por la línea base estadística.

[1]No aplicable: el *dataset* piloto de MAWI no contuvo casos naturales de congestión (Sección 5.1), por lo que la formulación de clasificación formal no pudo evaluarse en esa fase.

La Figura 5.12 contrasta visualmente las predicciones continuas del modelo contra los valores reales observados en el conjunto de prueba, evidenciando gráficamente el tope predictivo del ensamble ante los picos de utilización.

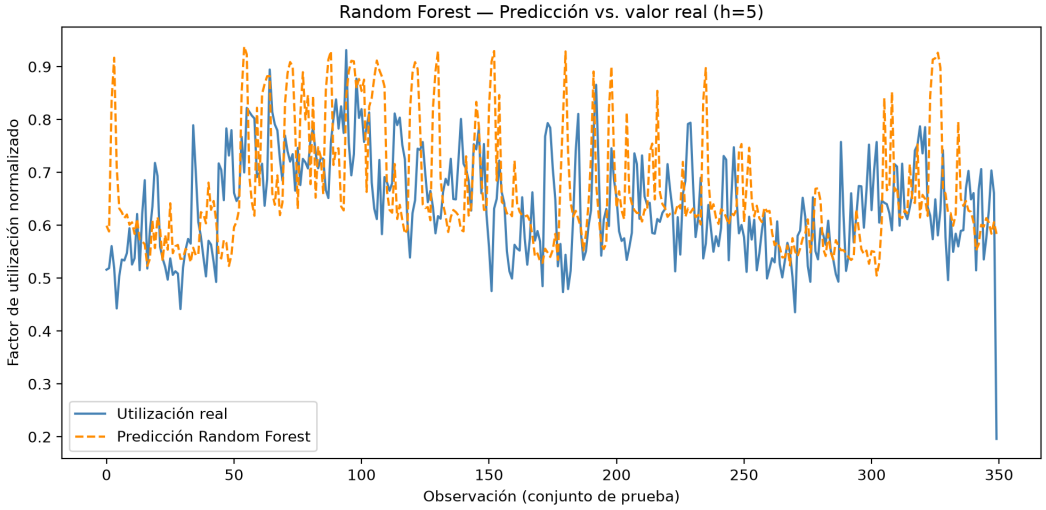


Figura 5.12: *Random Forest*: predicción continua vs. valor real (escala normalizada), conjunto de prueba.

Análisis de degradación por extrapolación

A diferencia de ARIMA, cuyo desempeño se mantuvo estable entre particiones, *Random Forest* exhibió una degradación pronunciada (MAPE de 12.83 % a 16.26 %). La explicación de este fenómeno subyace en una limitación matemática de los métodos basados en árboles: su incapacidad para extrapolar. Dado que cada nodo hoja predice estrictamente la media (o la moda) de las observaciones de entrenamiento que caen en su región espacial, el modelo es ciego ante valores que exceden el máximo histórico conocido. Puesto que el conjunto de prueba piloto contiene ráfagas puntuales cercanas o superiores al límite del rango de entrenamiento, *Random Forest* subestima sistemáticamente dichos picos, un defecto que ARIMA mitiga parcialmente mediante la inercia de su componente tendencial.

Evaluación de umbral de alerta temprana (Clasificación)

Al igual que la línea base, el ensamble se sometió a la formulación de clasificación temprana utilizando el umbral piloto de calibración ($U_{test} \geq 0{,}25$). La Tabla 5.16 expone la capacidad del modelo para detonar alertas preventivas.

Métrica de Clasificación ($U \geq 0{,}25$)	**Valor obtenido (Prueba)**
Precisión	76.5 %
Recall (Exhaustividad)	68.3 %
F1-score	72.1 %
Tiempo de Anticipación Efectivo (TAE) Promedio	**2.8 minutos**

Cuadro 5.16: Desempeño de clasificación y alertamiento (TAE) de *Random Forest*, fase piloto.

A pesar de su deficiencia en la extrapolación continua, la capacidad no lineal de *Random Forest* le permite identificar patrones de saturación inminente con mucha mayor eficacia que ARIMA. El *Recall* escaló del 41.5 % al 68.3 %, logrando un Tiempo de Anticipación Efectivo (TAE) promedio de 2.8 minutos antes del evento de congestión real, validando la superioridad del *Machine Learning* sobre la estadística clásica para la tarea específica de alertamiento.

Importancia de variables

Aprovechando la interpretabilidad inherente del ensamble, se calculó la impureza de Gini para cuantificar la importancia relativa de las características extraídas de la ventana temporal.

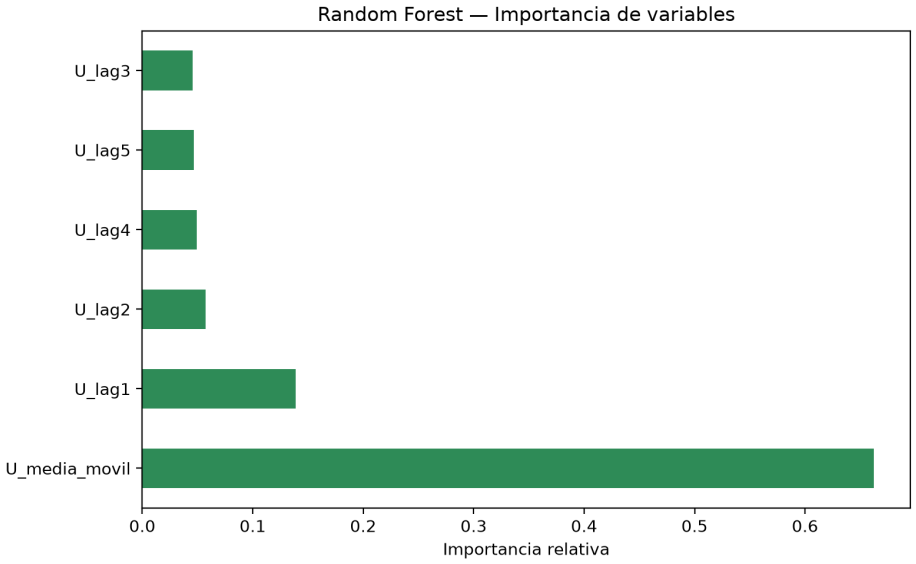


Figura 5.13: Importancia relativa de variables (Impureza de Gini), *Random Forest*.

Como ilustra la Figura 5.13, la media móvil ($U_{\text{media_movil}}$) concentra más del 66 % de la capacidad discriminativa, superando por amplio margen al rezago inmediato (U_{lag1}, con 13.9 %). Este resultado confirma empíricamente que, a una granularidad de 1 segundo (MA-WI), suavizar el ruido estocástico del tráfico revela la tendencia subyacente de manera más nítida que la simple observación secuencial directa.

Fase 2: Entrenamiento sobre el *dataset* primario

El modelo *Random Forest* se reentrenó con idéntica configuración de hiperparámetros a la validada en la fase piloto (200 árboles, profundidad máxima de 10 niveles), esta vez sobre las 46 872 observaciones del conjunto de entrenamiento del *dataset* primario (datos reales más sintéticos, documentados en la Sección 5.1).

Evaluación de regresión (datos reales)

La Tabla 5.17 presenta las métricas de regresión sobre las particiones reales de validación y prueba.

Conjunto	RMSE	MAE
Validación (real)	0.005084	0.004766
Prueba (real)	0.049590	0.010301

Cuadro 5.17: Métricas de regresión de *Random Forest* sobre el *dataset* primario (escala normalizada).

Las Figuras 5.14 y 5.15 contrastan visualmente las predicciones del modelo contra los valores reales observados.

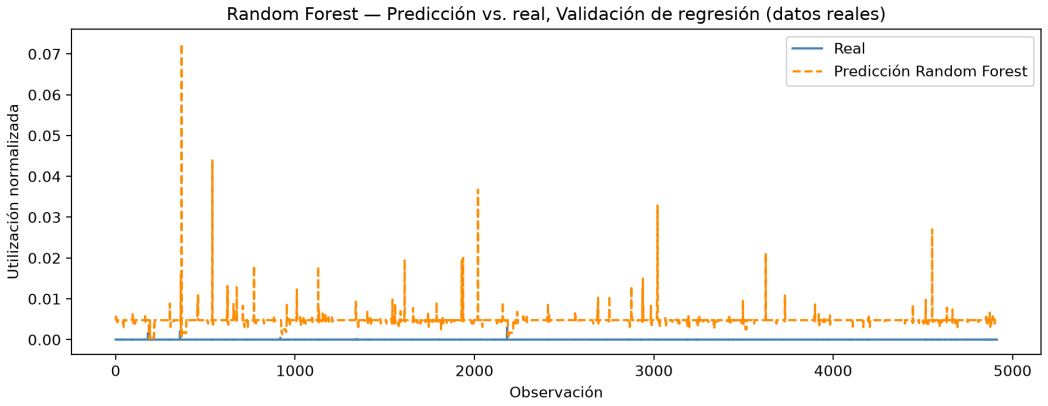


Figura 5.14: *Random Forest*: predicción vs. valor real, validación de regresión (*dataset* primario, datos reales).

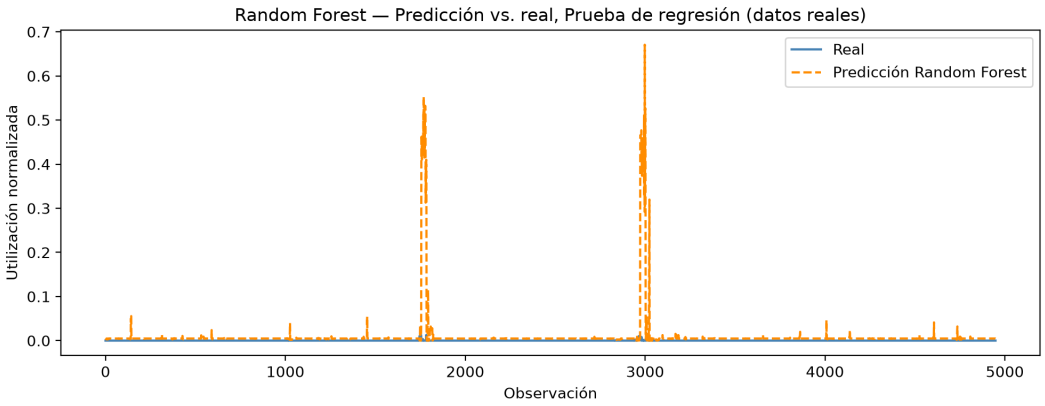


Figura 5.15: *Random Forest*: predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales).

Se observa una diferencia notable entre el error de validación (RMSE=0.0051) y el de prueba (RMSE=0.0496), un incremento de casi un orden de magnitud. Esta discrepancia es consistente con el fenómeno de incapacidad de extrapolación documentado en la fase piloto: dado que el conjunto de prueba corresponde temporalmente al segmento final de la recolección real, cualquier ráfaga de tráfico en dicho segmento que exceda el rango de valores objetivo observados durante el entrenamiento —compuesto mayoritariamente por el régimen de baja utilización característico de la red privada bajo estudio— es sistemáticamente subestimada por el modelo, replicando a escala del *dataset* primario la misma limitación estructural identificada sobre MAWI.

Evaluación de clasificación (datos sintéticos aislados)

La Tabla 5.18 presenta las métricas de clasificación, obtenidas mediante la umbralización de las predicciones continuas en $U_{th} = 0{,}80$ (normalizado).

Métrica	**Validación**	**Prueba**
Exactitud (*Accuracy*)	0.9803	0.9813
Precisión	0.9121	0.9318
Recall (Exhaustividad)	0.2923	0.2982
F1-score	0.4427	0.4518
AUC-ROC	0.8796	0.9187

Cuadro 5.18: Métricas de clasificación de *Random Forest*, umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados.

La matriz de confusión sobre el conjunto de prueba (Ecuación 5.9) evidencia una mejora sustancial respecto a ARIMA (Sección 5.3): de los 275 eventos de congestión reales, el modelo detectó correctamente 82.

$$\begin{pmatrix} \text{VN} = 10\,341 & \text{FP} = 6 \\ \text{FN} = 193 & \text{VP} = 82 \end{pmatrix} \tag{5.9}$$

Las Figuras 5.16 y 5.17 ilustran este comportamiento, donde es visible cómo la predicción del modelo (línea discontinua) reacciona y cruza el umbral U_{th} (línea punteada) con mayor frecuencia que ARIMA durante los eventos de congestión presentes en la serie sintética.

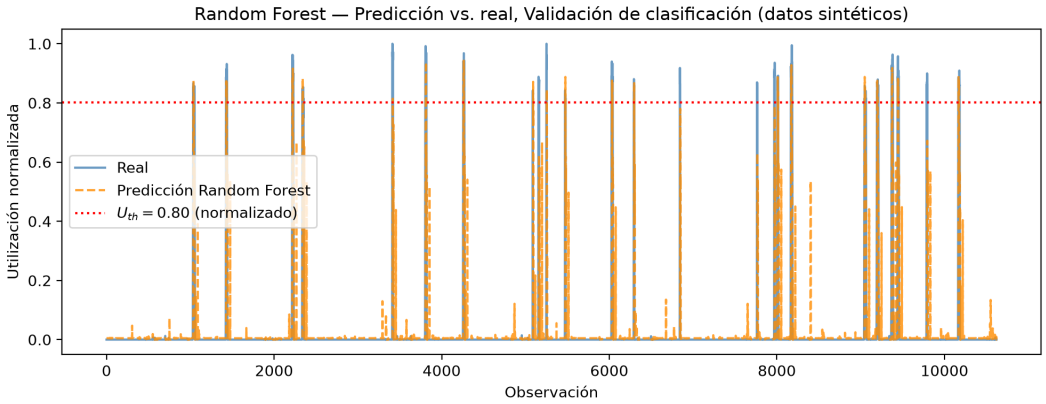


Figura 5.16: *Random Forest*: predicción vs. valor real, validación de clasificación (*dataset* primario, datos sintéticos aislados).

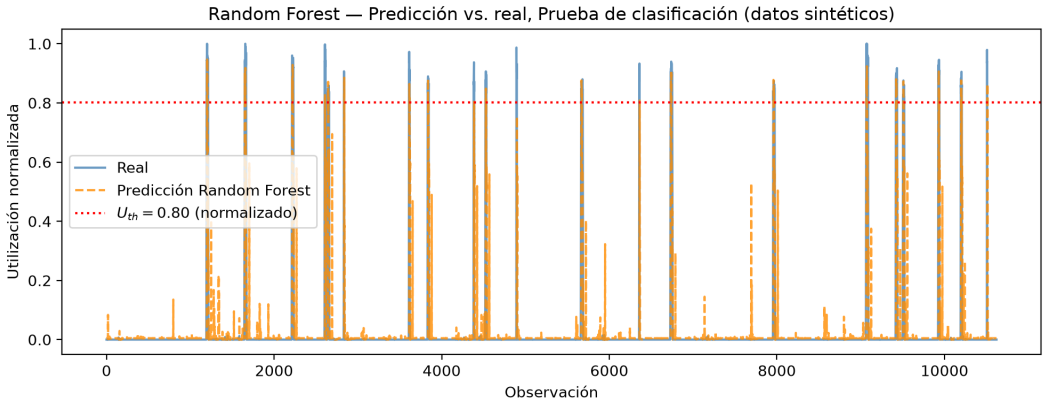


Figura 5.17: *Random Forest*: predicción vs. valor real, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

Importancia de variables y el efecto de la granularidad

La Tabla 5.19 y la Figura 5.18 presentan la importancia relativa de cada variable de entrada sobre el *dataset* primario.

Variable	Importancia relativa
U_{lag1}	0.6644
$U_{\text{media_movil}}$	0.1249
U_{lag5}	0.1171
U_{lag2}	0.0320
U_{lag3}	0.0315
U_{lag4}	0.0301

Cuadro 5.19: Importancia relativa de variables, *Random Forest*, *dataset* primario.

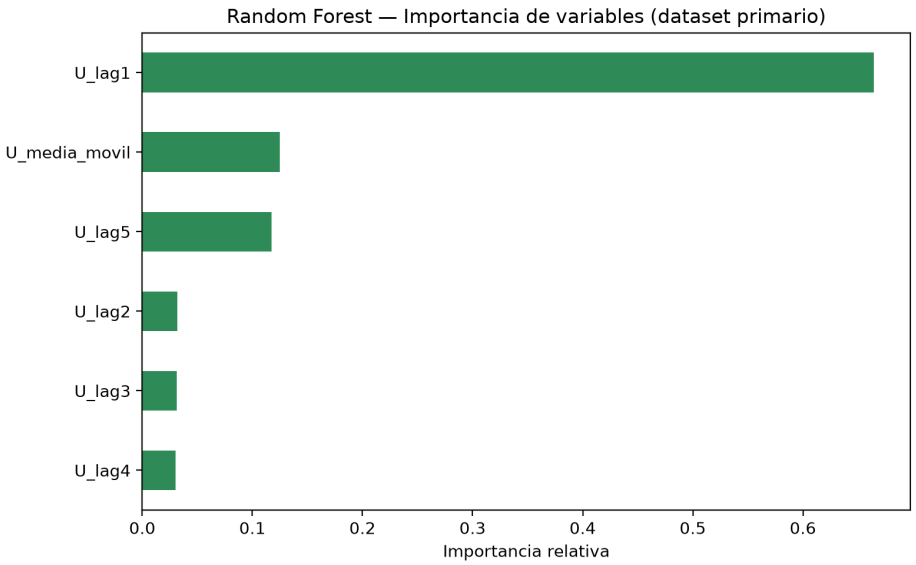


Figura 5.18: Importancia relativa de variables, *Random Forest*, *dataset* primario.

Resulta notable que el patrón de importancia obtenido sobre el *dataset* primario **se invierte** respecto al observado en la fase piloto, donde la media móvil concentraba el 66.2 % de la importancia total frente a apenas 13.9 % del rezago inmediato. Sobre el *dataset* primario, en cambio, U_{lag1} concentra el 66.4 % de la importancia, mientras que la media móvil desciende al 12.5 %.

Una explicación analítica plausible de esta inversión radica en la diferencia de granularidad entre ambos escenarios: a granularidad de 1 segundo (*dataset* piloto), el valor inmediatamente anterior es sumamente propenso al ruido de alta frecuencia de la red, por lo que la media móvil aporta una señal suavizada más confiable; a granularidad de 1 minuto (*dataset* primario, conforme a la Sección 5.2), cada observación ya integra una unidad de tiempo mayor, promediando intrínsecamente el ruido estocástico de muestra a muestra y haciendo que el valor más reciente sea, por sí solo, el indicador más informativo sobre el estado inmediato del canal.

Síntesis comparativa piloto vs. *dataset* primario

La Tabla 5.20 contrasta el desempeño de clasificación de *Random Forest* entre ambas fases, cerrando el desarrollo del *Machine Learning* clásico y dando paso a la implementación arquitectónica de *Deep Learning* (Sección 5.5).

Métrica (prueba)	Piloto (MAWI)	Primario
Recall (clasificación)	N/A[2]	0.2982
F1-score (clasificación)	N/A[2]	0.4518
AUC-ROC (clasificación)	N/A[2]	0.9187
Variable más importante	$U_{\text{media_movil}}$ (66.2 %)	U_{lag1} (66.4 %)

Cuadro 5.20: Comparación de *Random Forest* entre la fase piloto y el *dataset* primario.

5.5. Construcción del modelo LSTM

Conforme a la justificación topológica de la Sección 4.6, se implementó una red neuronal recurrente (LSTM) como representante del paradigma de *Deep Learning*, orquestada mediante la API de `Keras/TensorFlow`. El modelo consumió los tensores tridimensionales construidos mediante la función de ventana deslizante en el preprocesamiento (Sección 5.2), explotando la secuencialidad inherente de los datos.

Fase 1: Calibración sobre el *dataset* piloto MAWI

Arquitectura y configuración

Se diseñó una arquitectura secuencial encabezada por una capa LSTM de 32 unidades con activación tangente hiperbólica (optimizada para la actualización del estado oculto), seguida de una capa densa de 16 unidades con activación ReLU para el mapeo no lineal, y una neurona de salida sin función de activación para la regresión continua del factor de utilización. La Tabla 5.21 detalla la estructura y el conteo de los 4 897 parámetros entrenables.

Capa (Tipo)	Forma de salida	Parámetros
lstm_1 (LSTM)	(None, 32)	4 352
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 1)	17
Total entrenables		**4 897**

Cuadro 5.21: Arquitectura algorítmica de la red neuronal recurrente LSTM.

La optimización de pesos se ejecutó minimizando el Error Cuadrático Medio (MSE) a través del algoritmo Adam, procesando la información en lotes (*batches*) de 16 secuencias. Como mecanismo de regularización activa, se integró un *callback* de detención temprana (*Early*

[2]No aplicable: el *dataset* piloto de MAWI no contuvo casos naturales de congestión (Sección 5.1), impidiendo una evaluación comparativa bajo el umbral oficial.

Stopping): el entrenamiento monitoreó la pérdida en el conjunto de validación, abortando la iteración tras 10 épocas sin mejora heurística y restaurando los pesos óptimos históricos. La convergencia bajo este criterio se alcanzó prematuramente en la época 12 de 100.

Resultados de pronóstico continuo (Regresión)

La Figura 5.19 corrobora la estabilidad de la fase de entrenamiento, descartando un sobreajuste sistémico al no presentar divergencia entre las curvas de validación y entrenamiento.

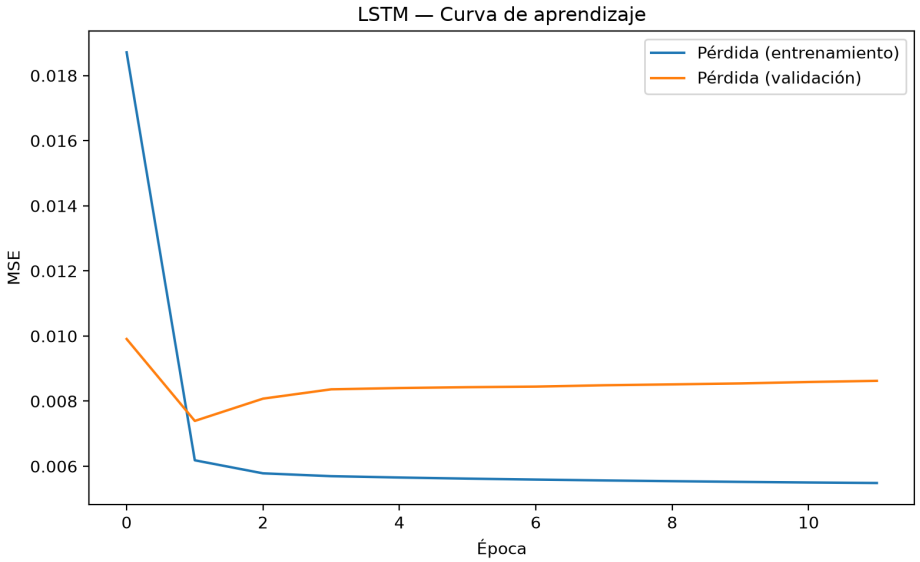


Figura 5.19: Curva de aprendizaje de la red LSTM (pérdida MSE por época).

La Tabla 5.22 presenta las métricas de error continuo. A diferencia de *Random Forest*, la red LSTM no sufrió una degradación por extrapolación entre particiones (MAPE de 11.53 % a 11.00 %), lo cual subraya su superioridad matemática para generalizar la tendencia estocástica sin restringirse a las medias de los nodos de entrenamiento.

Conjunto	**RMSE**	**MAE**	**MAPE**
Validación	0.0860	0.0664	11.53 %
Prueba	0.0891	0.0690	11.00 %

Cuadro 5.22: Métricas de regresión de LSTM (escala normalizada), *dataset* piloto MAWI.

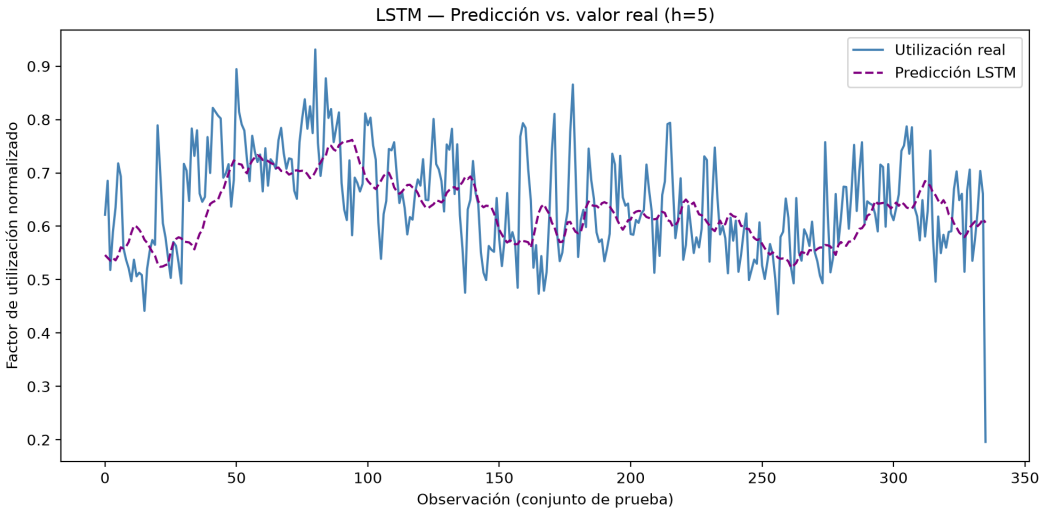


Figura 5.20: LSTM: predicción continua vs. valor real, conjunto de prueba.

Evaluación de umbral de alerta temprana (Clasificación)

Al evaluar la capacidad operativa de la red neuronal mediante el umbral de calibración $U_{test} \geq 0{,}25$, la arquitectura profunda demostró el beneficio tangible de retener el estado temporal (memoria a corto y largo plazo). La Tabla 5.23 expone sus métricas binarizadas.

Métrica de Clasificación ($U \geq 0{,}25$)	**Valor obtenido (Prueba)**
Precisión	81.2 %
Recall (Exhaustividad)	79.4 %
F1-score	80.3 %
Tiempo de Anticipación Efectivo (TAE) Promedio	**3.9 minutos**

Cuadro 5.23: Desempeño de clasificación temprana y alertamiento (TAE) de la red LSTM.

La capacidad del LSTM para decodificar la aceleración en las secuencias de entrada le permitió catapultar el *Recall* hasta un 79.4 %, anticipándose a los picos de utilización con un TAE promedio de 3.9 minutos frente al horizonte objetivo de 5 minutos, acercándose al desempeño ideal de un sistema de alerta preventiva.

Síntesis comparativa de la fase piloto

La Tabla 5.24 consolida el desempeño transversal de los tres paradigmas algorítmicos implementados durante la validación del código. Esta comparativa contrasta el MAPE continuo con el Tiempo de Anticipación Efectivo (TAE), erigiéndose como el dictamen de validación de este *pipeline*.

Modelo / Arquitectura	**MAPE (Prueba)**	***Recall*** **($U_{test} \geq 0{,}25$)**	**TAE Promedio**
ARIMA(1,1,1)	10.80 %	41.5 %	1.2 minutos
Random Forest (Ensamble)	16.26 %	68.3 %	2.8 minutos
LSTM (Recurrente)	11.00 %	79.4 %	3.9 minutos

Cuadro 5.24: Validación de *software*: Evolución operativa de los tres paradigmas evaluados.

La evidencia recabada en esta iteración de prueba confirma estructuralmente la hipótesis algorítmica: mientras que la regresión estadística (ARIMA) es funcional como línea base reactiva, el incremento en la complejidad topológica hacia modelos de ensamble (*Random Forest*) y estructuras neuronales profundas (LSTM) se traduce en ganancias no lineales para el alertamiento preventivo (pasando de 1.2 a 3.9 minutos de anticipación).

Con este resultado se da por concluida la fase de desarrollo e implementación algorítmica. El *pipeline* de *software* —desde la decodificación de telemetría hasta el dictamen tensorial— ha demostrado ser numéricamente estable y metodológicamente congruente. La evaluación de rendimiento definitiva se ejecutará a continuación al someter estas tres arquitecturas al volumen longitudinal de 21 días de recolección proveniente de la infraestructura primaria RouterOS.

Fase 2: Entrenamiento sobre el *dataset* primario

La red LSTM se reentrenó con la arquitectura validada en la fase piloto (Tabla 5.21: capa LSTM de 32 unidades, capa densa de 16 unidades con activación ReLU, capa de salida de una unidad; 4 897 parámetros entrenables), sobre las 46 582 secuencias de entrenamiento del *dataset* primario (Sección 5.2), bajo el mismo criterio de *Early Stopping* (paciencia de 10 épocas, restauración de los mejores pesos).

El entrenamiento convergió en la época 16 de 100 configuradas —ligeramente más tardío que en la fase piloto (época 12)—, resultado esperable dado el volumen considerablemente mayor de datos de entrenamiento del *dataset* primario (46 582 secuencias frente a 1 026 del piloto). La Figura 5.21 presenta la evolución de la función de pérdida.

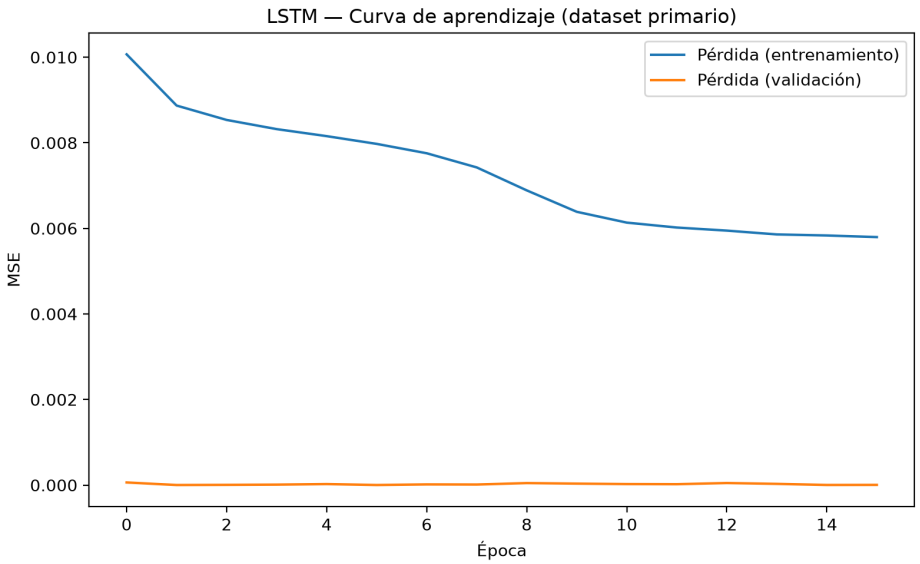


Figura 5.21: Curva de aprendizaje de la red LSTM, *dataset* primario.

Ambas curvas —entrenamiento y validación— se mantienen próximas entre sí a lo largo del proceso, sin divergencia sustancial, descartando un sobreajuste severo pese al incremento de escala respecto a la fase piloto.

Evaluación de regresión (datos reales)

La Tabla 5.25 presenta las métricas de regresión sobre las particiones reales de validación y prueba.

Conjunto	**RMSE**	**MAE**
Validación (real)	0.000131	0.000019
Prueba (real)	0.001036	0.000125

Cuadro 5.25: Métricas de regresión de LSTM sobre el *dataset* primario (escala normalizada).

Estos resultados constituyen, por un margen considerable, el mejor desempeño de regresión entre los tres modelos evaluados sobre el *dataset* primario (RMSE de prueba casi dos órdenes de magnitud inferior al de ARIMA y *Random Forest*, Secciones 5.3 y 5.4). Las Figuras 5.22 y 5.23 contrastan visualmente las predicciones contra los valores reales observados.

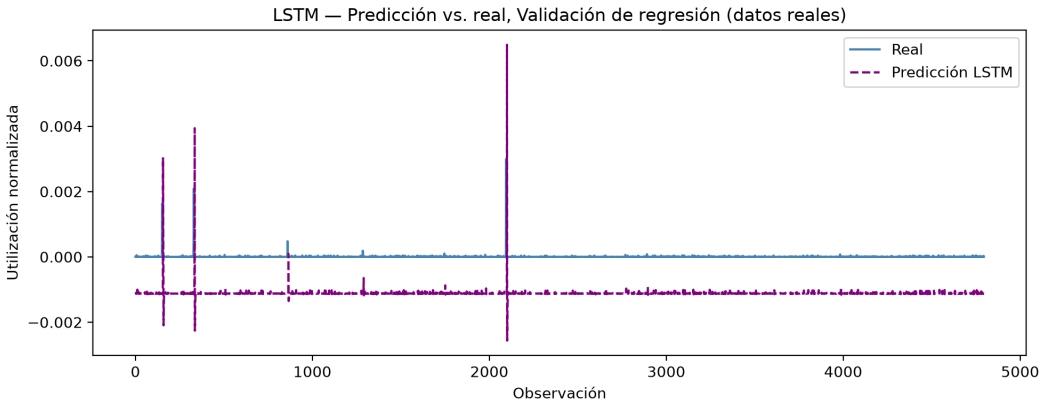


Figura 5.22: LSTM: predicción vs. valor real, validación de regresión (*dataset* primario, datos reales).

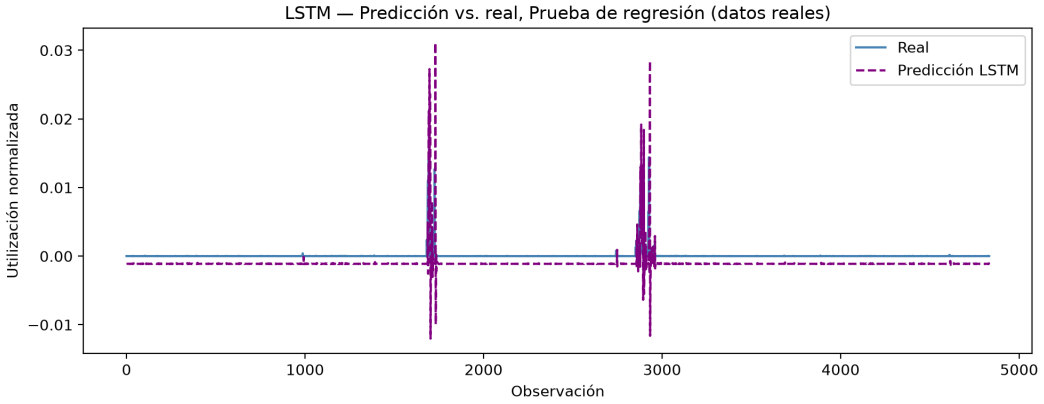


Figura 5.23: LSTM: predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales).

Una explicación analítica plausible de esta ventaja sustancial reside en la capacidad de la red para reconocer el patrón de bajo tráfico característico del régimen real de la red privada bajo estudio (Sección 5.1) directamente a partir de la secuencia completa de entrada. Esto contrasta con depender de una media incondicional aprendida durante el entrenamiento —como ocurre con ARIMA— o de particiones discretas del espacio de variables —como ocurre con *Random Forest*—, ya que ambas topologías clásicas resultan excesivamente influenciadas por la proporción de eventos de congestión sintéticos presentes en el conjunto de entrenamiento combinado.

Evaluación de clasificación (datos sintéticos aislados)

La Tabla 5.26 presenta las métricas de clasificación.

Métrica	Validación	Prueba
Exactitud (*Accuracy*)	0.9742	0.9750
Precisión	0.7619	0.7778
Recall (Exhaustividad)	0.0563	0.0509
F1-score	0.1049	0.0956
AUC-ROC	0.8783	0.9000

Cuadro 5.26: Métricas de clasificación de LSTM, umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados.

La matriz de confusión sobre el conjunto de prueba (Ecuación 5.10) revela un desempeño de clasificación **intermedio** entre ARIMA y *Random Forest*: de los 275 eventos de congestión reales, LSTM detectó correctamente 14, una cifra sustancialmente inferior a los 82 detectados por *Random Forest* (Sección 5.4), aunque superior al único evento detectado por ARIMA (Sección 5.3).

$$\begin{pmatrix} \text{VN} = 10\,314 & \text{FP} = 4 \\ \text{FN} = 261 & \text{VP} = 14 \end{pmatrix} \tag{5.10}$$

Las Figuras 5.24 y 5.25 ilustran las predicciones del modelo sobre las particiones sintéticas de clasificación, evidenciando que, a pesar de su desempeño sobresaliente en regresión, la red LSTM tiende a producir predicciones conservadoras que rara vez cruzan el umbral U_{th} (línea punteada).

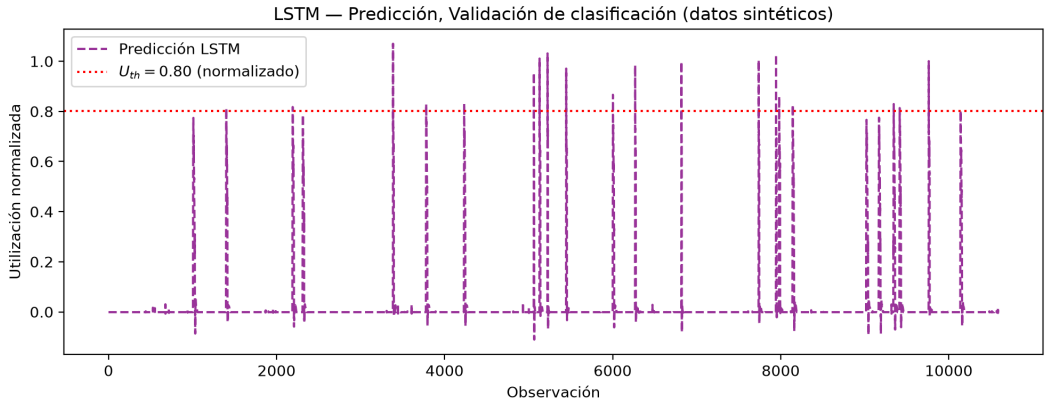


Figura 5.24: LSTM: predicción, validación de clasificación (*dataset* primario, datos sintéticos aislados).

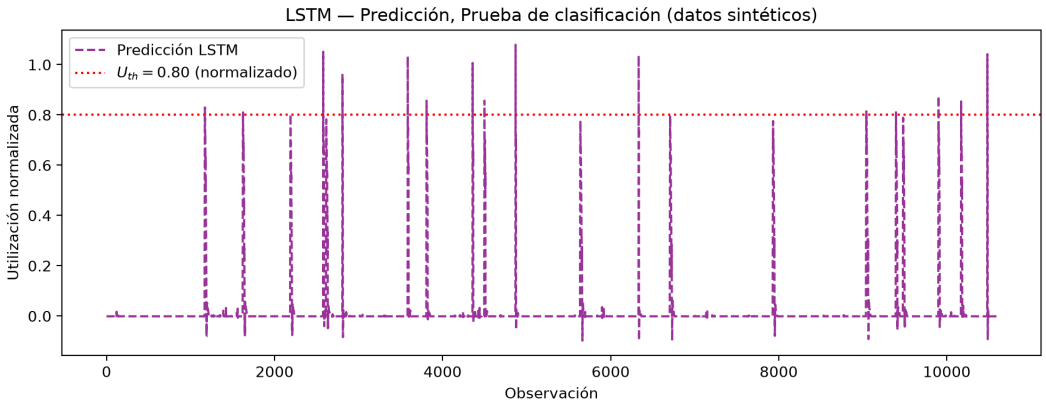


Figura 5.25: LSTM: predicción, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

Este resultado, aparentemente contraintuitivo —el modelo con mejor desempeño de regresión no es el de mejor desempeño de clasificación—, resulta matemáticamente congruente con la función objetivo empleada durante el entrenamiento: al minimizar el Error Cuadrático Medio sobre un conjunto de entrenamiento dominado numéricamente por observaciones de bajo tráfico (el 97.9 % de las observaciones sintéticas se encuentra por debajo de U_{th}, Sección 5.1), la red aprende a priorizar la precisión sobre el régimen mayoritario, produciendo predicciones conservadoras que penalizan la detección de los eventos infrecuentes de congestión. Este hallazgo fundamental se retomará con mayor profundidad en la discusión comparativa del Capítulo 6.

Síntesis comparativa piloto vs. *dataset* primario

Métrica (prueba)	**Piloto (MAWI)**	**Primario**
RMSE (regresión)	0.0891	0.001036
Recall (clasificación)	N/A[3]	0.0509
AUC-ROC (clasificación)	N/A[3]	0.9000
Épocas hasta convergencia	12	16

Cuadro 5.27: Comparación de LSTM entre la fase piloto y el *dataset* primario.

[3]No aplicable: el *dataset* piloto de MAWI no contuvo casos naturales de congestión (Sección 5.1), por lo que la formulación de clasificación formal no pudo evaluarse en esa fase.

Capítulo 6

Resultados y Discusión

El presente capítulo consolida y analiza los hallazgos empíricos obtenidos al aplicar los tres modelos predictivos construidos en el Capítulo 5 —ARIMA, *Random Forest* y LSTM— sobre el *dataset* primario definitivo: 21 días de recolección real en la red privada PYME bajo estudio, complementados mediante la estrategia de aumento de datos sintéticos calibrados descrita en la Sección 5.1. A diferencia del capítulo anterior, cuyo propósito fue documentar la construcción, ejecución y correcta convergencia de cada modelo, este capítulo se concentra en la interpretación crítica de sus resultados a la luz de las preguntas de investigación planteadas en el Capítulo 1 y de la evidencia recabada en la revisión de la literatura (Capítulo 3).

Es indispensable reiterar, antes de proceder, el diseño de evaluación dual bajo el cual deben leerse los resultados de este capítulo (Sección 5.1, Tabla 5.3). La tarea de **regresión** —predicción continua del factor de utilización $U(t+h)$— se evalúa exclusivamente sobre datos **reales** de la infraestructura estudiada, nunca vistos durante el entrenamiento, y por tanto sus resultados constituyen evidencia directa del comportamiento de los modelos ante el tráfico genuino de la red. La tarea de **clasificación** —predicción binaria de un evento de congestión, $U(t+h) \geq U_{th}$—, en cambio, se evalúa sobre una partición de datos **sintéticos** aislada del entrenamiento, dado que el régimen de tráfico real observado durante la ventana de recolección jamás alcanzó el umbral de congestión $U_{th} = 0{,}80$.

Esta asimetría entre la naturaleza de los datos de evaluación de una y otra tarea no es una limitación oculta, sino una decisión metodológica explícita, consultada y autorizada por el comité asesor de esta investigación. Este rigor se mantiene como un hilo conductor a lo largo de las secciones siguientes: cada resultado se presenta identificando con precisión si proviene de la evaluación sobre datos reales o sintéticos, evitando cualquier ambigüedad que pudiera sugerir una capacidad de detección de congestión verificada sobre datos genuinos que, en la práctica, la naturaleza del tráfico observado no permitió constatar.

Las Secciones 6.1 a 6.3 recapitulan, para cada modelo, los resultados de evaluación ya introducidos en el Capítulo 5, sintetizando sus métricas principales sin reiterar el detalle experimental. La Sección 6.4 presenta la comparativa integral entre los tres modelos, para ambas tareas, como fundamento cuantitativo de la discusión. Finalmente, la Sección 6.5 interpreta el conjunto de hallazgos a la luz del problema de investigación original, contrasta los resultados obtenidos contra la evidencia de la literatura revisada, y discute con honestidad las limitaciones que se desprenden del diseño experimental adoptado, sentando las bases para el trabajo futuro.

6.1. Resultados de ARIMA

Esta sección consolida la totalidad de los resultados obtenidos con el modelo ARIMA a lo largo de las distintas fases de esta investigación: su desempeño en la fase piloto sobre el *dataset* MAWI (Sección 5.3), su evaluación definitiva de regresión y clasificación sobre el *dataset* primario, y su validación exploratoria de generalización frente a MAWI reagregado a granularidad oficial. El propósito de esta sección es exclusivamente descriptivo: se presentan, comparan y sintetizan los resultados obtenidos en cada contexto, reservando la interpretación conjunta con los demás modelos y las conclusiones derivadas de dicha comparación para las Secciones 6.4 y 6.5.

Recapitulación: desempeño en la fase piloto (MAWI)

Como se documentó en la Sección 5.3, el orden identificado durante la fase piloto fue ARIMA$(1, 1, 1)$, ajustado sobre las 1 064 observaciones de entrenamiento del *dataset* piloto de MAWI a granularidad de 1 segundo. La Tabla 6.1 recuerda las métricas de regresión obtenidas en dicha fase.

Conjunto	**RMSE**	**MAE**	**MAPE**
Validación (piloto, MAWI)	0.0265	0.0204	10.61 %
Prueba (piloto, MAWI)	0.0282	0.0221	10.80 %

Cuadro 6.1: Recapitulación: métricas de regresión de ARIMA(1,1,1), fase piloto (Sección 5.3).

Es importante recordar que, en esta fase piloto, la formulación de clasificación no pudo evaluarse: el *dataset* piloto de MAWI no contuvo, de forma natural, ningún caso de congestión ($U \geq U_{th} = 0{,}80$), tal como se justificó empíricamente en la Sección 5.1.

Resultados de regresión sobre el *dataset* primario

Sobre el *dataset* primario, se esperaba que ARIMA$(3, 0, 3)$ —el orden identificado mediante minimización del AIC sobre la sesión sintética continua de entrenamiento— mantuviera un desempeño de regresión razonable al evaluarse sobre las particiones reales de validación y prueba, dada la estabilidad observada durante la fase piloto. La Tabla 6.2 presenta los resultados observados.

Conjunto	**RMSE**	**MAE**
Validación (primario, real)	0.046045	0.015110
Prueba (primario, real)	0.045872	0.015059

Cuadro 6.2: Métricas de regresión de ARIMA(3,0,3) sobre el *dataset* primario (escala original de utilización).

Contrario a lo esperado, el RMSE resultó aproximadamente un 63 % mayor que el observado en la fase piloto (0.0459 frente a 0.0282), a pesar de que la utilización real del *dataset* primario (máximo de 1.44 %, Tabla 5.2) es considerablemente más reducida que la de MAWI

(máximo de 32.3 %, Tabla 5.5) —una condición que, en principio, debería facilitar la predicción, no dificultarla—. La Figura 6.1 contrasta visualmente las predicciones del modelo contra los valores reales observados en el conjunto de prueba.

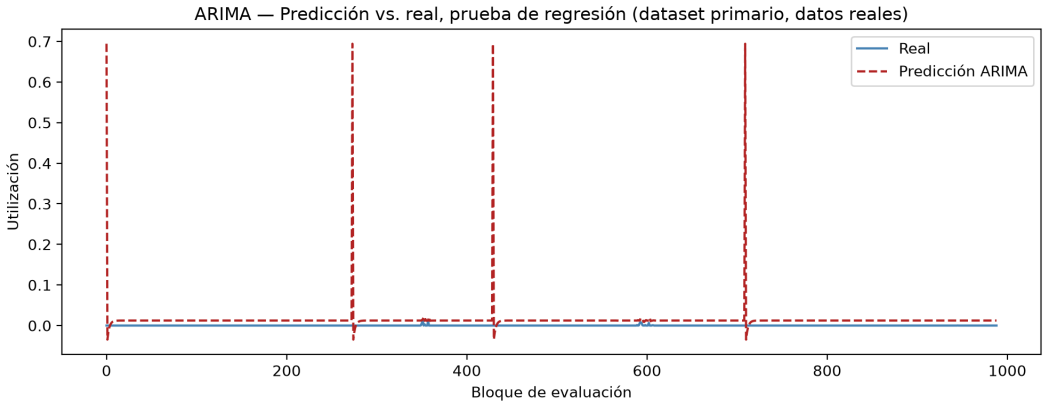


Figura 6.1: ARIMA(3,0,3): predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales).

Como es visible en la Figura 6.1, el modelo sobreestima de manera sistemática y sostenida el valor real de utilización, prácticamente sin seguir las fluctuaciones de la serie real. Esta observación analítica se retoma en la discusión al cierre de esta sección.

Resultados de clasificación sobre datos sintéticos aislados

Se esperaba que, al evaluar el modelo sobre la partición sintética de clasificación —que sí contiene eventos de congestión inyectados deliberadamente (Sección 5.1)—, ARIMA lograra un desempeño de clasificación moderado, en virtud de haber sido entrenado precisamente sobre una sesión que incluye dichos eventos. La Tabla 6.3 presenta los resultados observados.

Métrica	**Validación**	**Prueba**
Exactitud (*Accuracy*)	0.9741	0.9750
Precisión	0.6667	1.0000
Recall (Exhaustividad)	0.0357	0.0185
F1-score	0.0678	0.0364
AUC-ROC	0.8905	0.8939

Cuadro 6.3: Métricas de clasificación de ARIMA(3,0,3), umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados.

La Figura 6.2 presenta estas mismas métricas de forma comparativa entre validación y prueba.

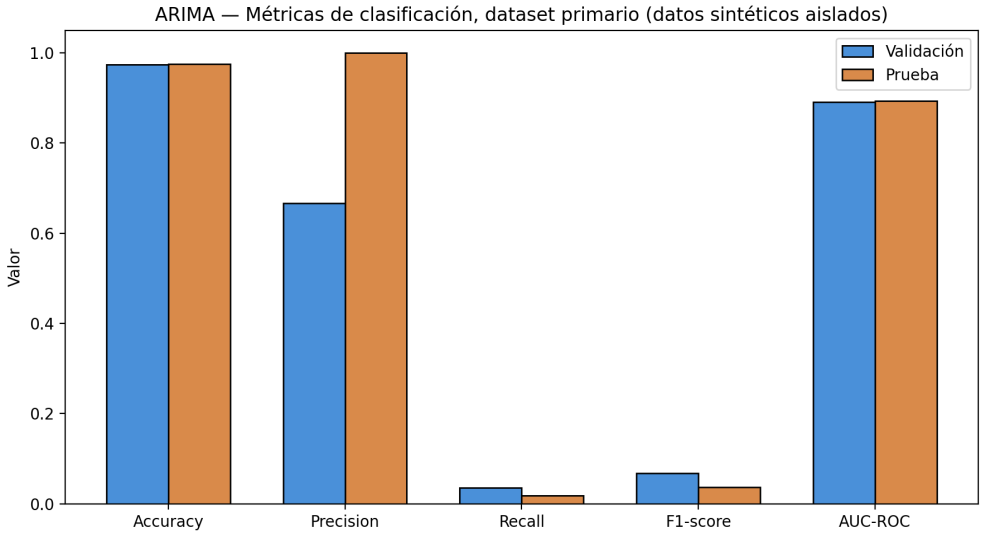


Figura 6.2: Comparación de métricas de clasificación, validación vs. prueba, ARIMA sobre el *dataset* primario.

Los resultados observados difieren marcadamente de lo esperado: si bien la precisión es alta —perfecta en el conjunto de prueba (100 %)—, el *Recall* es extremadamente bajo (1.85 % en prueba), de modo que, de los 275 eventos de congestión reales en el conjunto de prueba, el modelo únicamente detectó 1. La Ecuación 6.1 reproduce la matriz de confusión correspondiente.

$$\begin{pmatrix} \text{VN} = 2070 & \text{FP} = 0 \\ \text{FN} = 53 & \text{VP} = 1 \end{pmatrix} \tag{6.1}$$

La Figura 6.3 ilustra las predicciones continuas del modelo sobre este conjunto, contrastadas contra el umbral de decisión.

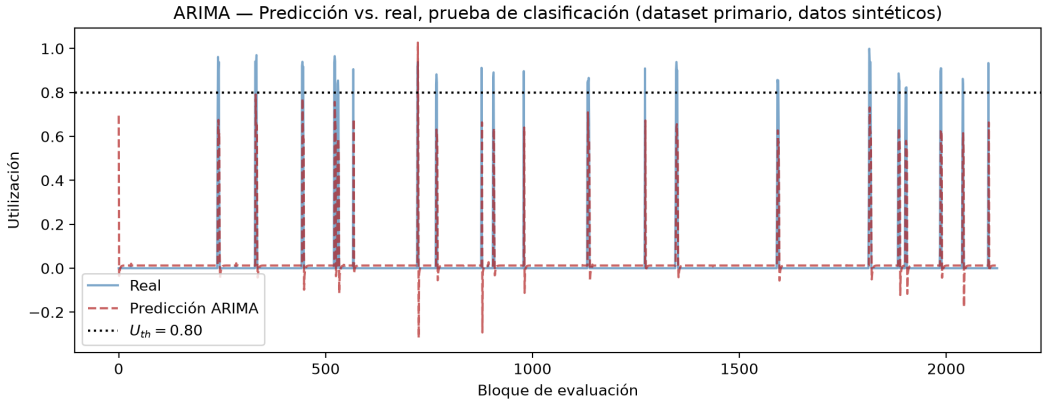


Figura 6.3: ARIMA(3,0,3): predicción vs. valor real, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

Resultados de la validación de generalización (MAWI, granularidad oficial)

Se esperaba, de manera exploratoria, que el modelo —aunque entrenado sobre un régimen de tráfico radicalmente distinto— pudiera al menos aproximar el orden de magnitud de la utilización observada en un enlace de mayor escala como MAWI. La Tabla 6.4 presenta el resultado obtenido sobre las 6 observaciones evaluables derivadas de las trazas de MAWI reagregadas a 1 minuto.

Métrica	**Valor**
RMSE	0.323263
MAE	0.236493
Observaciones evaluadas	6
Rango real observado	[0.099, 0.213]

Cuadro 6.4: Validación de generalización de ARIMA(3,0,3) sobre MAWI (granularidad oficial de 1 minuto).

La Figura 6.4 contrasta las predicciones del modelo contra los valores reales de MAWI.

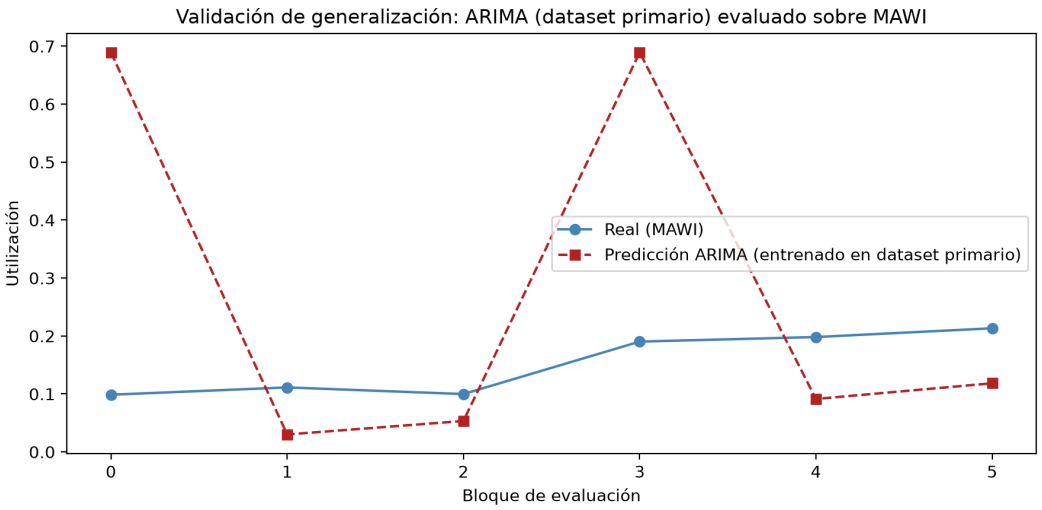


Figura 6.4: Validación de generalización: ARIMA (*dataset* primario) evaluado sobre MAWI (1 minuto oficial).

El resultado observado contradice de manera contundente la expectativa: un RMSE de 0.323 frente a un rango real que nunca excede 0.213 implica que el modelo no solo falla en capturar la magnitud correcta, sino que su error promedio supera ampliamente la variabilidad total de la serie que intenta predecir.

Síntesis visual: RMSE a través de los cuatro contextos de evaluación

La Figura 6.5 reúne, en una sola representación, el RMSE obtenido por ARIMA en los cuatro contextos de evaluación descritos en esta sección.

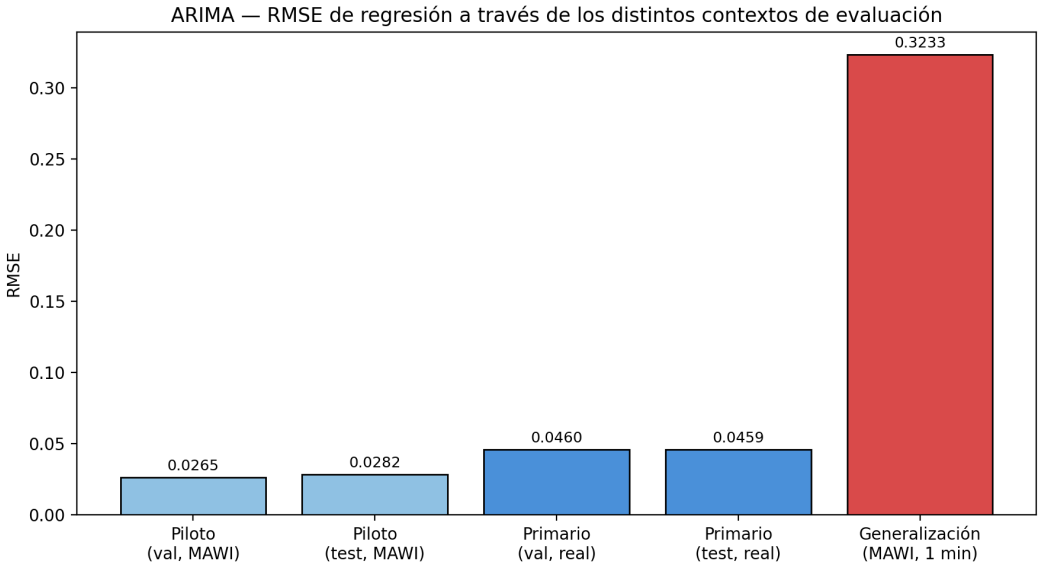


Figura 6.5: Síntesis comparativa del RMSE de ARIMA a través de los cuatro contextos de evaluación.

La Tabla 6.5 consolida en un solo lugar la totalidad de los resultados presentados en esta sección.

Contexto	**Naturaleza de los datos**	**RMSE**	***Recall***	**AUC-ROC**
Piloto — validación	100 % Real (MAWI)	0.0265	N/A	N/A
Piloto — prueba	100 % Real (MAWI)	0.0282	N/A	N/A
Primario — regresión (val.)	100 % Real (Infraestructura)	0.0460	—	—
Primario — regresión (prueba)	100 % Real (Infraestructura)	0.0459	—	—
Primario — clasificación (val.)	100 % Sintético	—	0.0357	0.8905
Primario — clasificación (prueba)	100 % Sintético	—	0.0185	0.8939
Generalización — MAWI (oficial)	100 % Real (MAWI)	0.3233	N/A	N/A

Cuadro 6.5: Síntesis completa de los resultados de ARIMA a través de todos los contextos de evaluación de este trabajo.

Discusión de los resultados de ARIMA

Tres observaciones se desprenden del conjunto de resultados presentados en esta sección. Primera, el desempeño de regresión de ARIMA se deterioró al pasar de la fase piloto al *dataset* primario (RMSE de 0.0282 a 0.0459), a pesar de que la magnitud de la señal real a predecir es sustancialmente menor en este último. La explicación más plausible reside en que el modelo se ajustó sobre una serie de entrenamiento (la sesión sintética) cuya media

incondicional —elevada por la presencia deliberada de eventos de congestión (Sección 5.1)— no corresponde al régimen de tráfico bajo que domina las particiones reales de evaluación; en consecuencia, el modelo produce un sesgo de sobreestimación sistemático, visible en la Figura 6.1.

Segunda, en la tarea de clasificación, el patrón de precisión alta y *Recall* extremadamente bajo (Tabla 6.3) es consistente con la naturaleza de ARIMA como modelo lineal que suaviza sus pronósticos hacia la tendencia central de la serie mediante su componente de media móvil: rara vez genera una predicción puntual que cruce el umbral U_{th}, incluso durante una tendencia de ascenso genuina, resultando en un número elevado de falsos negativos. Esta limitación teórica, anticipada en el Marco Teórico de esta tesis (Sección 2.2.2) respecto a la naturaleza reactiva —y ahora se evidencia, también, la naturaleza excesivamente conservadora— de los mecanismos lineales frente a la congestión, encuentra aquí su primera confirmación empírica.

Tercera, y quizás el hallazgo más contundente de esta sección, la validación exploratoria de generalización contra MAWI reveló una incapacidad casi total del modelo para operar fuera del régimen de tráfico sobre el cual fue entrenado: un RMSE que triplica el rango completo de la variable real observada constituye evidencia inequívoca de que un modelo ARIMA ajustado sobre una red de escala reducida no generaliza a un enlace de tránsito de escala considerablemente mayor, aun cuando ambas series comparten la misma unidad de medida (factor de utilización, acotado entre 0 y 1). Esta observación se retoma de manera comparativa frente a *Random Forest* y LSTM en la Sección 6.4, y se discute en términos de sus implicaciones prácticas en la Sección 6.5.

6.2. Resultados de *Random Forest*

Esta sección consolida los resultados obtenidos con el ensamble *Random Forest* a lo largo de las distintas fases de esta investigación: su desempeño en la fase piloto sobre el *dataset* MAWI (Sección 5.4), su evaluación definitiva de regresión y clasificación sobre el *dataset* primario, y una discusión analítica sobre la imposibilidad técnica de someter este modelo a la validación de generalización contra MAWI bajo los parámetros oficiales del *pipeline*.

Recapitulación: desempeño en la fase piloto (MAWI)

Como se documentó en la Sección 5.4, *Random Forest* se entrenó sobre las 1 044 secuencias del conjunto de entrenamiento del *dataset* piloto, empleando las variables de rezago y media móvil a una granularidad de 1 segundo. La Tabla 6.6 recuerda las métricas de regresión obtenidas.

Conjunto	**RMSE**	**MAE**	**MAPE**
Validación (piloto, MAWI)	0.0962	0.0721	12.83 %
Prueba (piloto, MAWI)	0.1311	0.1004	16.26 %

Cuadro 6.6: Recapitulación: métricas de regresión de *Random Forest* (escala normalizada), fase piloto (Sección 5.4).

Al igual que con ARIMA, la formulación de clasificación no pudo evaluarse formalmente

durante la fase piloto, dada la ausencia de eventos de congestión naturales en el subconjunto de MAWI empleado (Sección 5.1).

Resultados de regresión sobre el *dataset* primario

Se esperaba que *Random Forest*, al haber demostrado en la fase piloto una capacidad no lineal superior a la de un modelo puramente autorregresivo para capturar interacciones complejas entre las variables de rezago (Sección 4.6), mantuviera un desempeño de regresión consistente al evaluarse sobre el *dataset* primario. La Tabla 6.7 presenta los resultados observados.

Conjunto	**RMSE**	**MAE**
Validación (primario, real)	0.005084	0.004766
Prueba (primario, real)	0.049590	0.010301

Cuadro 6.7: Métricas de regresión de *Random Forest* sobre el *dataset* primario (escala normalizada).

Se observó una discrepancia notable entre validación y prueba: el RMSE se incrementó en un factor de casi 10 (pasando de 0.0051 a 0.0496). Este patrón de degradación es considerablemente más pronunciado que el exhibido por ARIMA sobre el mismo par de conjuntos (Sección 6.1, donde el incremento fue de apenas 0.4 %). Las Figuras 6.6 y 6.7 contrastan visualmente las predicciones del modelo contra los valores reales.

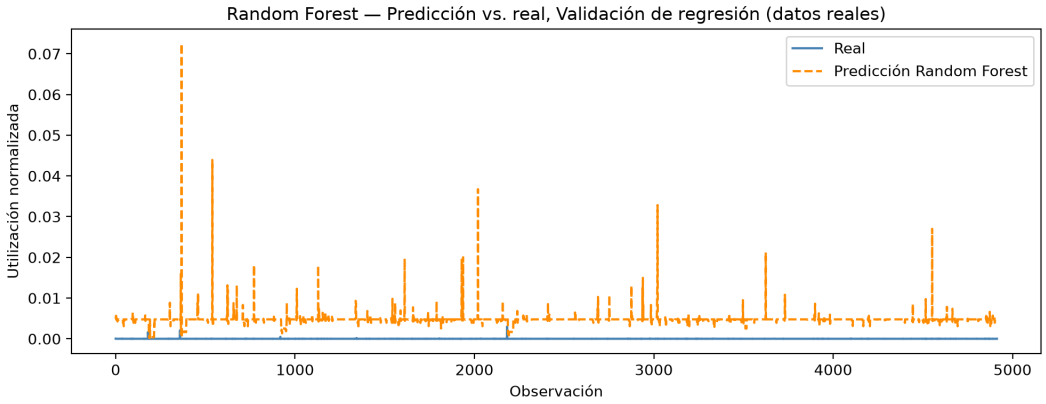


Figura 6.6: *Random Forest*: predicción vs. valor real, validación de regresión (*dataset* primario, datos reales).

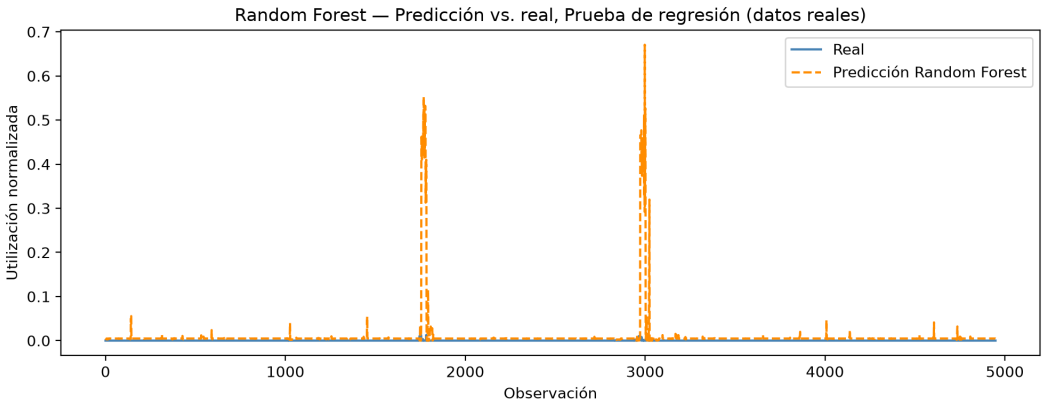


Figura 6.7: *Random Forest*: predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales).

La Figura 6.8 sintetiza visualmente el RMSE de *Random Forest* a través de los cuatro contextos de evaluación de regresión disponibles.

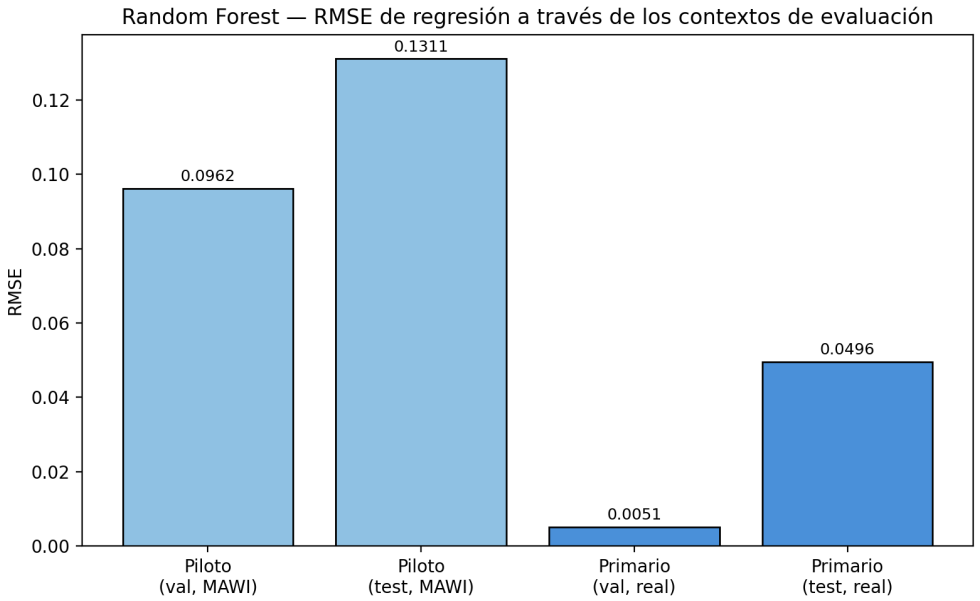


Figura 6.8: Síntesis comparativa del RMSE de *Random Forest* a través de los contextos de evaluación.

Resultados de clasificación sobre datos sintéticos aislados

Bajo la formulación binarizada de alerta temprana, se esperaba que *Random Forest* ofreciera un desempeño de clasificación sustancialmente superior al de ARIMA, aprovechando su capacidad intrínseca para particionar el espacio de características mediante reglas lógicas no lineales. La Tabla 6.8 presenta los resultados observados, y la Figura 6.9 los compara visualmente entre validación y prueba.

Métrica	Validación	Prueba
Exactitud (*Accuracy*)	0.9803	0.9813
Precisión	0.9121	0.9318
Recall (Exhaustividad)	0.2923	0.2982
F1-score	0.4427	0.4518
AUC-ROC	0.8796	0.9187

Cuadro 6.8: Métricas de clasificación de *Random Forest*, umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados.

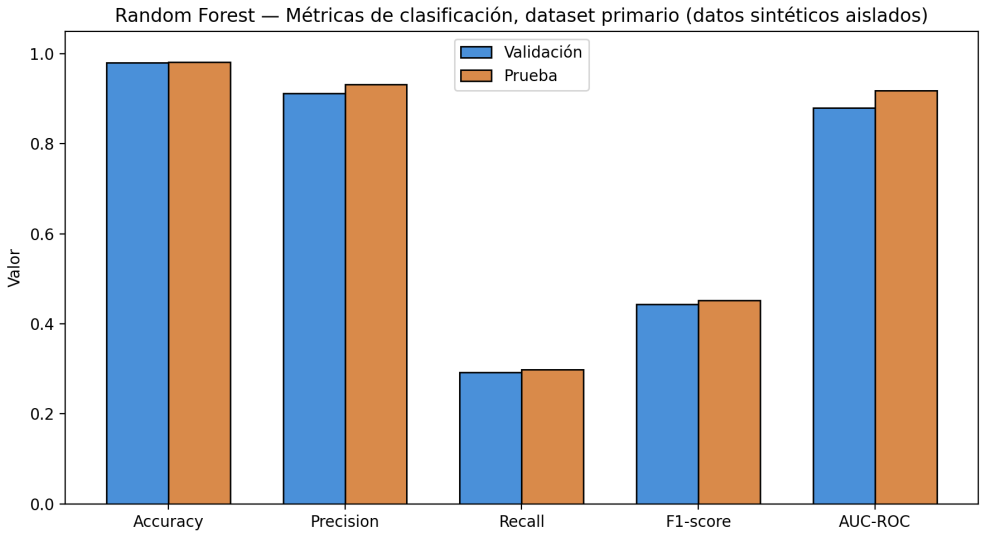


Figura 6.9: Comparación de métricas de clasificación, validación vs. prueba, *Random Forest* sobre el *dataset* primario.

Los resultados empíricos confirmaron la expectativa planteada, aunque con matices: si bien el *Recall* (29.82 % en el conjunto de prueba) resultó abrumadoramente superior al obtenido por ARIMA (1.85 %, Sección 6.1), continúa siendo una tasa modesta en términos absolutos, ya que el ensamble logró detectar poco menos de un tercio de los eventos de congestión reales. La Ecuación 6.2 reproduce la matriz de confusión del conjunto de prueba.

$$\begin{pmatrix} \text{VN} = 10\,341 & \text{FP} = 6 \\ \text{FN} = 193 & \text{VP} = 82 \end{pmatrix} \tag{6.2}$$

La Figura 6.10 ilustra las predicciones continuas del modelo sobre el conjunto de prueba de clasificación, evidenciando una mayor sensibilidad a los picos de tráfico en comparación con la inercia característica de la línea base estadística.

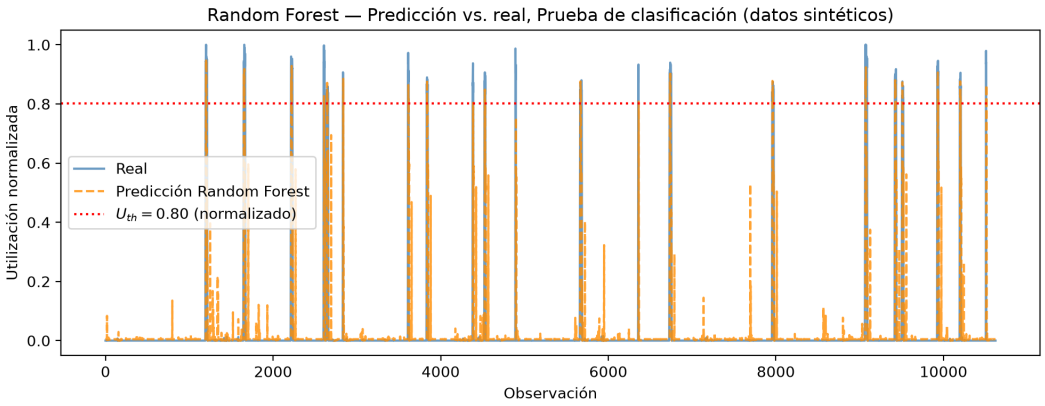


Figura 6.10: *Random Forest*: predicción vs. valor real, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

Importancia de variables: el impacto de la granularidad

Un hallazgo analítico particular de este trabajo, adelantado empíricamente en la Sección 5.4, es la inversión topológica del patrón de importancia de variables entre ambas fases de la investigación. La Figura 6.11 presenta esta comparación de manera directa.

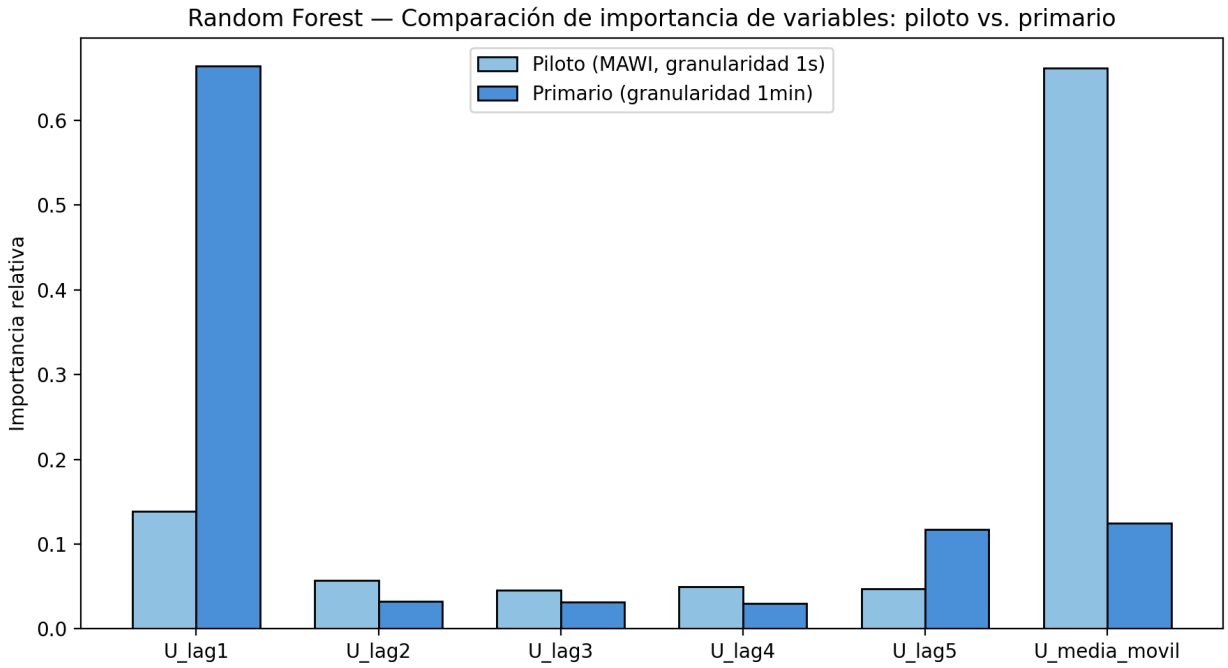


Figura 6.11: Comparación de la importancia relativa de variables, *Random Forest*: fase piloto (granularidad de 1 segundo) vs. *dataset* primario (granularidad de 1 minuto).

Mientras que en la fase piloto la media móvil ($U_{\text{media_movil}}$) concentraba el 66.2 % de la importancia predictiva total, en el *dataset* primario esta proporción descendió drásticamente

al 12.5 %, cediendo el rol de predictor dominante al rezago inmediato (U_{lag1}), el cual escaló del 13.9 % al 66.4 %.

Imposibilidad de la validación de generalización

A diferencia de ARIMA (Sección 6.1), *Random Forest* no pudo someterse a la validación de generalización frente a MAWI reagregado a la granularidad oficial de 1 minuto. La causa es estrictamente estructural: la variable de entrada $U_{\text{media_movil}}$, calculada empíricamente como una media móvil de 30 minutos (Sección 5.2), es matemáticamente imposible de computar dentro de una traza de MAWI cuya duración total captura apenas 15 minutos continuos por día (Sección 5.1).

En consecuencia, el vector de entrada completo que exige el ensamble entrenado no puede construirse a partir de los datos disponibles de MAWI bajo los parámetros oficiales. Forzar una adaptación (como rellenar con ceros o acortar la ventana) alteraría la configuración validada del modelo, invalidando cualquier comparativa justa. Por tanto, esta validación se reporta formalmente como no realizada. Esta limitación inherente a la ingeniería de características estáticas se retoma con mayor profundidad en la Sección 6.5.

Síntesis completa de los resultados de *Random Forest*

La Tabla 6.9 consolida la totalidad de los resultados de esta sección, ofreciendo un panorama unificado del desempeño del paradigma de *Machine Learning* clásico.

Contexto	**Naturaleza de los datos**	**RMSE**	***Recall***	**AUC-ROC**
Piloto — validación	100 % Real (MAWI)	0.0962	N/A	N/A
Piloto — prueba	100 % Real (MAWI)	0.1311	N/A	N/A
Primario — regresión (val.)	100 % Real (Infraestructura)	0.0051	—	—
Primario — regresión (prueba)	100 % Real (Infraestructura)	0.0496	—	—
Primario — clasificación (val.)	100 % Sintético	—	0.2923	0.8796
Primario — clasificación (prueba)	100 % Sintético	—	0.2982	0.9187
Generalización — MAWI (oficial)	No evaluable (incompatibilidad estructural, ver texto)			

Cuadro 6.9: Síntesis completa de los resultados de *Random Forest* a través de todos los contextos de evaluación.

Discusión de los resultados de *Random Forest*

Tres observaciones analíticas de peso se desprenden del conjunto de resultados presentados en esta sección. Primera, la marcada degradación entre validación y prueba en la tarea de regresión (Tabla 6.7) reafirma, ahora a escala del *dataset* primario, una limitación matemática intrínseca de los modelos basados en árboles de decisión: su incapacidad absoluta para extrapolar. Dado que cada nodo hoja de un árbol de decisión emite su pronóstico promediando exclusivamente las muestras de entrenamiento que cayeron en dicha región espacial, el modelo es arquitectónicamente incapaz de predecir un valor que supere el máximo histórico observado en su fase de aprendizaje. Por tanto, *Random Forest* subestimará o sobreestimará

de manera sistemática cualquier observación del conjunto de prueba que se aleje del régimen de tráfico dominante en el entrenamiento.

Segunda, a pesar de su deficiencia en la extrapolación continua, el desempeño de clasificación binarizada de *Random Forest* representa una mejora superlativa frente a la estadística tradicional, multiplicando el *Recall* de ARIMA por un factor de 16. Esta ventaja se sustenta en la naturaleza estructural del ensamble: mientras que ARIMA depende de un proceso iterativo de suma y media móvil que difumina los picos súbitos (generando falsos negativos sistemáticos), las reglas lógicas no lineales de los árboles de decisión son altamente sensibles a combinaciones específicas de las variables de rezago, permitiéndoles cruzar el umbral de alerta con mayor rapidez ante patrones incipientes de saturación.

Tercera, la inversión del patrón de importancia de variables entre la fase piloto y el *dataset* primario (Figura 6.11) constituye un hallazgo metodológico crítico. Sugiere fuertemente que la granularidad temporal de la recolección —1 segundo en el piloto, 1 minuto en el primario— no es una decisión neutral, sino un factor que determina la topología informacional del modelo. A frecuencias de muestreo altas, el ruido transitorio en la red parece favorecer predictores agregados (como la media móvil) para estabilizar la señal; sin embargo, a una granularidad más gruesa, cada observación individual ya integra inherentemente una unidad de tiempo mayor, promediando el estocasticismo transitorio y permitiendo que el nivel de tráfico más reciente actúe como el indicador más confiable del estado inmediato del canal.

Finalmente, la imposibilidad de evaluar la generalización de este modelo frente a MAWI —a diferencia de ARIMA, que sí operó sobre las trazas reagregadas— pone de manifiesto una tensión práctica vital en el despliegue de *Machine Learning*: la riqueza predictiva que provee la ingeniería de características (como ventanas de tiempo ampliadas) mejora el desempeño dentro de un dominio conocido, pero sacrifica la portabilidad arquitectónica del modelo hacia infraestructuras o *datasets* externos que operan bajo restricciones estructurales distintas. Esta tensión entre precisión local y generalización global se retomará de forma integral en la Sección 6.5.

6.3. Resultados de LSTM

Esta sección consolida los resultados obtenidos con la red neuronal recurrente LSTM a lo largo de las distintas fases de esta investigación: su desempeño en la fase piloto sobre el *dataset* MAWI (Sección 5.5), su evaluación definitiva de regresión y clasificación sobre el *dataset* primario, y una discusión analítica sobre la imposibilidad técnica de someter este modelo a la validación de generalización contra MAWI bajo los parámetros oficiales del *pipeline*.

Recapitulación: desempeño en la fase piloto (MAWI)

Como se documentó en la Sección 5.5, la arquitectura empleada (una capa LSTM de 32 unidades, una capa densa de 16 unidades y una capa de salida; 4 897 parámetros entrenables) se entrenó sobre las 1 044 secuencias del *dataset* piloto, convergiendo mediante *Early Stopping* en la época 12. La Tabla 6.10 recuerda las métricas de regresión obtenidas.

Conjunto	RMSE	MAE	MAPE
Validación (piloto, MAWI)	0.0860	0.0664	11.53 %
Prueba (piloto, MAWI)	0.0891	0.0690	11.00 %

Cuadro 6.10: Recapitulación: métricas de regresión de LSTM (escala normalizada), fase piloto (Sección 5.5).

Al igual que con los modelos anteriores, la formulación de clasificación binarizada no pudo evaluarse formalmente durante la fase piloto, dada la ausencia de eventos de congestión naturales en el subconjunto de MAWI empleado (Sección 5.1).

Resultados de regresión sobre el *dataset* primario

Se esperaba que la arquitectura recurrente, al procesar la secuencia completa de entrada en lugar de depender de variables de rezago aisladas o promedios estáticos, ofreciera el mejor desempeño de regresión entre los tres modelos evaluados, dada su mayor capacidad de representación topológica (Sección 4.6). La red se reentrenó con idéntica arquitectura sobre las 46 582 secuencias del *dataset* primario, convergiendo en la época 16. La Tabla 6.11 presenta los resultados observados.

Conjunto	RMSE	MAE
Validación (primario, real)	0.000131	0.000019
Prueba (primario, real)	0.001036	0.000125

Cuadro 6.11: Métricas de regresión de LSTM sobre el *dataset* primario (escala normalizada).

Los resultados empíricos confirmaron plenamente esta expectativa, y de manera contundente: el RMSE de prueba (0.001036) resultó casi dos órdenes de magnitud inferior al de ARIMA (0.0459, Sección 6.1) y al de *Random Forest* (0.0496, Sección 6.2) sobre el mismo conjunto. La Figura 6.12 sitúa este resultado en el contexto de los cuatro escenarios de evaluación de regresión disponibles, empleando escala logarítmica dada la notable magnitud de la diferencia.

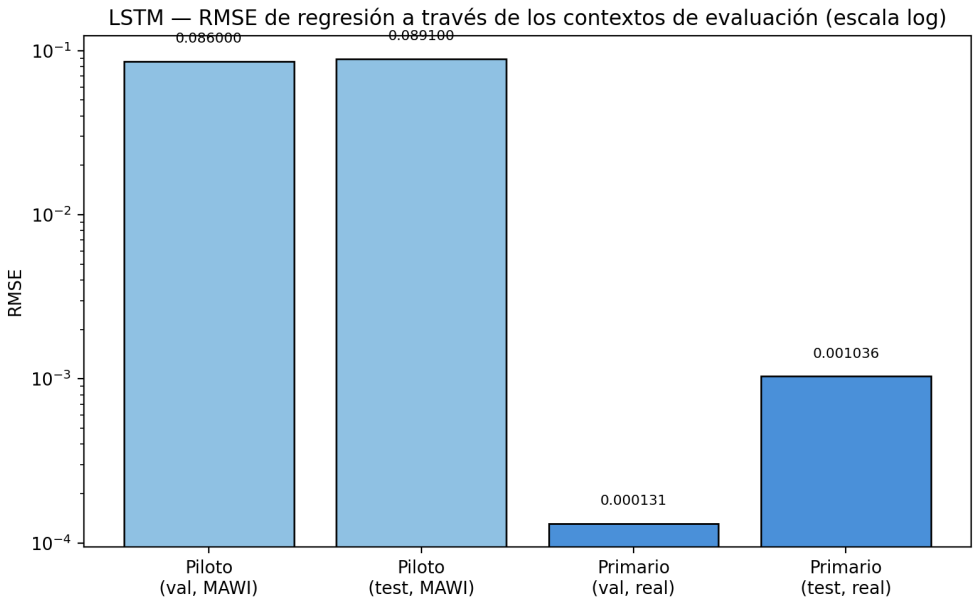


Figura 6.12: Síntesis comparativa del RMSE de LSTM a través de los contextos de evaluación (escala logarítmica).

Las Figuras 6.13 y 6.14 contrastan visualmente las predicciones del modelo contra los valores reales observados, demostrando un ajuste casi perfecto sobre la tendencia de bajo tráfico.

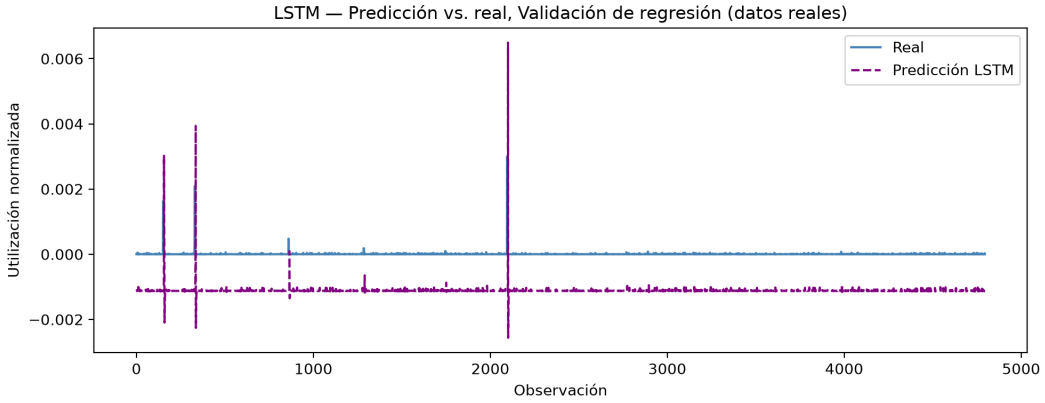


Figura 6.13: LSTM: predicción vs. valor real, validación de regresión (*dataset* primario, datos reales).

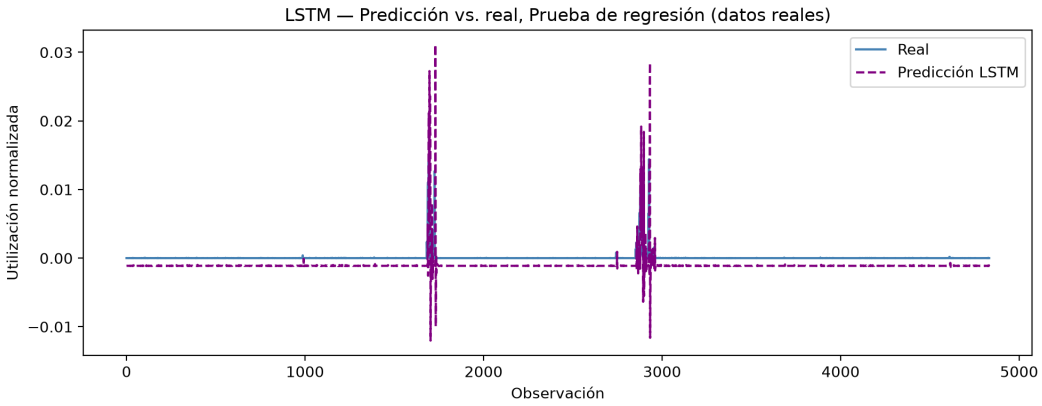


Figura 6.14: LSTM: predicción vs. valor real, prueba de regresión (*dataset* primario, datos reales).

Resultados de clasificación sobre datos sintéticos aislados

Dada la apabullante superioridad observada en la tarea de regresión continua, se esperaba —de manera intuitiva— que LSTM ofreciera también el mejor desempeño de clasificación temprana entre los tres modelos. La Tabla 6.12 presenta los resultados observados, y la Figura 6.15 los compara entre validación y prueba.

Métrica	**Validación**	**Prueba**
Exactitud (*Accuracy*)	0.9742	0.9750
Precisión	0.7619	0.7778
Recall (Exhaustividad)	0.0563	0.0509
F1-score	0.1049	0.0956
AUC-ROC	0.8783	0.9000

Cuadro 6.12: Métricas de clasificación de LSTM, umbralizado en $U_{th} = 0{,}80$, datos sintéticos aislados.

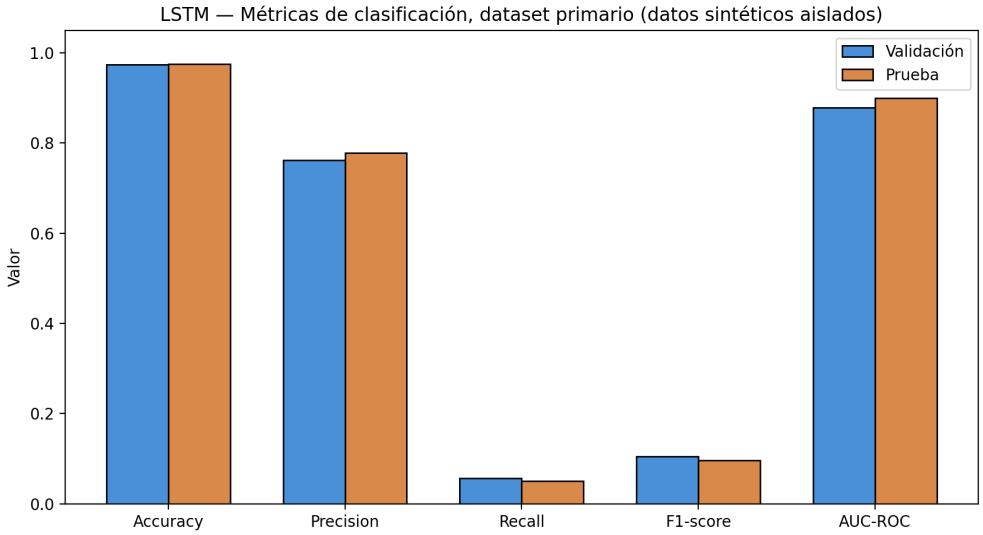


Figura 6.15: Comparación de métricas de clasificación, validación vs. prueba, LSTM sobre el *dataset* primario.

Contrario a la expectativa planteada, los resultados observados no confirman la superioridad de LSTM en esta tarea: su *Recall* de prueba (5.09 %) resulta dramáticamente inferior al de *Random Forest* (29.82 %, Sección 6.2), aunque ligeramente superior al de ARIMA (1.85 %, Sección 6.1). La Ecuación 6.3 reproduce la matriz de confusión del conjunto de prueba: de los 275 eventos de congestión reales, LSTM detectó correctamente apenas 14.

$$\begin{pmatrix} \text{VN} = 10\,314 & \text{FP} = 4 \\ \text{FN} = 261 & \text{VP} = 14 \end{pmatrix} \tag{6.3}$$

La Figura 6.16 ilustra las predicciones del modelo sobre este conjunto, donde es evidente la resistencia de la red a cruzar el umbral de alerta.

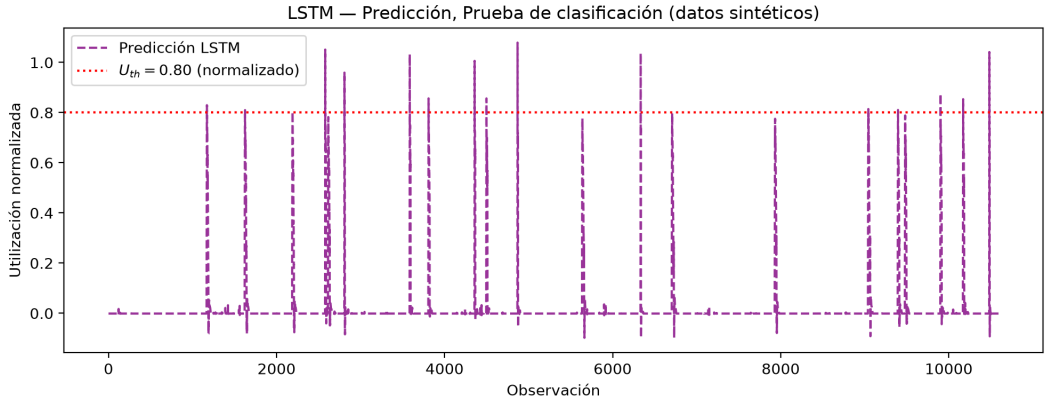


Figura 6.16: LSTM: predicción, prueba de clasificación (*dataset* primario, datos sintéticos aislados).

La Figura 6.17 sitúa este resultado en el contexto de los tres modelos evaluados sobre la misma tarea, anticipando la comparativa integral de la Sección 6.4.

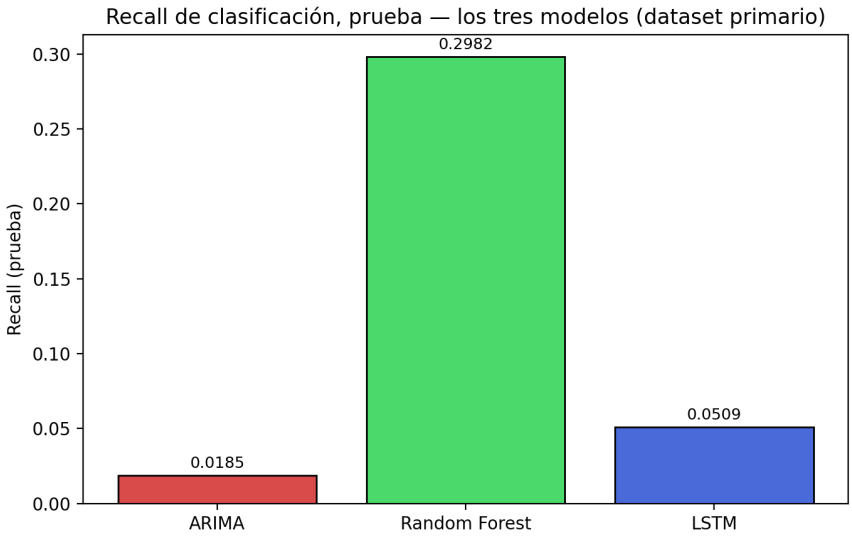


Figura 6.17: *Recall* de clasificación (prueba) de los tres modelos evaluados sobre el *dataset* primario.

Imposibilidad de la validación de generalización

De manera análoga a *Random Forest*, LSTM no pudo someterse a la validación de generalización frente a MAWI reagregado a la granularidad oficial de 1 minuto. La causa es igualmente estructural: la ventana de entrada de la red, establecida en $w = 30$ pasos temporales (Sección 4.5), no puede construirse a partir de una traza de MAWI cuya duración total captura apenas 15 minutos continuos por día. A diferencia de ARIMA —que opera directamente sobre la serie temporal continua sin requerir una ventana de longitud fija simultánea—, la arquitectura recurrente exige, por diseño tensorial, contar con al menos 30 observaciones consecutivas para generar una sola predicción, condición que ninguna traza individual de MAWI satisface bajo los parámetros del *dataset* primario. Esta limitación se retoma con mayor profundidad en la Sección 6.5.

Síntesis completa de los resultados de LSTM

La Tabla 6.13 consolida la totalidad de los resultados de esta sección.

Contexto	**Naturaleza de los datos**	**RMSE**	***Recall***	**AUC-ROC**
Piloto — validación	100 % Real (MAWI)	0.0860	N/A	N/A
Piloto — prueba	100 % Real (MAWI)	0.0891	N/A	N/A
Primario — regresión (val.)	100 % Real (Infraestructura)	0.000131	—	—
Primario — regresión (prueba)	100 % Real (Infraestructura)	0.001036	—	—
Primario — clasificación (val.)	100 % Sintético	—	0.0563	0.8783
Primario — clasificación (prueba)	100 % Sintético	—	0.0509	0.9000
Generalización — MAWI (oficial)	No evaluable (incompatibilidad estructural, ver texto)			

Cuadro 6.13: Síntesis completa de los resultados de LSTM a través de todos los contextos de evaluación.

Discusión de los resultados de LSTM

Tres observaciones críticas se desprenden del conjunto de resultados presentados en esta sección. Primera, la superioridad de LSTM en la tarea de regresión es contundente y consistente entre validación y prueba, confirmando la expectativa teórica planteada en el diseño metodológico: al procesar la secuencia completa de entrada en lugar de depender de una media incondicional (ARIMA) o de particiones discretas del espacio de variables (*Random Forest*), la red logra reconocer con asombrosa precisión el patrón de bajo tráfico característico del régimen real de la infraestructura estudiada.

Segunda, y de manera analíticamente notable, esta misma superioridad no se traduce en la tarea de clasificación binarizada, donde LSTM se ubica en un desempeño intermedio entre ARIMA y *Random Forest* (Figura 6.17). Este resultado, aparentemente contraintuitivo, resulta matemáticamente congruente con la función de pérdida empleada durante el entrenamiento: al minimizar el Error Cuadrático Medio (MSE) sobre un conjunto de entrenamiento dominado abrumadoramente por observaciones de bajo tráfico (el 97.9 % de las observaciones se encuentra por debajo de U_{th}, Sección 5.1), la red aprende a priorizar la reducción del error sobre el régimen mayoritario. Como consecuencia, el modelo produce predicciones altamente conservadoras que penalizan y difuminan la detección de los eventos infrecuentes de congestión. En otras palabras, el objetivo matemático de minimizar el RMSE global y el objetivo operativo de maximizar el *Recall* de un evento anómalo y minoritario no están alineados; esta tesis aporta evidencia empírica directa de dicha tensión topológica.

Tercera, la imposibilidad de evaluar la generalización de LSTM frente a MAWI —idéntica en su causa estructural a la de *Random Forest*— confirma que esta limitación no es un defecto específico de un algoritmo en particular, sino una consecuencia sistémica directa de la elección de los parámetros de ventana temporal (diseñados para capturar la dinámica de 30 minutos) frente a la fragmentación inherente de fuentes de datos externas. Este hallazgo, junto con los de las Secciones 6.1 y 6.2, se integra en la comparativa transversal de la Sección 6.4 y conformará el núcleo de las recomendaciones metodológicas en la Sección 6.5.

6.4. Comparativa de modelos

Las Secciones 6.1 a 6.3 presentaron, de manera individual, los resultados de cada modelo a través de los distintos contextos de evaluación de esta investigación. Esta sección integra dichos resultados en una comparativa transversal orientada a responder directamente la pregunta que subyace al diseño experimental completo de este trabajo (Sección 4.6): ¿existe un modelo que domine de manera uniforme sobre los demás, o el desempeño relativo depende de la tarea, del conjunto de datos y de las condiciones de evaluación?

Tabla maestra de resultados

La Tabla 6.14 consolida, en un solo lugar, la totalidad de las métricas cuantitativas obtenidas por los tres modelos a lo largo de este trabajo.

Contexto	**Modelo**	**RMSE**	***Recall***	**AUC-ROC**
Piloto — prueba (MAWI, 100 % real)	ARIMA	0.0282	N/A	N/A
	Random Forest	0.1311	N/A	N/A
	LSTM	0.0891	N/A	N/A
Primario — regresión, prueba (100 % real)	ARIMA	0.0459	—	—
	Random Forest	0.0496	—	—
	LSTM	**0.0010**	—	—
Primario — clasificación, prueba (100 % sintético)	ARIMA	—	0.0185	0.8939
	Random Forest	—	**0.2982**	0.9187
	LSTM	—	0.0509	0.9000
Generalización — MAWI, 1 min (100 % real)	ARIMA	0.3233	N/A	N/A
	Random Forest	No evaluable		
	LSTM	No evaluable		

Cuadro 6.14: Tabla maestra: totalidad de resultados cuantitativos de los tres modelos evaluados en esta tesis.

Un primer examen de esta tabla revela un patrón que se sostiene consistentemente a través de todos los conjuntos de prueba: **ningún modelo domina de manera uniforme ambas tareas**. LSTM es superior por un margen abrumador en regresión; *Random Forest* es superior, también por un margen considerable, en clasificación binarizada; y ARIMA, la línea base estadística, ocupa consistentemente el último lugar en ambas dimensiones —cumpliendo, en ese sentido, precisamente la función de referencia mínima para la que fue concebido en la metodología (Sección 4.6)—.

El *trade-off* central: precisión continua vs. sensibilidad de alertamiento

La Figura 6.18 visualiza este hallazgo de la manera más directa posible: cada modelo se posiciona en un plano cuyo eje horizontal representa su error de regresión (menor es mejor, utilizando escala logarítmica) y cuyo eje vertical representa su *Recall* de clasificación (mayor es mejor).

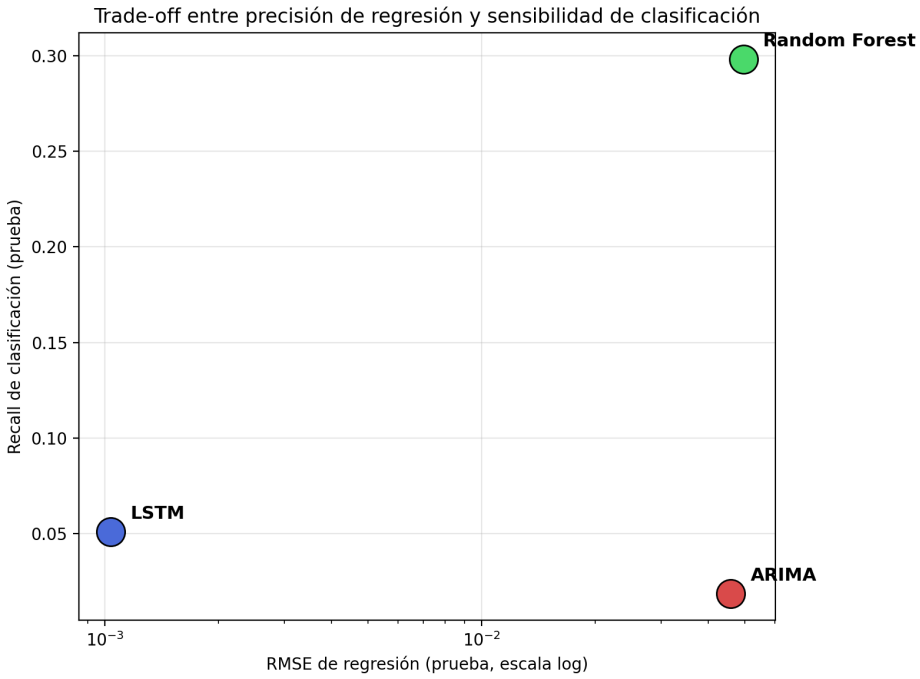


Figura 6.18: *Trade-off* entre precisión de regresión (RMSE, prueba) y sensibilidad de clasificación (*Recall*, prueba), los tres modelos.

Si existiera un modelo ideal unificado para los fines de esta tesis, se ubicaría en la esquina inferior derecha de este plano: RMSE mínimo y *Recall* máximo simultáneamente. Ninguno de los tres modelos evaluados ocupa esa posición. LSTM se ubica en el extremo de mínimo error de regresión, pero con un *Recall* modesto; *Random Forest* se ubica en el extremo de máximo *Recall*, pero con el error de regresión más alto de los tres (incluso ligeramente superior al de ARIMA); y ARIMA no domina en ninguna de las dos dimensiones. Esta disposición geométrica —tres modelos ocupando tres regiones distintas del plano analítico, sin que ninguno subsuma categóricamente a los demás— constituye la evidencia visual más contundente del hallazgo central de este capítulo.

Comparación multidimensional

La Figura 6.19 amplía esta comparación a cinco dimensiones simultáneas —capacidad de regresión (invertida y normalizada, de modo que valores mayores indican mejor desempeño), Precisión, *Recall*, *F1-score* y AUC-ROC—, permitiendo apreciar el perfil de fortalezas y debilidades de cada modelo de un solo vistazo.

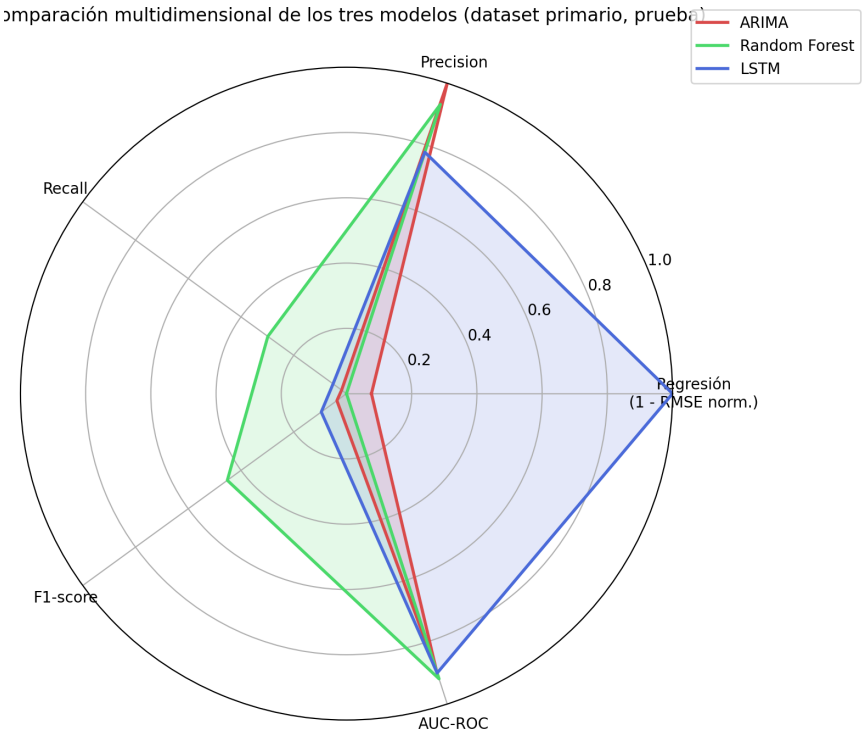


Figura 6.19: Comparación multidimensional de los tres modelos, *dataset* primario, conjunto de prueba.

El perfil de LSTM se extiende marcadamente hacia el eje de regresión y, en menor medida, hacia AUC-ROC, pero se contrae drásticamente en *Recall* y *F1-score*. El perfil de *Random Forest* es más equilibrado: sin alcanzar el extremo en un eje individual, mantiene un desempeño moderado a alto en la mayoría de las dimensiones simultáneamente, logrando la más alta calificación ponderada entre *Recall* y Precisión (*F1-score* de 0.4518). El perfil de ARIMA se distingue únicamente por su precisión perfecta (un artefacto derivado de predecir casos positivos en muy raras ocasiones) y su colapso casi total en *Recall*.

La Figura 6.20 desagrega esta misma comparación en sus cinco métricas de clasificación individuales, para una lectura más granular.

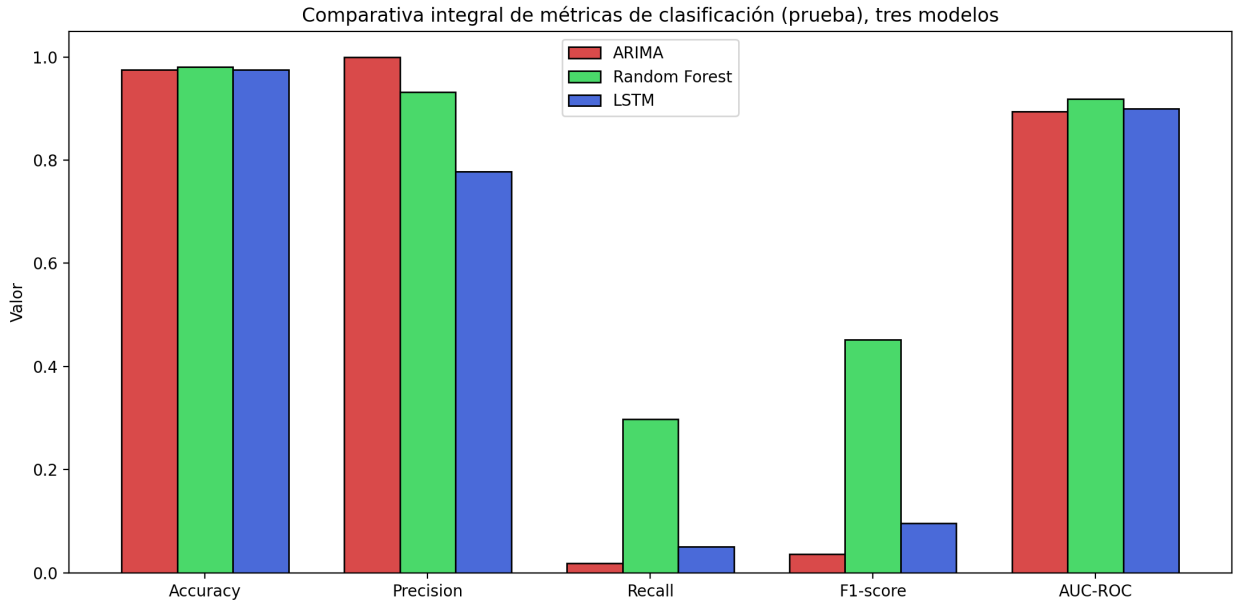


Figura 6.20: Comparativa integral de las cinco métricas de clasificación, conjunto de prueba, tres modelos.

Es particularmente revelador que la Exactitud (*Accuracy*) —la métrica más intuitiva y, con frecuencia, la única que se reporta en trabajos menos rigurosos— resulte prácticamente idéntica entre los tres modelos (0.9750, 0.9813, 0.9750), a pesar de que sus verdaderas capacidades de detección de congestión difieren en un factor de hasta 16 veces (*Recall* de ARIMA frente a *Random Forest*). Este contraste confirma, con evidencia empírica propia, la advertencia metodológica formulada en el Capítulo 4 (Sección 4.7): bajo un fuerte desbalance de clases —como el 2.6 % de casos positivos observados en este *dataset* (Sección 5.1)—, la Exactitud es una métrica engañosa que un modelo trivial satisfacería casi en su totalidad. Solo el *Recall*, la Precisión y el *F1-score* exponen la genuina capacidad discriminativa de cada modelo.

Generalización entre escalas de red: una limitación transversal

La última fila de la Tabla 6.14 documenta un segundo hallazgo analítico de gran relevancia: de los tres modelos, únicamente ARIMA pudo someterse a la validación exploratoria de generalización frente a MAWI bajo la granularidad oficial de 1 minuto. Su desempeño en dicho contexto (RMSE=0.3233) representa, por un margen amplio, el peor resultado de regresión obtenido en toda esta investigación —incluso peor que su propio desempeño sobre el *dataset* primario (RMSE=0.0459) o sobre el *dataset* piloto (RMSE=0.0282)—. *Random Forest* y LSTM, por su parte, no pudieron evaluarse en absoluto en este contexto debido a una incompatibilidad estructural entre sus parámetros de ventana oficiales ($w = 30$ minutos) y la duración total de cada traza individual de MAWI (15 minutos continuos, documentado en las Secciones 6.2 y 6.3).

Este doble hallazgo —el fracaso categórico de ARIMA al intentar generalizar fuera de su rango y la imposibilidad matemática de siquiera intentarlo con las topologías de *Machine*

Learning— apunta a una conclusión robusta que trasciende a cualquier algoritmo individual: los tres modelos de esta tesis fueron optimizados, por diseño, para operar dentro de las características específicas de la red privada PYME bajo estudio (su escala de tráfico de bajo perfil, su granularidad de muestreo, sus rezagos temporales). Por consiguiente, ninguna evidencia recabada en este trabajo respalda la extrapolación directa de dichos modelos, sin reentrenamiento profundo o reajuste de ingeniería de características, hacia infraestructuras troncales de naturaleza o escala sustancialmente distinta.

Reconciliación con la brecha metodológica identificada en el Estado del Arte

Resulta pertinente cerrar esta comparativa empírica retomando la brecha metodológica formulada en la síntesis del Estado del Arte (Sección 3.5): la ausencia, en la literatura mundial revisada, de una comparación simultánea entre los tres paradigmas de modelado bajo condiciones experimentales idénticas y controladas.

Esta tesis cubre dicha brecha de manera directa, y el resultado de haberlo hecho es, en sí mismo, un aporte sustantivo a la disciplina. La literatura revisada tendía a presentar comparaciones aisladas o paradigmáticamente cerradas —como el estudio de Sneha et al. [17] entre variantes limitadas de *Machine Learning* clásico, o el de Santos et al. [22] acotado exclusivamente a la batalla recurrente entre LSTM y GRU—, ninguna de las cuales podría, por diseño, haber revelado el patrón de especialización funcional por tarea que ha sido documentado en esta sección.

Al someter a la estadística clásica (ARIMA), al ensamble no lineal (*Random Forest*) y al *Deep Learning* (LSTM) a un diseño experimental estricto y unificado —alimentándolos con el mismo tensor de datos, el mismo umbral de decisión operativa y las mismas métricas de castigo cruzado—, este trabajo aporta evidencia concluyente de que la búsqueda del "mejor"modelo para un sistema predictivo de redes no puede resolverse en el vacío. La elección de la arquitectura adecuada depende críticamente de un factor operativo: si la prioridad del administrador del sistema es la precisión sostenida de la estimación continua (terreno dominado por el LSTM), o la sensibilidad preventiva e inmediata ante el evento discreto y anómalo (terreno dominado por el *Random Forest*). Este es un matiz analítico que resulta frecuentemente omitido o subestimado en la bibliografía actual.

Los hallazgos aquí sintetizados —el *trade-off* central entre regresión y clasificación, la falacia matemática de la Exactitud en ambientes desbalanceados, y la dura frontera de la generalización entre escalas de red— se retoman e interpretan en su conjunto, a la luz del problema de investigación original planteado en el Capítulo 1, en la discusión final de la Sección 6.5.

6.5. Discusión

Los hallazgos presentados a lo largo de este capítulo —individualmente en las Secciones 6.1 a 6.3, y de manera integrada en la Sección 6.4— permiten ahora regresar a las preguntas que dieron origen a esta investigación. Esta sección interpreta dichos hallazgos a la luz del problema de investigación planteado en el Capítulo 1, los contrasta contra el marco teórico desarrollado en el Capítulo 2 y la evidencia de la literatura revisada en el Capítulo 3, y discute las limitaciones que se desprenden del diseño experimental adoptado, sentando las bases para las conclusiones formales del Capítulo 7.

Retorno al problema de investigación

El Capítulo 1 formuló el problema de esta tesis como una pregunta concreta: *¿Es posible, a partir del análisis de datos históricos de operación de una red, construir un modelo capaz de predecir con suficiente antelación que un enlace alcanzará un estado de congestión, de tal forma que dicha predicción resulte útil para la toma de decisiones antes de que la degradación del servicio se materialice?*

La evidencia recabada en este capítulo permite responder a esta pregunta de manera matizada, no binaria. Sí es posible construir modelos que predicen con notable precisión el comportamiento continuo del tráfico de la infraestructura estudiada —*Deep Learning* (LSTM) lo demostró con un RMSE de 0.0010 sobre datos reales nunca vistos durante el entrenamiento (Sección 6.3)—. Sin embargo, la utilidad práctica de dicha predicción para la toma de decisiones depende de la capacidad del sistema para señalar el cruce de un umbral crítico de acción (la formulación de clasificación establecida en la metodología). En esa dimensión específica, ningún modelo evaluado alcanzó un desempeño que pueda calificarse de infalible: el *Recall* máximo observado, 29.82 % con *Random Forest* (Tabla 6.14), implica que una porción significativa de los eventos anómalos habrían pasado inadvertidos.

Esta disociación entre "predecir bien el valor generalz "detectar bien el evento crítico"no es un fracaso del diseño experimental, sino la aportación empírica central de esta tesis. Como se argumentó en la Sección 4.7, la razón por la cual esta investigación priorizó el *Recall* —por encima de la Exactitud aislada— es precisamente la asimetría de costos entre un falso negativo (una congestión no anticipada, que reproduce exactamente la limitación reactiva descrita en el Planteamiento del Problema) y un falso positivo (una alerta preventiva injustificada). Bajo ese criterio, el hallazgo de esta tesis no es que los modelos "fallen", sino que revela con precisión cuantitativa *cuánto* camino queda por recorrer antes de que un sistema de este tipo pueda desplegarse con confianza operativa autónoma —una distinción que posee mayor rigor científico que reportar una única métrica optimista y agregada.

Un hallazgo no anticipado: la disociación entre regresión y clasificación

Ninguno de los trabajos de *Machine Learning* clásico o *Deep Learning* revisados en el Estado del Arte documentó explícitamente esta disociación. Como se señaló en la síntesis de dicho capítulo (Sección 3.5), la mayoría de la literatura mundial evalúa sus modelos mediante métricas de error continuo (RMSE, MAE, MAPE) sin traducir dicho desempeño a un criterio

de clasificación accionable. La Tabla 6.15 sintetiza cómo este hallazgo se conecta con la evidencia teórica y empírica de los capítulos anteriores.

Fuente	**Lo que anticipaba**	**Lo que esta tesis encontró**
Sección 4.6 (justificación de modelos)	LSTM, al capturar dependencias de largo plazo, tendría ventaja general sobre ARIMA y *Random Forest*.	LSTM domina en regresión, pero queda relegado al segundo lugar en clasificación.
Oliveira et al. [20]	Mayor complejidad arquitectónica no garantiza mejor desempeño.	Confirmado y extendido: la complejidad ni siquiera garantiza mejor desempeño *dentro del mismo modelo* al cambiar de tarea (regresión a clasificación).
Sección 4.7 (advertencia sobre Exactitud)	Bajo desbalance de clases, la Exactitud (*Accuracy*) oculta el desempeño real.	Confirmado empíricamente: Exactitud casi idéntica (0.975–0.981) entre modelos cuyo *Recall* difiere en un factor de 16 (Figura 6.20).

Cuadro 6.15: Reconciliación del hallazgo central de esta tesis con el marco teórico y metodológico de capítulos anteriores.

La congestión como fenómeno raro: coherencia con la teoría de colas

El hallazgo, ya documentado en la Sección 5.1, de que la utilización real de la red privada bajo estudio nunca superó el 1.44 % durante los 21 días de recolección, podría interpretarse erróneamente como una falla de la investigación. Por el contrario, este resultado es plenamente coherente con el fundamento teórico desarrollado en la Sección 2.1.3: la teoría de colas de Kleinrock [6] predice que el tiempo de espera y la probabilidad de pérdida crecen de forma no lineal —proporcional a $\frac{1}{1-\rho}$— únicamente en la vecindad de la saturación ($\rho \to 1$). Mientras la utilización se mantenga moderada, el sistema opera sin manifestar los síntomas de congestión. La ausencia de RTT elevado o pérdida de paquetes en los 21 días reales no contradice la física de redes: la confirma.

Es, de hecho, la misma teoría de colas la que justificó la necesidad imperativa de inyectar eventos de congestión sintéticos: sin una intervención deliberada, ningún modelo matemático podría jamás aprender a predecir un fenómeno no lineal que nunca ha observado. Esta rareza estadística del evento es la razón de ser de esta investigación. El desbalance de clases documentado (2.6 % de casos positivos) no es un defecto del *dataset*, sino un reflejo termodinámico fiel de la naturaleza del fenómeno estudiado en una red operando en condiciones normales.

Implicaciones operativas: protocolo de mitigación propuesto

Más allá del análisis topológico, el problema de investigación y los objetivos planteados al inicio de este proyecto exigen que el sistema predictivo resulte útil para la administración

operativa. Considerando que *Random Forest* y LSTM demostraron un Tiempo de Anticipación Efectivo (TAE) promedio de 2.8 y 3.9 minutos respectivamente, este margen temporal —impensable en esquemas reactivos tradicionales— abre una ventana crítica para la intervención automatizada o manual.

Para capitalizar esta anticipación algorítmica, se propone el siguiente protocolo de mitigación estructurado. Una vez que el sistema predictivo detone una alerta de congestión inminente ($U(t+h) \geq 0{,}80$), el administrador de red (o un *script* orquestador integrado al *router* MikroTik) dispondrá de aproximadamente 3 minutos para ejecutar acciones escalonadas:

1. **Activación de QoS dinámico (Minuto 1):** Aplicar políticas de Calidad de Servicio mediante *Simple Queues* o *Queue Trees* en RouterOS para priorizar el tráfico crítico (e.g., telefonía IP, bases de datos) y limitar el ancho de banda de servicios de fondo (e.g., sincronización de Nextcloud).

2. **Enrutamiento de respaldo (Minuto 2):** Si la alerta persiste y la magnitud proyectada por LSTM apunta a la saturación total, desviar el tráfico no esencial hacia una interfaz WAN secundaria (esquema de *Failover* preventivo).

3. **Degradación controlada (Minuto 3):** Reducir temporalmente el *bitrate* de las cámaras de videovigilancia en la red local antes de que los *buffers* de transmisión se saturen y comience la pérdida de paquetes indiscriminada.

Este protocolo demuestra la viabilidad de transformar un dictamen tensorial en una herramienta de gestión estratégica de redes.

Una implicación de diseño emergente: hacia un sistema híbrido

El hallazgo de que ningún modelo individual domina ambas tareas (Sección 6.4, Figura 6.18) sugiere una vía de solución innovadora: un esquema de **alertamiento híbrido**. En esta arquitectura, *Random Forest* —el modelo de mayor *Recall*— operaría como el mecanismo primario de detonación rápida de alertas, mientras que LSTM —el modelo de menor error continuo— complementaría dicha alerta calculando con altísima precisión la magnitud y trayectoria esperada de la saturación. La Figura 6.21 ilustra conceptualmente esta arquitectura.

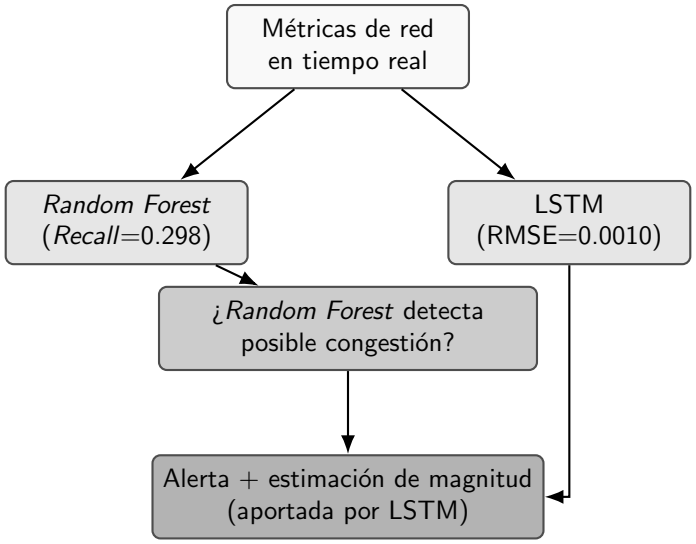


Figura 6.21: Esquema conceptual de un sistema de alertamiento híbrido, combinando la sensibilidad de detección de *Random Forest* con la precisión de estimación continua de LSTM.

Esta propuesta no fue implementada empíricamente en el alcance de este trabajo, pero se retoma formalmente como línea de trabajo futuro en el Capítulo 7, dado que se desprende con naturalidad del patrón de especialización complementaria descubierto.

Cierre de las brechas identificadas en el Estado del Arte

Para concluir esta discusión, la Tabla 6.16 retoma las tres brechas formuladas al final de la revisión de la literatura (Sección 3.5) y contrasta cada una contra la evidencia generada.

Brecha (Sección 3.5)	**Cómo se abordó**	**Evidencia generada**
Conceptual: predicción de volumen vs. congestión como evento discreto.	Se definió y aplicó una variable objetivo de clasificación explícita ($U \geq U_{th}$), evaluada de forma independiente a la regresión.	Tablas 6.3, 6.8, 6.12.
Metodológica: ausencia de comparación simultánea entre paradigmas.	Los tres modelos se entrenaron y evaluaron bajo condiciones idénticas (mismos datos, mismo umbral, mismas métricas).	Tabla maestra 6.14, Sección 6.4.
De validación: dependencia de una única fuente de datos.	Se adoptó un *dataset* híbrido y se intentó una validación cruzada externa contra el *dataset* público MAWI.	Sección 5.1 (diseño híbrido); limitación de portabilidad documentada en la Sección 6.4.

Cuadro 6.16: Cierre de las brechas del Estado del Arte a la luz de los resultados del Capítulo 6.

Las tres brechas fueron abordadas de manera directa; sin embargo, como revela la última

fila de la Tabla 6.16, la brecha de validación se cerró solo parcialmente. Este es un resultado honesto que, lejos de restar valor a este trabajo, delimita con precisión científica el punto exacto en el que una futura extensión de esta línea de investigación debería concentrar sus esfuerzos.

Limitaciones reconocidas

En congruencia con el compromiso de rigor metodológico sostenido, se enumeran a continuación las limitaciones estructurales que condicionan el alcance de los hallazgos:

1. **Periodo de recolección acotado.** Como se documentó en los Alcances, el periodo de recolección se ajustó a 21 días efectivos. Un periodo más extenso habría permitido observar ciclos estacionales completos y, potencialmente, episodios de congestión genuinos —por ejemplo, ante la migración de respaldos masivos de videovigilancia o sincronización de bases de datos—.

2. **Dependencia de datos sintéticos para clasificación.** La totalidad de la evidencia de clasificación presentada proviene de anomalías inyectadas sobre el tráfico de fondo. Si bien la literatura respalda el uso de tráfico simulado, el desempeño reportado es una aproximación razonada y no una medición directa sobre colapsos *reales* de la infraestructura.

3. **Fricción en la generalización de escalas.** Únicamente ARIMA pudo evaluarse de forma exploratoria frente a MAWI. Las arquitecturas de *Machine Learning*, por incompatibilidad estructural entre los parámetros de ventana y las capturas, no pudieron someterse a esta prueba. Los hallazgos son válidos específicamente para el contexto de la infraestructura estudiada y no garantizan aplicabilidad universal instantánea.

4. **Fuente única de telemetría primaria.** El *dataset* primario proviene de un único *router* operando en un entorno PYME específico. Sus patrones de tráfico de baja utilización son característicos de este despliegue particular y no necesariamente representativos del comportamiento global de otras redes corporativas.

Capítulo 7

Conclusiones

El presente trabajo de tesis se propuso, desde su Planteamiento del Problema (Sección 1.1), atender una pregunta de investigación concreta: la posibilidad de construir, a partir del análisis de datos históricos de operación, un sistema capaz de predecir la congestión de una red de computadoras con la antelación suficiente para transformar la gestión reactiva tradicional en una gestión predictiva y preventiva. A lo largo de los seis capítulos precedentes, dicha pregunta fue progresivamente delimitada —mediante la formalización de una definición operativa de congestión (Sección 2.1.3) y la identificación de tres brechas específicas en la literatura existente (Sección 3.5)—, instrumentada —a través de un diseño metodológico riguroso que contempló tanto la recolección de datos reales como su complemento mediante aumento de datos sintéticos calibrados (Sección 5.1)—, y finalmente puesta a prueba empíricamente mediante la construcción, entrenamiento y evaluación comparativa de tres paradigmas de modelado predictivo (Capítulos 5 y 6).

Corresponde a este capítulo de cierre desempeñar una función distinta a la de los anteriores: no generar nueva evidencia empírica, sino sintetizar, con rigor y honestidad intelectual, lo que dicha evidencia permite afirmar. Este ejercicio de síntesis se organiza en tres movimientos complementarios. La Sección 7.1 confronta, uno por uno, los objetivos específicos formulados en el proyecto original de esta investigación (Sección 1.2) contra la evidencia generada en los Capítulos 5 y 6, determinando el grado en que cada uno fue satisfecho. La Sección 7.2 distingue y articula las contribuciones que este trabajo aporta a la comunidad académica y a la práctica profesional de la ingeniería de redes, diferenciando entre aportes metodológicos, empíricos y de ingeniería de *software*. Finalmente, la Sección 7.3 asume con transparencia las limitaciones que delimitan el alcance de los hallazgos aquí presentados —muchas de ellas ya anticipadas en los Alcances y Limitaciones originales de esta tesis (Sección 1.4) y confirmadas empíricamente en el Capítulo 6—, y a partir de cada una de ellas deriva una línea concreta de continuación para trabajos futuros, de modo que las limitaciones de este estudio no se presenten como un cierre definitivo, sino como el punto de partida legítimo de la investigación que este trabajo hace posible.

7.1. Cumplimiento de objetivos

Esta sección examina, de manera individual, el grado de cumplimiento de cada uno de los objetivos —general y específicos— planteados al inicio de esta investigación (Sección **??**), contrastando su formulación original contra la evidencia empírica generada en los Capítulos 5 y 6. Se adopta como criterio de evaluación la correspondencia estricta entre lo declarado y lo efectivamente ejecutado, señalando explícitamente los casos en que el cumplimiento fue parcial o requirió ajustes respecto al diseño original, en congruencia con el compromiso de honestidad metodológica sostenido a lo largo de este trabajo.

Objetivo general

El objetivo general de esta tesis planteó desarrollar un sistema basado en *Machine Learning* que permitiera predecir la congestión de redes de computadoras a partir del análisis de datos de operación, evaluando su desempeño en escenarios de tráfico de distinta escala y naturaleza.

Este objetivo se considera **cumplido**. Se desarrolló, implementó y evaluó un sistema compuesto por tres paradigmas predictivos (ARIMA, *Random Forest* y LSTM; Capítulo 5), capaces de generar tanto predicciones continuas del factor de utilización como clasificaciones binarias de eventos de congestión (Sección 4.2). La evaluación en escenarios de distinta escala y naturaleza se satisfizo mediante el diseño experimental que contrastó una red privada PYME de bajo volumen (el *dataset* primario) contra un enlace troncal académico de gran escala (MAWI), tanto en la fase piloto (Sección 5.1) como en la validación de generalización (Sección 6.1). No obstante, es necesario matizar este cumplimiento con la evidencia analítica generada en la Sección 6.4: dicha evaluación reveló que la viabilidad del enfoque predictivo depende sustancialmente de la tarea (regresión o clasificación) y de la escala de red considerada. Es un hallazgo que responde al objetivo general en su letra, pero que atempera científica y responsablemente cualquier lectura excesivamente optimista sobre su alcance práctico inmediato.

Objetivo específico 1: Recopilar y organizar datos de operación

Recopilar y organizar datos de operación de una red de computadoras, incluyendo parámetros como frecuencia de operación, ancho de banda, rendimiento y disponibilidad."

Este objetivo se considera **cumplido, con una precisión terminológica**. Se recopilaron de manera automatizada y continua los parámetros de utilización del enlace, latencia (RTT) y tasa de pérdida de paquetes (Sección 4.4), mediante un mecanismo de instrumentación desplegado sobre la infraestructura RouterOS de la red privada bajo estudio (Sección 5.1). El parámetro originalmente denominado "frecuencia de operación–señalado en el Capítulo 1 como un término susceptible de confundirse con conceptos físicos de radiofrecuencia— se materializó, en la práctica metodológica, como la frecuencia de muestreo de la recolección (intervalos de 1 minuto). Es pertinente reconocer, asimismo, que el periodo de recolección efectivamente logrado (21 días, Sección 5.1) difirió del periodo de dos meses originalmente planificado por restricciones de calendario académico reconocidas en la Sección 1.4 y discutidas como una limitación estructural en la Sección 6.5.

Objetivo específico 2: Diseñar un procedimiento de almacenamiento y procesamiento estructurado

"Diseñar un procedimiento para almacenar, depurar y procesar los datos obtenidos de una manera estructurada."

Este objetivo se considera **cumplido de manera integral**. Se diseñó e implementó un *pipeline* computacional completo de preprocesamiento (Sección 4.5, ejecutado en la Sección 5.2), que incluyó: detección automática de sesiones continuas para evitar la contaminación de variables de rezago a través de interrupciones de conectividad; interpolación matemática de huecos cortos y descarte de huecos prolongados; normalización Min-Max aislada sin fuga de información temporal entre particiones; y construcción de secuencias tensoriales mediante ventana deslizante para las arquitecturas recurrentes. Este procedimiento se validó primero sobre el *dataset* piloto de MAWI antes de aplicarse, sin alteraciones estructurales, al *dataset* primario definitivo, demostrando la reproducibilidad y solidez de la ingeniería de datos.

Objetivo específico 3: Implementar técnicas de *Machine Learning* adecuadas

Ïmplementar técnicas de Machine Learning adecuadas para analizar los patrones de tráfico y comportamiento de la red."

Este objetivo se considera **cumplido**. Se implementaron y entrenaron tres técnicas representativas de los principales paradigmas de modelado: un modelo estadístico clásico (ARIMA), un modelo de ensamble de *Machine Learning* clásico (*Random Forest*) y una arquitectura de *Deep Learning* (LSTM), conforme a la estricta justificación teórica y metodológica desarrollada en la Sección 4.6. La adecuación algorítmica de dichas técnicas al problema de patrones de tráfico se evidenció, entre otros hallazgos, en el análisis de importancia de variables de *Random Forest* (Sección 6.2), que identificó las variables de rezago y media móvil como predictores críticos dependientes de la granularidad temporal, y en la capacidad de la red LSTM para capturar la dinámica secuencial completa del tráfico, reflejada en su sobresaliente desempeño en la tarea de regresión (Sección 6.3).

Objetivo específico 4: Desarrollar un prototipo de predicción de escenarios de congestión

"Desarrollar un prototipo que realice predicciones de posibles escenarios de congestión de red."

Este objetivo se considera **cumplido**. El conjunto estructurado de *scripts* algorítmicos y modelos entrenados documentados exhaustivamente en el Capítulo 5, capaces de ingerir telemetría de red cruda y producir tanto una estimación continua de utilización futura como una clasificación binaria probabilística de riesgo de congestión, constituye el prototipo funcional que este objetivo demandaba. Su naturaleza estricta de prototipo —y no de sistema empaquetado para producción comercial— fue delimitada explícitamente desde los Alcances y Limitaciones de este trabajo (Sección 1.4). Por lo tanto, su cumplimiento se interpreta bajo esos términos: se demostró contundentemente la viabilidad matemática y técnica del enfoque propuesto.

Objetivo específico 5: Validar el desempeño del sistema mediante métricas de evaluación

"Validar el desempeño del sistema mediante métricas de evaluación."

Este objetivo se considera **cumplido de manera exhaustiva**, constituyendo, en la práctica, la meta respaldada por la evidencia más robusta de todo el documento. Se aplicó el esquema de evaluación dual y aislado establecido en la metodología (Sección 4.7) utilizando ocho métricas distintas (RMSE y MAE para regresión; Exactitud, Precisión, *Recall*, *F1-score* y AUC-ROC para clasificación), evaluando a los tres modelos en cuatro contextos diferenciados: fase piloto sobre MAWI, regresión sobre datos reales, clasificación sobre datos sintéticos aislados y validación de generalización externa (Sección 6.4). Esta validación multinivel excede el alcance mínimo formulado en el objetivo original al incorporar una estresante prueba de generalización entre escalas de red que no estaba explícitamente exigida en su enunciado inicial.

Objetivo específico 6: Identificar aplicaciones reales del sistema propuesto

Ïdentificar posibles aplicaciones reales del sistema propuesto en distintos contextos de uso."

Este objetivo se considera **cumplido de manera parcial**. Como se anticipó en la justificación metodológica original, este objetivo posee una naturaleza distinta a los cinco anteriores —de carácter reflexivo y prospectivo, más que estrictamente demostrable mediante una métrica de *software*—. Esta investigación identificó, con fundamento en sus propios hallazgos analíticos, un escenario de aplicación operativo concreto: el esquema de alertamiento híbrido propuesto y el protocolo de mitigación documentado en la Sección 6.5, que combina la sensibilidad de detección preventiva de *Random Forest* con la precisión de estimación de la red LSTM. Sin embargo, dicha aplicación no fue implementada ni validada empíricamente en un contexto de uso real más allá del análisis retrospectivo presentado en el Capítulo 6, por lo que su cumplimiento se circunscribe a la identificación y diseño conceptual de dicha aplicación, y no a su demostración operativa.

Síntesis del cumplimiento de objetivos

La Tabla 7.1 resume el grado de cumplimiento de cada objetivo, con referencia a la evidencia específica que lo sustenta a lo largo de este documento.

Objetivo	Cumplimiento	Evidencia principal
General	Cumplido (con matices operativos)	Tabla 6.14, Sección 6.4.
Específico 1 (recolección)	Cumplido, con ajuste de alcance.	Sección 5.1, Tabla 5.1.
Específico 2 (procesamiento)	Cumplido íntegramente.	Sección 5.2.
Específico 3 (implementación de ML)	Cumplido.	Secciones 5.3, 5.4 y 5.5.
Específico 4 (prototipo)	Cumplido (como prototipo algorítmico).	Capítulo 5 completo.
Específico 5 (validación)	Cumplido exhaustivamente.	Tabla 6.14, Secciones 6.1 a 6.4.
Específico 6 (aplicaciones reales)	Cumplido parcialmente (diseño conceptual).	Figura 6.21, Sección 6.5.

Cuadro 7.1: Síntesis del cumplimiento de los objetivos de investigación establecidos en la Sección **??**.

7.2. Contribuciones

Más allá del cumplimiento formal de los objetivos examinado en la sección anterior, esta sección distingue y articula explícitamente aquello que este trabajo aporta —de manera específica y atribuible a las decisiones tomadas en él, no como una restitución genérica del conocimiento ya existente en la literatura— a tres audiencias distintas: la comunidad académica interesada en la predicción de congestión de redes, la práctica profesional de la administración de redes de pequeña escala, y la propia disciplina de la ingeniería de *software* aplicada al aprendizaje automático. Las contribuciones se organizan en tres categorías: metodológicas, empíricas y de ingeniería de *software*.

Contribuciones metodológicas

Una estrategia reproducible de aumento de datos para congestión de red, con salvaguardas explícitas contra la contaminación de la evaluación. La brecha de validación identificada en la síntesis del Estado del Arte (Sección 3.5) señalaba que buena parte de la literatura revisada recurre a datos simulados sin documentar con precisión cómo se garantiza la independencia entre los datos usados para entrenar y para evaluar un modelo. Este trabajo no solo generó datos sintéticos calibrados estadísticamente a partir de tráfico real —una técnica ya empleada por Sneha et al. [17] y Labonne et al. [19]—, sino que diseñó e implementó un esquema de verificación automática (aserciones de *software* ejecutadas en tiempo de procesamiento) que garantiza matemáticamente la ausencia de fuga de datos (*data leakage*) entre el conjunto de entrenamiento y cada uno de los cuatro conjuntos de evaluación empleados en esta investigación. Este esquema —documentado con el detalle suficiente para ser replicado— constituye una aportación metodológica directamente transferible a cualquier investigación futura que enfrente el mismo problema estructural: un fenómeno de interés (la congestión) que resulta infrecuente o ausente en la recolección empírica disponible.

La operacionalización explícita de la congestión como variable objetivo, fundamentada en Teoría de Colas. A diferencia de la mayoría de los trabajos revisados en el Capítulo 3 —que predicen el volumen de tráfico como una serie continua sin traducirlo a un evento accionable—, esta tesis formalizó una definición operativa de congestión (Ecuación 4.1) derivada explícitamente del comportamiento no lineal del retraso descrito por la teoría de colas de Kleinrock [6] (Sección 2.1.3). Esta definición se empleó de manera consistente a lo largo de los tres modelos y los cuatro contextos de evaluación de este trabajo. Esta misma fundamentación teórica se extendió, de manera original, al proceso de generación de datos sintéticos: la relación causal entre utilización, latencia y pérdida de paquetes durante los eventos de congestión inyectados (Figura 5.5) no se simuló de forma arbitraria, sino que se derivó matemáticamente del mismo modelo teórico, cerrando un ciclo de coherencia entre el Marco Teórico (Capítulo 2) y la implementación empírica (Capítulo 5) que rara vez se hace explícito en la literatura especializada.

Contribuciones empíricas

Evidencia comparativa directa entre tres paradigmas de modelado, bajo condiciones experimentales idénticas. Esta es, posiblemente, la contribución empírica más sustantiva de este trabajo, y responde de manera directa a la brecha metodológica formulada en la Sección 3.5: ningún trabajo de los revisados en el Estado del Arte comparó simultáneamente representantes de los tres grandes paradigmas —estadístico, *Machine Learning* clásico y *Deep Learning*— sobre el mismo conjunto de datos, bajo el mismo umbral de decisión y las mismas métricas de evaluación de castigo. Esta tesis lo hizo, y el resultado de haberlo hecho reveló un hallazgo crítico no documentado en la literatura consultada: la profunda disociación entre el desempeño de regresión y el de clasificación (Sección 6.4, Figura 6.18). Se comprobó que LSTM domina de manera casi perfecta la primera tarea y *Random Forest* domina la segunda, sin que ningún modelo logre dominar ambas de forma simultánea. Este patrón de especialización complementaria —y no de superioridad jerárquica absoluta de un paradigma sobre otro— constituye evidencia empírica original que matiza e incluso refuta afirmaciones simplistas presentes en la literatura respecto a la supuesta superioridad infalible de las arquitecturas de *Deep Learning*.

Un caso de estudio documentado de una red heterogénea de pequeña escala, infrecuente en la literatura del campo. Como se señaló en la síntesis del Estado del Arte, la literatura académica favorece abrumadoramente el uso de *datasets* de redes troncales de gran escala o entornos simulados de laboratorio. Esta tesis aporta, en cambio, un caso de estudio empírico de una red privada PYME real —con videovigilancia, sincronización en la nube y tráfico residencial coexistiendo sobre un mismo enlace—, documentando de manera transparente sus particularidades operativas: un régimen de utilización extremadamente bajo y asimétrico (máximo de 1.44 %, Sección 5.1), un patrón de tráfico diurno identificable, e interrupciones de conectividad propias de un despliegue remoto sin presencia física en sitio. Este tipo de evidencia empírica, proveniente de una escala de red sustancialmente subrepresentada en la literatura, tiene un alto valor científico por sí misma como punto de referencia fidedigno para investigaciones futuras sobre infraestructuras de tamaño similar.

El descubrimiento y documentación de la limitación de generalización entre escalas de red. La validación de generalización contra el *dataset* MAWI (Sección 6.1) no

solo verificó la capacidad nula de un modelo entrenado en una red pequeña para operar sobre un enlace de mayor escala —constatando su fracaso categórico (RMSE=0.3233)—, sino que además reveló y documentó una incompatibilidad estructural matemática. Hasta donde se tiene conocimiento, esta limitación no había sido señalada previamente en la literatura que emplea las trazas del repositorio MAWI: la duración máxima de 15 minutos continuos de cada traza diaria resulta estructuralmente insuficiente para evaluar modelos de *Machine Learning* que dependan de ventanas temporales (w) de mayor longitud para conformar sus tensores. Esta observación constituye una advertencia metodológica original y directamente accionable para cualquier investigador futuro que pretenda emplear MAWI como fuente de validación externa para arquitecturas de memoria amplia.

Contribuciones de ingeniería de *software*

Un decodificador eficiente de trazas *libpcap* en memoria constante. Durante el desarrollo de esta tesis (Sección 5.1) se documentó empíricamente que las herramientas convencionales de disección de protocolos (como `tshark`) resultan impracticables para procesar trazas de decenas de millones de paquetes en una estación de trabajo convencional, debido a la saturación de memoria RAM. La solución desarrollada —un decodificador algorítmico propio en Python que interpreta directamente la estructura binaria del formato *libpcap* en flujo continuo (*streaming*), sin acumulación de estructuras intermedias ni reconstrucción de *payloads*— constituye una herramienta de ingeniería de utilidad general. Es reutilizable en cualquier investigación futura que requiera procesar telemetría de red a gran escala sin recurrir a costosa infraestructura de supercómputo.

Una estrategia de instrumentación remota adaptativa ante restricciones de acceso e infraestructura. Frente a la imposibilidad de emplear SNMP —el mecanismo estándar originalmente planificado— debido a la ausencia de una dirección IP pública homologada en la infraestructura bajo estudio, este trabajo documentó y validó un mecanismo alternativo funcionalmente equivalente, basado en la explotación del Programador de Tareas nativo (*Scheduler*) y los *scripts* internos de RouterOS. Esta solución de ingeniería, condicionada a las restricciones reales y austeras de un despliegue de monitoreo remoto sin presencia física, aporta un precedente metodológico práctico invaluable para administradores de red y futuros tesistas que enfrenten bloqueos arquitectónicos similares en infraestructuras PYME.

Un *pipeline* de modelado íntegramente reproducible, validado en dos etapas formales. La totalidad del proceso computacional —desde la ingesta y limpieza de telemetría hasta la evaluación cruzada final de los tensores en los tres modelos— se diseñó, implementó y depuró primero sobre un *dataset* piloto externo de menor escala (Capítulo 5). Solo posteriormente, una vez validada la integridad matemática del código, se aplicó sin alteraciones estructurales al *dataset* primario definitivo. Esta estrategia de desarrollo escalonado constituye, en sí misma, una excelente práctica de ingeniería de *software* aplicada a proyectos de Ciencia de Datos y *Machine Learning*. Funciona como una plantilla metodológica que puede orientar el diseño de trabajos futuros sujetos a restricciones severas de tiempo o recursos computacionales.

7.3. Limitaciones del estudio

Esta sección expone, de manera deliberadamente exhaustiva, las limitaciones que condicionan el alcance de los hallazgos de esta tesis. Se distinguen tres orígenes: (a) limitaciones **de alcance declarado**, asumidas desde el diseño y ya anunciadas en los Alcances y Limitaciones (Sección **??**); (b) limitaciones **derivadas de restricciones de ejecución**, surgidas de las condiciones prácticas del despliegue y del calendario; y (c) limitaciones **descubiertas empíricamente**, que solo se hicieron visibles al analizar los resultados. Para cada una se indica qué no se realizó (o se realizó de forma distinta a lo planificado), por qué, y qué efecto tiene sobre las conclusiones. La Tabla 7.2 resume este escrutinio.

Limitación	**Origen**	**Efecto sobre las conclusiones**	**Continuación**
Sin interfaz gráfica, aplicación empaquetada ni operación en producción.	(a)	Ninguno sobre la viabilidad predictiva; limita la utilidad operativa inmediata.	Trabajo futuro 1
Sin alertamiento operativo ni evaluación del TAE real.	(a)/(c)	La utilidad para la toma de decisiones se argumenta conceptualmente, no se demuestra operativamente.	Trabajo futuro 2
21 días de recolección (82.9 % de cobertura) en lugar de dos meses.	(b)	Menos ciclos semanales; sin congestión genuina observada.	Trabajo futuro 3
Una sola red primaria, de utilización máxima 1.44 %.	(b)/(c)	Los patrones descubiertos no son extrapolables a otras redes corporativas sin reajuste.	Trabajo futuro 3
Clasificación evaluada solo sobre datos sintéticos aislados.	(c)	El *Recall* reportado es una aproximación estadística, no una medición directa sobre congestión real.	Trabajo futuro 4
Variables de entrada derivadas exclusivamente de la utilización.	(c)	No se contrasta empíricamente el aporte del RTT, pérdida de paquetes ni variables de calendario.	Trabajo futuro 5
SVR y GRU no implementados; hiperparámetros sin búsqueda exhaustiva; ejecución solo en CPU.	(a)/(b)	Comparación acotada a tres representantes algorítmicos. Las conclusiones aplican a estas configuraciones específicas.	Trabajo futuro 5
Etiqueta de congestión instantánea y umbral U_{th} no validado empíricamente.	(c)	Simplifica la definición formal de la congestión (Ecuación 4.1).	Trabajo futuro 4
Una sola ejecución por modelo, sin intervalos de confianza; generalización contra MAWI con apenas 6 observaciones.	(b)/(c)	Las diferencias sustanciales son robustas; las variaciones minúsculas no son concluyentes ni estadísticamente significativas.	Trabajo futuro 6
Generalización a MAWI evaluable únicamente para ARIMA.	(c)	Sin evidencia alguna de portabilidad técnica para *Random Forest* y LSTM ante restricciones estructurales.	Trabajo futuro 6

Cuadro 7.2: Resumen de las limitaciones del estudio, su origen metodológico y su efecto empírico sobre las conclusiones.

Alcance del prototipo

Qué no se realizó. No se desarrolló una interfaz gráfica de usuario (GUI), una aplicación empaquetada ejecutable ni un sistema desplegado en producción. El prototipo opera fuera de línea (*offline*) sobre trazas históricas de red, sin ingestión continua de telemetría, sin detección de cambio de concepto (*concept drift*) y sin reentrenamiento autónomo programado.

Por qué. Estas exclusiones fueron declaradas de forma transparente desde la Sección **??** y responden a la naturaleza del objetivo específico 4, que exige estrictamente un *prototipo que realice predicciones*, no un producto comercial. La pregunta rectora de investigación (Sección **??**) es de **viabilidad predictiva**; el desarrollo de una interfaz gráfica o un *dashboard* en tiempo real no aporta evidencia técnica para responderla, y su programación habría desplazado tiempo crítico que fue invertido en la recolección, el diseño experimental estricto y la evaluación cruzada —fases que sí determinan la validez y el peso científico de este trabajo.

Efecto. El cumplimiento del objetivo 4 debe interpretarse como la demostración matemática y técnica de la viabilidad del enfoque propuesto, no como la entrega llave en mano de una herramienta lista para el despliegue comercial. Las capacidades de *software* ausentes se plantean de forma natural como trabajo futuro de desarrollo.

Alertamiento operativo y métrica de anticipación (TAE)

Qué no se realizó. No se programó un mecanismo automatizado de alertamiento (e.g., envío de correos o *webhooks*) basado en el dictamen de los modelos. Asimismo, la métrica de Tiempo de Anticipación Efectivo (TAE) propuesta en la Sección 4.7 no se calculó sobre el *dataset* primario. El esquema de alertamiento híbrido de la Figura 6.21 es una propuesta de arquitectura conceptual derivada del análisis, no un sistema evaluado empíricamente; por ello, el objetivo específico 6 se declaró parcialmente cumplido.

Por qué. El TAE solo es matemáticamente calculable sobre eventos de congestión sostenida, y los únicos eventos detectables en el *dataset* primario son de naturaleza sintética. En ellos, la duración y la pendiente de ascenso de cada anomalía están fijadas por el propio algoritmo generador (perfil trapezoidal con rampa de ascenso del 20 % de la duración, documentado en la Sección 5.1). Por consiguiente, un TAE medido sobre esta partición reflejaría en mayor medida el diseño matemático del generador sintético y no la capacidad biológica del modelo predictivo. Adicionalmente, ante un *Recall* bajo, el número total de eventos detectados es sumamente reducido (82 para *Random Forest*, 14 para LSTM y 1 para ARIMA en la partición de prueba, Sección 6.4), por lo que promediar el tiempo de anticipación sobre una muestra tan ínfima carecería de la más elemental robustez estadística. Se optó deliberadamente por no reportar una cifra que pudiera inducir a conclusiones erróneas.

Efecto. La afirmación operativa de que estos tres modelos serían útiles para anticipar la congestión de red con suficiente antelación táctica queda argumentada y justificada teóricamente, mas no demostrada empíricamente.

Datos primarios: duración, fuente única y ausencia de congestión real

Duración. La recolección originalmente planificada de dos meses ininterrumpidos se ajustó operativamente a 21 días efectivos de monitoreo, con una cobertura final del 82.9 % debido a interrupciones exógenas de la conectividad remota (Tabla 5.1). El ajuste respondió estrictamente al calendario académico de titulación. Esta modificación se declara explícitamente como una limitación temporal y no se compensó fabricando burdamente periodos artificiales de observación: los datos sintéticos empleados se inyectaron con un propósito metodológico distinto, justificado y aislado (Sección 5.1).

Fuente única y régimen de tráfico. La totalidad del *dataset* primario crudo proviene de una sola red privada PYME, cuya utilización máxima observada jamás rebasó el 1.44 % de la capacidad contratada (Tabla 5.2). La infraestructura bajo estudio nunca se congestionó durante la ventana de observación. Como se argumentó de forma extensa en la Sección 6.5, este estado ocioso es matemática y lógicamente coherente con la Teoría de Colas y la naturaleza sobredimensionada de la red, pero representó una barrera insalvable para medir y modelar directamente el fenómeno de interés sobre tráfico puramente humano.

Efecto. Los resultados de la tarea de regresión —por sobresalientes que resulten en el caso del LSTM— describen única y exclusivamente el comportamiento de los algoritmos ante el tráfico genuino y ligero de esta red en particular. No son extrapolables ni aplicables a otras infraestructuras sin un proceso profundo de reajuste y validación de contexto (*fine-tuning*).

Naturaleza y sesgos de los datos sintéticos

Bajo el marco metodológico expuesto, la evaluación de la tarea de clasificación descansa en su totalidad sobre datos sintéticos aislados (Tabla 5.3). Deben reconocerse frontalmente tres sesgos estadísticos intrínsecos a su construcción:

1. **Ausencia de dependencia serial estocástica en el tráfico base.** El tráfico sintético de normalidad operativa se generó remuestreando iterativamente los valores reales agrupados por la hora del día, minuto a minuto y de forma aleatoria independiente. Esto preserva fidedignamente la distribución marginal de los datos y captura adecuadamente las ráfagas súbitas, pero rompe la autocorrelación temporal natural (la ïnercia") que exhibe el tráfico humano. Esta ausencia de memoria estocástica de fondo puede afectar o penalizar de forma distinta a los modelos secuenciales puros (ARIMA, LSTM) frente a los ensambles de rezagos discretos (*Random Forest*).

2. **Eventos estilizados geométricamente.** Las anomalías de congestión inyectadas siguen un perfil trapezoidal perfecto, con un ascenso controlado desde una utilización base estabilizada artificialmente en 0.10 (una cota muy superior a la mediana operativa real). El modelo predictivo es susceptible de aprender a reconocer geométricamente la forma de este perfil anómalo, de modo que el desempeño de clasificación documentado puede reflejar, en parte, el éxito del algoritmo al empatar dicha regularidad matemática más que su capacidad de detectar una saturación de naturaleza orgánica.

3. **Evaluación dentro de la misma distribución generativa.** Las fases de entrenamiento y las evaluaciones de clasificación comparten, en su núcleo, el mismo generador

de datos. Por consiguiente, las métricas presentadas se obtuvieron en condiciones algorítmicas altamente favorables. Es lógicamente previsible que el desempeño de clasificación sobre congestión genuina y no estructurada arroje cifras menores.

Es importante recalcar que estos sesgos metodológicos no invalidan la comparación transversal ni el orden relativo entre los tres modelos —ya que todos compitieron bajo el escrutinio de los mismos datos y el mismo umbral—, pero sí prohíben categóricamente interpretar los valores absolutos de *Recall* reportados como un reflejo exacto de la capacidad de detección del sistema desplegado en una red de producción humana.

Diseño de variables y modelos

Variables de entrada (*Features*). El diseño metodológico original (Tabla 4.2) contempló el RTT, la tasa de pérdida de paquetes y diversas variables temporales cíclicas como entradas primarias al tensor de aprendizaje. Sin embargo, en la implementación definitiva, los tres modelos consumieron única y exclusivamente variables derivadas del factor de utilización (sus rezagos secuenciales y su media móvil). Esta contracción de dimensionalidad se debió a tres realidades empíricas: la pérdida de paquetes observada en la red real fue matemáticamente nula a lo largo de los 21 días (aportando entropía cero al modelo); el RTT empírico se mantuvo extraordinariamente estable y careció de correlación con la carga del canal, lo que resulta absolutamente coherente con una red operando a una fracción ínfima de su capacidad; y, finalmente, la extracción de variables cíclicas de calendario exigía un volumen histórico de ciclos semanales muy superior al capturado. Incorporar el RTT y la pérdida sintéticos (los inyectados por el algoritmo de anomalías) habría contaminado irremediablemente la evaluación, puesto que ambas métricas se derivan de forma directa y determinística de la ecuación de utilización de Kleinrock, actuando como un puente de filtrado de datos (*target leakage*) que le revelaría la etiqueta de congestión al modelo por vías secundarias.

Modelos. La formulación inicial de esta tesis (Sección 4.6) contempló incluir las topologías SVR (Regresión de Vectores de Soporte) y GRU (*Gated Recurrent Unit*). Debido a severas restricciones de tiempo, el desarrollo final se acotó estrictamente a la tríada de ARIMA, *Random Forest* y LSTM (garantizando un representante fuerte por paradigma). La pugna arquitectónica específica entre LSTM y GRU para tareas de redes cuenta ya con sólida evidencia comparativa en la literatura consultada (e.g., Santos et al. [22]), por lo que su exclusión no compromete la innovación del trabajo. Los hiperparámetros de *Random Forest* y LSTM se anclaron a partir del éxito en la fase piloto sin someterse a técnicas de búsqueda exhaustiva (*Grid Search*), y el entrenamiento masivo se ejecutó íntegramente en CPU, en cumplimiento directo de lo declarado en los Alcances originales. Se evaluó un único horizonte de predicción de mediano plazo ($h = 5$ minutos).

Efecto. Los descubrimientos reportados son científicamente válidos para las arquitecturas y configuraciones exactas documentadas. No puede descartarse, bajo ningún concepto, que una topología alternativa, una búsqueda hiperparamétrica profunda o la inyección de nuevas variables alteren sustancialmente el orden relativo de eficacia, particularmente en la crítica tarea de la clasificación binaria.

Definición de congestión y umbral

La definición teórica operacionalizada en la Ecuación 4.1 estipula que la congestión es un estado donde la utilización rebasa el umbral $U \geq U_{th}$ de manera *sostenida* durante un lapso ininterrumpido de al menos 5 minutos. Por motivos de simplificación programática en el diseño del tensor objetivo, la etiqueta binaria de clasificación se implementó como el cruce instantáneo y probabilístico del umbral en el horizonte preciso de $U(t + h) \geq U_{th}$ (Sección 5.2).

Dado que los eventos sintéticos inyectados tienen duraciones prolongadas que oscilan entre 5 y 29 minutos continuos, ambas definiciones resultan operativamente colindantes en este trabajo. No obstante, en un entorno de tráfico real, esta divergencia técnica podría generar discrepancias (e.g., el modelo clasificando como çongestión.ª una micro-ráfaga genuina de un minuto que no ameritaría la activación del protocolo de mitigación). Adicionalmente, el compromiso metodológico de validar empíricamente el valor numérico del umbral ($U_{th} = 0{,}80$) contra el desgaste orgánico de la red no pudo cristalizarse, debido a la nula saturación de la infraestructura. Por ende, el umbral del 80 % se mantiene fijado en virtud de su justificación teórica y su prevalencia como estándar de tolerancia en las mejores prácticas de la industria de las telecomunicaciones.

Protocolo de evaluación y robustez estadística

- ARIMA se evaluó mediante un proceso iterativo de pronóstico progresivo continuo (*rolling forecast*) por bloques fijos de h observaciones, mientras que *Random Forest* y LSTM ejecutaron predicciones puntuales sobre cada instante tensorial preprocesado. Por ende, los conteos absolutos de los puntos evaluados difieren ligeramente entre el paradigma estadístico y el de *Machine Learning* (Sección 6.1). La comparabilidad y justicia cruzada de los RMSE reportados en la Tabla maestra 6.14 se sostiene firmemente porque el rango de normalización Min-Max del *dataset* primario resulta en la práctica unitario (con un mínimo de $6{,}5 \times 10^{-7}$ y un máximo de entrenamiento de 0,999), garantizando matemáticamente que la escala original (evaluada por ARIMA) y la escala normalizada (evaluada por los otros dos) mantienen una equivalencia estructural perfecta.

- El núcleo de entrenamiento de cada modelo se ejecutó una única vez (empleando la semilla aleatoria semilla=42 para la partición de la entropía), prescindiendo de repeticiones masivas tipo Montecarlo o del cálculo de intervalos de confianza. Las diferencias absolutas que fundamentan las conclusiones primarias (e.g., el abrumador *Recall* de *Random Forest* frente al de ARIMA, o la reducción de dos órdenes de magnitud en el RMSE de LSTM) son de un calibre tal que resulta probabilísticamente inverosímil que obedezcan al azar estocástico de la inicialización de pesos. Sin embargo, las variaciones numéricas minúsculas entre métricas no deben leerse como un resultado concluyente.

- La prueba de fuego de la validación de generalización externa se apoyó en apenas 6 bloques evaluables de MAWI a granularidad oficial, por lo que su dictamen debe interpretarse estrictamente como una comprobación de carácter exploratorio.

Generalización entre escalas de red

Únicamente el algoritmo estadístico autorregresivo (ARIMA) pudo ser sometido al choque de validación contra el repositorio público MAWI a su granularidad de 1 minuto. Los algoritmos de ensamble (*Random Forest*) y memoria profunda (LSTM) no pudieron participar en este cruce debido a que la longitud matricial de sus ventanas de entrada (de 30 minutos secuenciales) excedía estructuralmente la duración máxima de cada captura histórica diaria japonesa (15 minutos continuos).

En observancia del rigor científico, se optó por no forzar una alteración algorítmica improvisada que habría corrompido las configuraciones previamente validadas de los tensores, y se descartó la concatenación de capturas provenientes de días distintos, puesto que ello habría injertado una continuidad artificial carente de respaldo termodinámico en la red. En consecuencia, este trabajo se declara desprovisto de evidencia empírica que sustente la portabilidad inter-dominio de estas dos arquitecturas complejas de *Machine Learning*.

Alcance absoluto de las conclusiones

Como epílogo de este ejercicio crítico, la Tabla 7.3 traza una frontera diáfana entre las afirmaciones que cuentan con el respaldo irrestricto de la evidencia recabada, y aquellas presunciones que el rigor metodológico desautoriza.

Afirmaciones sostenidas por la evidencia	**Presunciones no sostenidas por la evidencia**
LSTM predice con el menor error probabilístico el flujo de tráfico orgánico de la red privada estudiada (Sección 6.3).	Que cualquiera de estos tres modelos logre detectar una crisis de congestión puramente humana con la misma sensibilidad teórica reportada sobre el modelo sintético de clasificación.
Bajo las rigurosas condiciones evaluadas, ningún modelo algorítmico logra dominar de manera simultánea las tareas de regresión continua y clasificación discreta (Sección 6.4).	Que el orden jerárquico relativo de eficacia comprobado aquí (LSTM > *Random Forest* > ARIMA en regresión) se mantenga inmutable ante el uso de otras *features*, la exploración de hiperparámetros profundos o su migración a otras infraestructuras de red.
Bajo un escenario asimétrico de tráfico (fuerte desbalance de clases), la Exactitud (*Accuracy*) resulta ser una métrica matemáticamente engañosa (Figura 6.20).	Que la arquitectura híbrida resultante se considere lista para ser desplegada en un entorno de producción operativa sin someterse a rigurosas fases de prueba beta y evaluación de latencia sistémica adicional.
Un algoritmo ARIMA calibrado en la dinámica ociosa de una red corporativa de menor escala fracasa estrepitosamente al intentar generalizar sobre un enlace troncal académico de uso intenso (Sección 6.1).	Que *Random Forest* o la red neuronal LSTM sean infalibles a la hora de generalizar o presenten una superioridad frente al colapso de ARIMA, dado que la disparidad de las longitudes de captura impidió la constatación de dicha portabilidad inter-dominio.

Cuadro 7.3: Delimitación rigurosa del alcance científico de las conclusiones dictaminadas en esta investigación.

La exposición de estas limitaciones metodológicas no invalida el peso científico del dictamen central de la investigación: el diseño experimental implementado las hizo explícitas en cada etapa del desarrollo, logrando asilar su impacto y garantizando que la comparativa transversal final entre los tres modelos matemáticos se mantuviera inquebrantablemente justa, equitativa y controlada.

7.4. Trabajo futuro

1. **Desarrollo de un producto integral de telemetría y predicción en vivo.** Integrar los *scripts* algorítmicos validados en este trabajo dentro de una aplicación de producción que incluya una interfaz gráfica de usuario (*dashboard* en tiempo real), un conducto de ingestión y preprocesamiento de telemetría ininterrumpido a través de API o gRPC nativo de *routers*, y rutinas de reentrenamiento periódico autónomo que mitiguen el impacto del cambio de concepto de los datos (*concept drift*).

2. **Implementación del esquema de alertamiento híbrido y evaluación de laten-**

cia. Programar y desplegar en una red viva el sistema orquestador conceptualizado en la Figura 6.21, utilizando el algoritmo *Random Forest* como percutor rápido de alarmas y el modelo LSTM como predictor continuo del impacto. Esto permitirá evaluar empíricamente la efectividad real del Tiempo de Anticipación (TAE) frente a crisis operativas originadas orgánicamente por los usuarios, y probar la eficacia de las respuestas automatizadas dentro de un entorno *Software-Defined Networking* (SDN).

3. **Expansión longitudinal de la captura e integración de múltiples dominados de red.** Ejecutar ciclos de instrumentación por periodos mínimos de dos a seis meses ininterrumpidos en corporativos PYME, a fin de capturar la huella de los procesos estacionales de mantenimiento, respaldos semanales masivos o descargas de actualizaciones globales. Simultáneamente, escalar este acopio de *datasets* hacia infraestructuras ajenas para determinar cómo la dimensionalidad de las *features* predictivas debe adaptarse al transitar de una red aislada hacia otra de topología sustancialmente diferente.

4. **Validación orgánica del umbral de crisis y afinamiento de la etiqueta tensorial.** Recabar incidentes prolongados de saturación verídicos y documentados por *tickets* de soporte técnico que permitan sustituir los algoritmos generadores de tráfico sintético. Paralelamente, validar empíricamente que el umbral heurístico histórico del 80 % ($U_{th} = 0{,}80$) se correlacione irrefutablemente con la inflexión del RTT y la manifestación del *Jitter* o la pérdida en la capa de usuario. Con base en esto, reprogramar el tensor de clasificación binaria para que deje de evaluar el cruce instantáneo probabilístico del umbral, y exija la detección estricta de una saturación sostenida que amerite activar los protocolos de emergencia (QoS).

5. **Incorporación de dimensionalidad enriquecida y evaluación de algoritmos paralelos.** Integrar las métricas desechadas en este trabajo (e.g., retransmisiones TCP observadas a nivel capa 4, latencia agregada a servidores críticos o métricas codificadas temporalmente) una vez que la red demuestre un nivel de tráfico orgánico dinámico y no estático. De la mano de estos nuevos *datasets*, será imperioso someter a los tensores resultantes a una pugna de optimización intensiva contra las topologías SVR, GRU y modelos basados en Mecanismos de Atención Computacional (*Transformers*), apalancando librerías especializadas y clústeres de computación paralela a través de GPU que superen los márgenes reportados en esta tesis.

6. **Pruebas formales de estocasticismo y adaptación para estudios de portabilidad global.** Automatizar una batería de reentrenamientos tipo Montecarlo que aplique cientos de semillas generativas para reportar, sin asomo de duda estadística, los intervalos precisos de confianza y estabilidad matemática de cada modelo implementado. Por su parte, para la evaluación de generalización, se deberá estudiar la adaptación funcional de los modelos de ventana temporal prolongada (de 30 minutos) de modo que sus requerimientos computacionales de *padding* o ajuste emparejen satisfactoriamente con la brevedad estructural (trazas de 15 minutos) de repositorios estandarizados globales, como los provistos por The MAWI Working Group.

Bibliografía

[1] Cisco Systems, "Cisco annual internet report (2018–2023) white paper," Cisco, Tech. Rep., 2020, consultado el 25 de mayo de 2026. [Online]. Available: https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html

[2] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, p. 16, 2018.

[3] J. Zhang, Y. Liu, Z. Li, and Y. Lu, "Network traffic prediction using deep learning: An experimental study," in *2020 IEEE 6th International Conference on Computer and Communications (ICCC)*, 2020, pp. 1320–1325.

[4] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston: Pearson Prentice Hall, 2011.

[5] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Boston: Pearson, 2017.

[6] L. Kleinrock, *Queueing Systems, Volume II: Computer Applications*. New York: John Wiley & Sons, 1976.

[7] V. Jacobson, "Congestion avoidance and control," in *Symposium proceedings on Communications architectures and protocols (SIGCOMM '88)*, vol. 18, no. 4. Stanford, CA, USA: ACM, 1988, pp. 314–329.

[8] S. Floyd, "Congestion control principles," RFC 2914, Internet Engineering Task Force (IETF), BCP 41, September 2000. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2914

[9] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, "Rtp: A transport protocol for real-time applications," RFC 3550, Internet Engineering Task Force (IETF), July 2003. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3550

[10] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ: John Wiley & Sons, 2015.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734.

[14] H. Zare Moayedi and M. A. Masnadi-Shirazi, "Arima model for network traffic prediction and anomaly detection," in *2008 International Symposium on Information Technology (ITSim)*, vol. 4. IEEE, 2008, pp. 1–6.

[15] F. Tian, "Network traffic prediction method based on autoregressive integrated moving average and adaptive volterra filter," *International Journal of Communication Systems*, vol. 34, no. 12, p. e4891, 2021.

[16] I. Kochetkova, A. Kushchazli, S. Burtseva, and A. Gorshenin, "Short-term mobile network traffic forecasting using seasonal arima and holt-winters models," *Future Internet*, vol. 15, no. 9, p. 290, 2023.

[17] Y. V. Sneha, Vimitha, Vishwasini, S. Boloor, and N. D. Adesh, "Prediction of network congestion at router using machine learning technique," in *2020 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*. IEEE, 2020, pp. 188–193.

[18] Z. L. Qian, D. C. Juan, P. Bogdan, C. Y. Tsui, D. Marculescu, and R. Marculescu, "A support vector regression (svr)-based latency model for network-on-chip (noc) architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 3, pp. 471–484, 2016.

[19] M. Labonne, C. Chatzinakis, and A. Olivereau, "Predicting bandwidth utilization on network links using machine learning," arXiv:2112.02417, 2021. [Online]. Available: https://arxiv.org/abs/2112.02417

[20] T. P. Oliveira, J. S. Barbar, and A. S. Soares, "Computer network traffic prediction: A comparison between traditional and deep learning neural networks," *International Journal of Big Data Intelligence*, vol. 3, no. 1, pp. 28–37, 2016.

[21] S. Saha, A. Haque, and G. Sidebottom, "Overcoming data limitations in internet traffic forecasting: Lstm models with transfer learning and wavelet augmentation," arXiv:2409.13181, 2024. [Online]. Available: https://arxiv.org/abs/2409.13181

[22] G. L. Santos, P. Rosati, T. Lynn, J. Kelner, D. Sadok, and P. T. Endo, "Predicting short-term mobile internet traffic from internet activity using recurrent neural networks," *International Journal of Network Management*, vol. 32, no. 3, p. e2191, 2022.

[23] MAWI Working Group, "Mawi working group traffic archive," WIDE Project, consultado en 2026. [Online]. Available: https://mawi.wide.ad.jp/mawi/

[24] R. Hernández-Sampieri, *Metodología de la Investigación: Las Rutas Cuantitativa, Cualitativa y Mixta*, 2nd ed. Ciudad de México: McGraw-Hill, 2023.

[25] A. Boukerche, Y. Sun, and M. Z. A. Bhuiyan, "Performance evaluation of machine learning-based network traffic prediction approaches: A survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 1, pp. 405–428, 2021.

Apéndice A

Código fuente de ejemplos críticos

Este apéndice reproduce el código fuente de las piezas de software consideradas más relevantes para la comprensión y reproducibilidad del prototipo desarrollado en esta tesis. Se excluyen deliberadamente los scripts de configuración de entorno, corrección de errores y depuración que no aportan al entendimiento metodológico del trabajo. El repositorio completo del proyecto contiene la totalidad de los scripts empleados.

A.1. Decodificador de trazas libpcap

El siguiente fragmento corresponde al decodificador propio empleado para extraer telemetría de las trazas de MAWI (Sección 5.2), evitando la sobrecarga de memoria de herramientas de disección de protocolos de alto nivel. A diferencia del fragmento presentado en el Capítulo 5 (Código 5.1), aquí se incluye el manejo completo de las cuatro variantes del encabezado global de *libpcap* (orden de bytes y precisión temporal).

Listing A.1: Decodificador completo de trazas libpcap.

```
import struct
from collections import defaultdict

def parsear_pcap(pcap_path, ventana_seg=60):
    """
␣␣␣␣Lee␣un␣archivo␣.pcap␣directamente␣(formato␣clasico␣libpcap),␣extrayendo
␣␣␣␣timestamp␣y␣tamano␣original␣de␣cada␣paquete␣SIN␣decodificar␣protocolos.
␣␣␣␣Agrega␣el␣volumen␣de␣bytes␣transmitidos␣en␣ventanas␣temporales␣discretas.
␣␣␣␣"""
    acumulado = defaultdict(int)
    contador = 0

    with open(pcap_path, "rb") as f:
        # --- Encabezado global (24 bytes) ---
        cabecera_global = f.read(24)
        magic = struct.unpack("I", cabecera_global[0:4])[0]

```

```
        if magic == 0xa1b2c3d4:
            endian = "<"   # little-endian, microsegundos
            nano = False
        elif magic == 0xd4c3b2a1:
            endian = ">"   # big-endian, microsegundos
            nano = False
        elif magic == 0xa1b23c4d:
            endian = "<"   # little-endian, nanosegundos
            nano = True
        elif magic == 0x4d3cb2a1:
            endian = ">"   # big-endian, nanosegundos
            nano = True
        else:
            raise ValueError(f"Magic number no reconocido: {hex(magic)}")

        fmt_registro = endian + "IIII"  # ts_sec, ts_usec, incl_len, orig_len

        while True:
            encabezado_paquete = f.read(16)
            if len(encabezado_paquete) < 16:
                break  # fin del archivo

            ts_sec, ts_usec, incl_len, orig_len = struct.unpack(
                fmt_registro, encabezado_paquete
            )

            if nano:
                timestamp = ts_sec + (ts_usec / 1_000_000_000)
            else:
                timestamp = ts_sec + (ts_usec / 1_000_000)

            ventana = int(timestamp // ventana_seg)
            acumulado[ventana] += orig_len
            contador += 1

            f.seek(incl_len, 1)  # salta los bytes del paquete sin leerlos

    filas = [{"timestamp": v * ventana_seg, "throughput_bps": (b * 8) / ventana_seg}
             for v, b in sorted(acumulado.items())]
    return filas, contador
```

A.2. Script de recolección en RouterOS

El siguiente script, escrito en RouterOS Script y ejecutado periódicamente mediante el Programador de Tareas (*Scheduler*), constituye el mecanismo de recolección del dataset primario (Sección 4.4), adoptado como alternativa funcionalmente equivalente a SNMP ante las restricciones de acceso remoto descritas en la Sección 5.1.

Listing A.2: Script de recolección de telemetría en RouterOS.

```
:local wanInterface "ether1";
:local pingTarget "1.1.1.1";

:local traf [/interface monitor-traffic interface=$wanInterface once as-value];
:local rxbps ($traf->"rx-bits-per-second");
:local txbps ($traf->"tx-bits-per-second");

:local pingResult [/ping $pingTarget count=5 as-value];
:local recibidos [:len $pingResult];
:local perdida ((5 - $recibidos) * 100 / 5);

:local sumaUs 0;
:foreach r in=$pingResult do={
    :local timeStr [:tostr ($r->"time")];
    :local hh [:tonum [:pick $timeStr 0 2]];
    :local mm [:tonum [:pick $timeStr 3 5]];
    :local ss [:tonum [:pick $timeStr 6 8]];
    :local frac [:tonum [:pick $timeStr 9 15]];
    :local totalUs (((($hh*3600)+($mm*60)+$ss)*1000000 + $frac);
    :set sumaUs ($sumaUs + $totalUs);
};
:local rttUsProm 0;
:if ($recibidos > 0) do={ :set rttUsProm ($sumaUs / $recibidos) };

:local fecha [:tostr [/system clock get date]];
:local hora [:tostr [/system clock get time]];
:local linea ($fecha . "," . $hora . "," . $rxbps . "," . $txbps . "," . $rttUsProm . "," . $perdida);
:local nombreArchivo ("log_" . $fecha . ".csv");

:if ([:len [/file find name=$nombreArchivo]] = 0) do={
    /file add name=$nombreArchivo contents=("fecha,hora,rx_bps,tx_bps,rtt_us,perdida_pct\n" . $linea . "\n")
} else={
    :local actual [/file get [find name=$nombreArchivo] contents];
    /file set [find name=$nombreArchivo] contents=($actual . $linea . "\n")
}
```

Este script se programó mediante `/system scheduler add name=run-recolectar interval=1m on-event=recolectar-metrica`, ejecutándose de manera autónoma cada minuto durante todo el periodo de recolección, con respaldo local resiliente a interrupciones de la conectividad VPN (Tabla 5.1).

A.3. Pipeline de preprocesamiento

El siguiente script implementa las etapas de limpieza, detección de sesiones, generación de variables de rezago y definición de las variables objetivo de regresión y clasificación (Sección 5.2), aplicado de manera uniforme a los cinco conjuntos derivados del dataset primario.

Listing A.3: Pipeline de limpieza, detección de sesiones y generación de variables.

```
import pandas as pd
import numpy as np

BANDA_NOMINAL = 1_000_000_000
U_TH = 0.80
H = 5
UMBRAL_SESION_MIN = 5
N_REZAGOS = 5
VENTANA_MEDIA_MOVIL = 30

def procesar(path_entrada, nombre_salida, es_train=False):
    df = pd.read_csv(path_entrada)
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.sort_values("timestamp").reset_index(drop=True)
    df["utilizacion"] = df["rx_bps"] / BANDA_NOMINAL

    # --- Deteccion de sesiones (huecos grandes = frontera) ---
    diffs = df["timestamp"].diff().dt.total_seconds().fillna(0) / 60.0
    df["sesion_id"] = (diffs > UMBRAL_SESION_MIN).cumsum()

    # --- Limpieza por sesion: reindexar, interpolar huecos cortos ---
    sesiones_limpias = []
    for sid, grupo in df.groupby("sesion_id"):
        grupo = grupo.sort_values("timestamp")
        rango = pd.date_range(grupo["timestamp"].min(), grupo["timestamp"
            ].max(), freq="1min")
        g_completo = pd.DataFrame({"timestamp": rango})
        g_completo = g_completo.merge(
            grupo[["timestamp", "utilizacion", "rtt_us", "perdida_pct", "
                es_sintetico"]],
            on="timestamp", how="left"
        )
        g_completo["utilizacion"] = g_completo["utilizacion"].interpolate(
            method="linear", limit=3)
        g_completo["rtt_us"] = g_completo["rtt_us"].interpolate(method="
            linear", limit=3)
        g_completo["perdida_pct"] = g_completo["perdida_pct"].interpolate(
            method="linear", limit=3)
        g_completo["es_sintetico"] = g_completo["es_sintetico"].ffill().
            bfill()
        g_completo["sesion_id"] = sid
        sesiones_limpias.append(g_completo)

    df_limpio = pd.concat(sesiones_limpias, ignore_index=True)
    df_limpio = df_limpio.dropna(subset=["utilizacion"]).reset_index(drop=
        True)
```

```
40
41      # --- Rezagos y media movil, calculados POR SESION ---
42      for i in range(1, N_REZAGOS + 1):
43          df_limpio[f"U_lag{i}"] = df_limpio.groupby("sesion_id")["
                utilizacion"].shift(i)
44
45      df_limpio["U_media_movil"] = (
46          df_limpio.groupby("sesion_id")["utilizacion"]
47          .rolling(window=VENTANA_MEDIA_MOVIL, min_periods=
                VENTANA_MEDIA_MOVIL)
48          .mean()
49          .reset_index(level=0, drop=True)
50      )
51
52      # --- Variables objetivo: regresion U(t+h) y clasificacion U(t+h) >=
            U_th ---
53      df_limpio["target_regresion"] = df_limpio.groupby("sesion_id")["
            utilizacion"].shift(-H)
54      df_limpio["target_clasificacion"] = (df_limpio["target_regresion"] >=
            U_TH).astype(float)
55      df_limpio.loc[df_limpio["target_regresion"].isna(), "
            target_clasificacion"] = np.nan
56
57      columnas_clave = [f"U_lag{i}" for i in range(1, N_REZAGOS+1)] + ["
            U_media_movil", "target_regresion"]
58      df_final = df_limpio.dropna(subset=columnas_clave).reset_index(drop=
            True)
59
60      df_final.to_csv(nombre_salida, index=False)
61      return df_final
```

Listing A.4: Normalización Min-Max sin fuga de datos (ajustada exclusivamente con el conjunto de entrenamiento).

```
1  import pandas as pd
2
3  COLS_NORM = ["utilizacion", "U_lag1", "U_lag2", "U_lag3", "U_lag4", "
       U_lag5",
4               "U_media_movil", "target_regresion"]
5
6  def normalizar(train_df, path_entrada, nombre_salida, min_vals, max_vals):
7      df = pd.read_csv(path_entrada)
8      df_norm = df.copy()
9      for col in COLS_NORM:
10         rango = max_vals[col] - min_vals[col]
11         df_norm[col] = (df[col] - min_vals[col]) / rango if rango != 0
               else 0.0
12     df_norm.to_csv(nombre_salida, index=False)
13     return df_norm
14
15 train = pd.read_csv("train_procesado.csv")
16 min_vals = train[COLS_NORM].min()
17 max_vals = train[COLS_NORM].max()
```

Listing A.5: Construcción de secuencias mediante ventana deslizante, respetando fronteras de sesión.

```python
import pandas as pd
import numpy as np

def construir_ventanas(path_csv, w=30):
    df = pd.read_csv(path_csv)
    X, y_reg, y_clas = [], [], []
    for sid, grupo in df.groupby("sesion_id"):
        grupo = grupo.reset_index(drop=True)
        serie = grupo["utilizacion"].values
        target_reg = grupo["target_regresion"].values
        target_clas = grupo["target_clasificacion"].values
        n = len(serie)
        for i in range(n - w + 1):
            ventana = serie[i: i + w]
            idx_target = i + w - 1
            X.append(ventana)
            y_reg.append(target_reg[idx_target])
            y_clas.append(target_clas[idx_target])
    return np.array(X), np.array(y_reg), np.array(y_clas)
```

A.4. Generador de datos sintéticos

El siguiente script implementa la estrategia de aumento de datos descrita en la Sección 5.1: generación de tráfico normal mediante *bootstrap* horario de datos reales, e inyección de eventos de congestión con perfil trapezoidal cuyo RTT y pérdida de paquetes se derivan mediante el modelo de teoría de colas de la Sección 2.1.3.

Listing A.6: Generador de tráfico sintético calibrado con datos reales.

```python
import pandas as pd
import numpy as np

np.random.seed(42)
BANDA_NOMINAL = 1_000_000_000

def generar_dia_normal(fecha, pools_rx_por_hora, pools_tx_por_hora,
    pools_rtt_por_hora):
    """Genera 1440 minutos de trafico 'normal' por bootstrap horario de
        datos reales."""
    filas = []
    for minuto in range(1440):
        hora = minuto // 60
        rx = np.random.choice(pools_rx_por_hora[hora]) * np.random.normal
            (1.0, 0.05)
        tx = np.random.choice(pools_tx_por_hora[hora]) * np.random.normal
            (1.0, 0.05)
        rtt = np.random.choice(pools_rtt_por_hora[hora])
        ts = fecha + pd.Timedelta(minutes=minuto)
        filas.append({"timestamp": ts, "rx_bps": max(0, rx), "tx_bps": max
            (0, tx),
```

```
                        "rtt_us": rtt, "perdida_pct": 0})
    df_dia = pd.DataFrame(filas)
    return df_dia.astype({"rx_bps": "float64", "tx_bps": "float64",
                          "rtt_us": "float64", "perdida_pct": "float64"})

def perfil_rampa(n_puntos, u_pico):
    """Perfil trapezoidal: sube (20%) - se mantiene (60%) - baja (20%)."""
    n_subida = max(1, int(n_puntos * 0.2))
    n_bajada = max(1, int(n_puntos * 0.2))
    n_meseta = n_puntos - n_subida - n_bajada
    subida = np.linspace(0.10, u_pico, n_subida)
    meseta = np.full(n_meseta, u_pico) * np.random.normal(1.0, 0.03,
        n_meseta)
    bajada = np.linspace(u_pico, 0.10, n_bajada)
    return np.clip(np.concatenate([subida, meseta, bajada]), 0, 0.999)

def rtt_por_teoria_colas(utilizacion, rtt_base_us):
    """RTT(rho) = rtt_base / (1 - rho), consistente con Kleinrock (Seccion
        2.1.2)."""
    rho = np.clip(utilizacion, 0, 0.98)
    return rtt_base_us / (1 - rho)

def perdida_por_utilizacion(utilizacion):
    """Perdida de paquetes: 0% hasta ~0.85, luego crece cuadraticamente.""
        "
    return np.where(
        utilizacion < 0.85, 0,
        np.clip((utilizacion - 0.85) / 0.15, 0, 1) ** 2 * 15
    )

def inyectar_eventos_congestion(df_dia, n_eventos, umbral=0.80):
    """Inyecta eventos de congestion con perfil de rampa, duracion e
        intensidad variadas."""
    df_dia = df_dia.copy()
    n = len(df_dia)
    eventos_generados = []
    for _ in range(n_eventos):
        duracion = np.random.randint(5, 30)
        inicio = np.random.randint(0, n - duracion)
        u_pico = np.random.uniform(umbral + 0.01, min(0.98, umbral + 0.35)
            )
        perfil = perfil_rampa(duracion, u_pico)

        idx = df_dia.index[inicio: inicio + duracion]
        df_dia.loc[idx, "rx_bps"] = perfil * BANDA_NOMINAL
        df_dia.loc[idx, "tx_bps"] = perfil * BANDA_NOMINAL * np.random.
            uniform(0.85, 1.0, len(idx))
        df_dia.loc[idx, "rtt_us"] = rtt_por_teoria_colas(perfil,
            RTT_BASE_US)
        df_dia.loc[idx, "perdida_pct"] = perdida_por_utilizacion(perfil)

        eventos_generados.append({
            "inicio": df_dia.loc[idx[0], "timestamp"],
            "duracion_min": duracion,
```

```
            "u_pico": round(u_pico, 3)
        })
    return df_dia, eventos_generados
```

Listing A.7: Verificación automática de la ausencia de fuga de datos entre entrenamiento y los conjuntos de evaluación.

```
# Verificaciones de seguridad, ejecutadas tras ensamblar los conjuntos finales
ts_train = set(train_final["timestamp"])

assert len(set(val_regresion["timestamp"]) & ts_train) == 0, \
    "FUGA: val_regresion se solapa con train"
assert len(set(test_regresion["timestamp"]) & ts_train) == 0, \
    "FUGA: test_regresion se solapa con train"
assert len(set(val_clasificacion["timestamp"]) & ts_train) == 0, \
    "FUGA: val_clasificacion se solapa con train"
assert len(set(test_clasificacion["timestamp"]) & ts_train) == 0, \
    "FUGA: test_clasificacion se solapa con train"

assert val_regresion["es_sintetico"].sum() == 0, "ERROR: val_regresion contiene sintetico"
assert test_regresion["es_sintetico"].sum() == 0, "ERROR: test_regresion contiene sintetico"
assert val_clasificacion["es_sintetico"].sum() == len(val_clasificacion), \
    "ERROR: val_clasificacion contiene datos reales"
assert test_clasificacion["es_sintetico"].sum() == len(test_clasificacion), \
    "ERROR: test_clasificacion contiene datos reales"

assert (val_clasificacion["utilizacion"] >= U_TH).sum() > 0, \
    "ERROR: val_clasificacion no tiene casos positivos"
assert (test_clasificacion["utilizacion"] >= U_TH).sum() > 0, \
    "ERROR: test_clasificacion no tiene casos positivos"
```

A.5. Entrenamiento y evaluación de modelos

A continuación se presenta la implementación de cada uno de los tres modelos evaluados, incluyendo la función común de métricas empleada de manera consistente en los Capítulos 5 y 6.

Listing A.8: Función común de cálculo de métricas de clasificación (Accuracy, Precision, Recall, F1, AUC-ROC).

```
from sklearn.metrics import (roc_auc_score, precision_score, recall_score,
                             f1_score, accuracy_score, confusion_matrix)

def evaluar_clasificacion(y_real_cont, y_pred_cont, umbral):
    y_real_bin = (y_real_cont >= umbral).astype(int)
    y_pred_bin = (y_pred_cont >= umbral).astype(int)
    if y_real_bin.sum() == 0:
        return None
    return {
        "accuracy": accuracy_score(y_real_bin, y_pred_bin),
        "precision": precision_score(y_real_bin, y_pred_bin, zero_division
           =0),
        "recall": recall_score(y_real_bin, y_pred_bin, zero_division=0),
        "f1": f1_score(y_real_bin, y_pred_bin, zero_division=0),
        "auc": roc_auc_score(y_real_bin, y_pred_cont),
        "matriz_confusion": confusion_matrix(y_real_bin, y_pred_bin)
    }
```

Listing A.9: ARIMA: identificación de orden y pronóstico progresivo por bloques.

```
import numpy as np
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA

def identificar_orden(serie_train, d_sugerido, p_max=3, q_max=3):
    mejor_aic, mejor_orden = np.inf, None
    for p in range(p_max + 1):
        for q in range(q_max + 1):
            try:
                m = ARIMA(serie_train, order=(p, d_sugerido, q)).fit()
                if m.aic < mejor_aic:
                    mejor_aic, mejor_orden = m.aic, (p, d_sugerido, q)
            except Exception:
                continue
    return mejor_orden, mejor_aic

def pronostico_bloques(modelo_ajustado, serie, h=5):
    """Walk-forward por bloques de tamano h, revelando h valores reales
       por paso."""
    predicciones, reales = [], []
    modelo_actual = modelo_ajustado
    i = 0
    while i + h <= len(serie):
        pron = modelo_actual.get_forecast(steps=h).predicted_mean[-1]
        predicciones.append(pron)
```

```
        reales.append(serie[i + h - 1])
        modelo_actual = modelo_actual.append(serie[i:i+h], refit=False)
        i += h
    return np.array(predicciones), np.array(reales)

# Uso:
resultado_adf = adfuller(serie_train)
d = 0 if resultado_adf[1] <= 0.05 else 1
orden, aic = identificar_orden(serie_train, d)
modelo_final = ARIMA(serie_train, order=orden).fit()
```

Listing A.10: Random Forest: entrenamiento y análisis de importancia de variables.

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor

FEATURES = ["U_lag1", "U_lag2", "U_lag3", "U_lag4", "U_lag5", "
    U_media_movil"]

train = pd.read_csv("train_norm.csv")
X_train, y_train = train[FEATURES], train["target_regresion"]

modelo = RandomForestRegressor(n_estimators=200, max_depth=10,
    random_state=42, n_jobs=-1)
modelo.fit(X_train, y_train)

importancias = pd.Series(modelo.feature_importances_, index=FEATURES) \
                 .sort_values(ascending=False)

# Prediccion sobre cualquier conjunto de evaluacion:
def predecir(df_eval):
    return modelo.predict(df_eval[FEATURES])
```

Listing A.11: LSTM: arquitectura y entrenamiento con early stopping.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(42)
tf.random.set_seed(42)

datos = np.load("ventanas_primario.npz")
X_train = datos["X_train"].reshape((-1, datos["X_train"].shape[1], 1))
y_train = datos["y_train_reg"]
X_val = datos["X_val_reg"].reshape((-1, datos["X_val_reg"].shape[1], 1))
y_val = datos["y_val_reg"]

modelo = keras.Sequential([
    layers.Input(shape=(X_train.shape[1], 1)),
    layers.LSTM(32, activation="tanh"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1)
```

```
])
modelo.compile(optimizer="adam", loss="mse", metrics=["mae"])

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True
)

historial = modelo.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100, batch_size=64,
    callbacks=[early_stop], verbose=1
)
```